SkyControl: A Hybrid AI-Assisted Air Traffic Management System

Closing Communication, Separation and Emergency-Handling Gaps in Current ATC, Validated on Real OpenSky Traffic

A Project Report

submitted in partial fulfilment of the requirements of the course

by

HARSHIT CHAUDHARY

[REGISTER NUMBER]

GOPIKA K S

[REGISTER NUMBER]

Under the guidance of

Dr. Sivaranjini Nagarajan

School of Computer Science Engineering and Information Systems

VIT, Vellore

October, 2026

EXECUTIVE SUMMARY

Air traffic control (ATC) still depends on a controller listening to every voice readback, remembering every cleared level, noticing similar callsigns and coordinating emergencies by telephone, supported by safety nets that look only about two minutes ahead. SkyControl is a hybrid human-AI air traffic management laboratory that does not try to replace this system. It closes nine specific gaps between today's tools and the people who use them, and it keeps the controller in command through explicit *context switches* between four automation levels (AUTO, CONSENT, ADVISORY and MANUAL) whenever the automation might be wrong.

Every result is computed on a real 90-minute OpenSky Network recording of London-Amsterdam traffic (906 aircraft, 86,447 position reports after removing 708 duplicates), with emergencies injected into the real traffic and 21 step-by-step scenarios. The system is a complete web application: a FastAPI back end with a SQLite database, a login portal with verified (PBKDF2-hashed) credentials that rejects wrong credentials and stores every attempt with its reason, a three.js 3D replay of the real airspace, a scenario lab that shows each scenario being handled in ten steps, a communication assistant and a statistical analysis section. It is tested by 93 automated tests (90% statement coverage), packaged with Docker and delivered through a Jenkins pipeline that ran end to end; the work was planned in six Scrum sprints for Jira.

On held-out real traffic the learned trajectory predictor reduced the five-minute position error from 6.845 NM to 4.482 NM (34.5% lower, $p < 0.001$). The intent-aware conflict probe raised 72.2% fewer alerts than a naive probe with the same five-minute look-ahead, at the alert rate of a two-minute safety net. Dead-reckoned coasting cut the error of dropped tracks from 6.24 NM to 1.48 NM, and 21 duplicate (ghost) targets were found in the real data. The readback verifier caught 98.0% of injected readback errors under simulated speech-recognition noise; in the modelled sector, undetected communication errors fell from 40.55 to 3.59 per 10,000 clearances. For 60 emergencies injected into real traffic, SkyControl found 2.73 conflicts with real aircraft per case and resolved 91.5% of them with verified level changes in about 90 ms. Across the 21 scenarios all 34 aircraft conflicts, runway

conflicts, hazard breaches and fuel-deadline misses of the comparator were removed, and the modelled time to issue all instructions fell from 117.1 s to 38.7 s.

The results are labelled honestly as *measured on real data*, *modelled* (with sensitivity analyses) or *scenario*. SkyControl complements STCA, MTCD, CLAM, AMAN and CPDLC rather than beating them: it adds a second check behind the controller's hearback, a longer and quieter conflict look-ahead, an integrity layer for surveillance data, earlier hold prediction and an emergency assistant, all governed by an automation policy that knows when to step back. It is an academic simulation, not certified ATC software.

TABLE OF CONTENTS

CONTENTS **Page No.**

Executive Summary ..... **ii**

Table of Contents ..... **iv**

List of Figures ..... **vii**

List of Tables ..... **ix**

Abbreviations ..... **x**

Symbols and Notations ..... **xi**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 3

1.3 Objective 4

1.4 Problem Statement 4

1.5 Scope of the Work 5

1.6 Contributions 5

1.7 Organisation of the Report 6

2. LITERATURE SURVEY AND GAP ANALYSIS **7**

2.1 How Air Traffic Control Works Today 7

2.2 Pilot-Controller Communication and Human Error 7

2.3 Conflict Detection and Safety Nets 8

2.4 Trajectory Prediction with Machine Learning 9

2.5 Surveillance Data: ADS-B and the OpenSky Network 9

2.6 Arrival Management and Airborne Holding 9

2.7 Anomaly Detection in Flight Data 10

2.8 Levels of Automation and Human-Automation Teaming 10

2.9 Summary of the Literature 11

2.10 Gap Analysis of Current ATC 11

3. TECHNICAL SPECIFICATION **14**

3.1 Functional Requirements 14

3.2 Non-Functional Requirements 15

3.3 Dataset Specification 15

3.4 Hardware and Software Requirements 17

3.5 Technology Stack 17

3.6 Security Specification 17

3.7 Interface Specification 18

3.8 Acceptance Criteria 19

4. DESIGN APPROACH AND DETAILS **20**

4.1 Design Approach....20
4.2 System Architecture....21
4.3 UML Design....23
4.4 Materials and Methods....29
4.5 Codes and Standards....38
4.6 Constraints, Alternatives and Trade-offs....38
5. IMPLEMENTATION....40
5.1 Project Structure....40
5.2 Back End....40
5.3 Login Portal and Attempt Storage....41
5.4 Analytics Pipeline....42
5.5 Scenario Lab Engine....43
5.6 Three-Dimensional Front End....44
5.7 Communication Assistant....44
5.8 Testing....44
5.9 Containerisation....45
5.10 Continuous Integration and Delivery....46
6. SCHEDULE, TASKS AND MILESTONES....47
6.1 Agile Methodology....47
6.2 Product Backlog and Epics....47
6.3 Sprint Plan....48
6.4 Sprint Backlogs....49
6.5 Jira Configuration....50
6.6 Version Control with GitHub....51
6.7 Jenkins Pipeline....51
6.8 Deployment with Docker....52
6.9 Milestones....53
7. RESULTS AND DISCUSSION....54
7.1 Evaluation Design....54
7.2 Login Portal and Security....55
7.3 Trajectory Prediction (G3)....56
7.4 Conflict Alerting (G3)....57
7.5 Surveillance Integrity (G4)....59
7.6 Communication Safety (G1, G2)....60
7.7 Holding and Hold Prediction (G5)....63
7.8 Detecting Emergency-Like Behaviour and Level Busts (G7)....64
7.9 Emergencies Injected into Real Traffic (G6)....65
7.10 Scenario Lab Results....68
7.11 Summary of Statistical Results....76
7.12 How SkyControl Complements Current ATC....76

7.13 Software Testing and Performance....77
7.14 User Interface....78
7.15 Cost Analysis....88
7.16 Discussion and Threats to Validity....89
8. SUMMARY....91
8.1 Findings....91
8.2 Conclusions....92
8.3 Recommendations and Future Work....92
9. REFERENCES....94
APPENDIX A SAMPLE CODE, USER MANUAL AND ADDITIONAL SCREENSHOTS....−
A.1 User Manual and Demonstration Script....−
A.2 Context-Switch Policy (backend/core/automation.py)....−
A.3 Exact Conflict Verification (backend/core/separation.py)....−
A.4 Container Smoke Test (scripts/smoke_test.sh)....−
A.5 Docker Compose File....−
A.6 Additional Screenshots....−

LIST OF FIGURES

Fig. 1.1 Real London-Amsterdam Traffic Replayed in the SkyControl 3D Airspace.....2
Fig. 2.1 ATC Gaps and Fixes Page of the Application.....13
Fig. 3.1 Aircraft in View During the 90-Minute Recording.....16
Fig. 3.2 Position Reports by Altitude Band and the Most Frequent Operators.....16
Fig. 4.1 Component Architecture of SkyControl.....21
Fig. 4.2 Data Flow from the OpenSky Recording to the User Interface.....22
Fig. 4.3 Deployment Diagram.....22
Fig. 4.4 Use Case Diagram.....24
Fig. 4.5 Class Diagram of the Main Components.....25
Fig. 4.6 Sequence Diagram of the Login Portal.....26
Fig. 4.7 Sequence Diagram of Emergency Injection on Real Traffic.....26
Fig. 4.8 Activity Diagram of the Ten-Step Handling Pipeline.....27
Fig. 4.9 State Diagram of the Automation Levels (Context Switches).....28
Fig. 4.10 Entity-Relationship Diagram of the Main Tables.....29
Fig. 5.1 Login Portal Rejecting a Wrong Password and Showing the Attempts Remaining. .42
Fig. 6.1 Project Timeline of the Six Sprints.....48
Fig. 6.2 Velocity per Sprint and Release Burndown (from the Sprint Plan).....49
Fig. 7.1 Login Attempt Log with Outcome, Reason, IP Address and Browser.....55
Fig. 7.2 Prediction Error by Horizon: Straight Line versus Learned Predictor.....56
Fig. 7.3 Five-Minute Prediction Error by Altitude Band.....57
Fig. 7.4 Permutation Importance of the Predictor Features (Five-Minute Horizon).....57
Fig. 7.5 Alert Load and Warning Time of the Four Alerting Methods.....58
Fig. 7.6 Gaps Between Position Reports in the Real Recording.....59
Fig. 7.7 Position Error at Reappearance: Frozen Track versus Dead-Reckoned Coast.....60
Fig. 7.8 Share of Readbacks Flagged by the Verifier, by Error Type.....61
Fig. 7.9 Readback Error Scenario: the Verifier Catches FL140 Instead of FL120.....61
Fig. 7.10 Sensitivity of the Communication Model and the Datalink Trade-off.....62
Fig. 7.11 Aircraft Holding per Stack During the Recording.....63
Fig. 7.12 Total Holding Time per Stack.....63
Fig. 7.13 Hold Predictor: ROC Curve on Held-Out Arrivals and Model Coefficients.....64
Fig. 7.14 Detection of Injected Emergencies by the Four Detectors.....65
Fig. 7.15 Emergencies Injected into Real Traffic: Handling Time and Conflicts Resolved…66
Fig. 7.16 Selecting a Real Aircraft in the 3D Airspace to Inject an Emergency.....66
Fig. 7.17 Hybrid Emergency Response for a Fuel Emergency on a Real Aircraft.....67
Fig. 7.18 Problems Left by the Comparator and by SkyControl, and Automation Levels Chosen.....69
Fig. 7.19 Modelled Time to Issue All Instructions in Each Scenario.....69
Fig. 7.20 Fuel Emergency Paused at the Approval Gate for a Controller.....70
Fig. 7.21 Fuel Emergency: the Ten Handling Steps after Supervisor Approval.....71
Fig. 7.22 Fuel Emergency: Conventional and SkyControl Workflows (Modelled).....71

Fig. 7.23 Readback Error Scenario: Handling Steps....72
Fig. 7.24 Readback Error Scenario: Outcome Checks....73
Fig. 7.25 Converging Traffic Scenario in the 3D Scenario Lab....74
Fig. 7.26 Converging Traffic: Outcome Checks Before and After....74
Fig. 7.27 Ghost Target: Context Switches Logged During the Event....75
Fig. 7.28 AI Disagreement: Context Switch from CONSENT to MANUAL....75
Fig. 7.29 AI Disagreement: Trusting the Learned Model versus the Conservative Plan....76
Fig. 7.30 Median and 95th-Percentile Latency under Load (8 Concurrent Clients)....78
Fig. 7.31 Command Centre (Dashboard)....79
Fig. 7.32 Scenario Lab Catalogue of 21 Scenarios....80
Fig. 7.33 Statistical Analysis: Summary Tab....81
Fig. 7.34 Statistical Analysis: Conflict Alerting Tab....81
Fig. 7.35 Statistical Analysis: Surveillance Integrity Tab....82
Fig. 7.36 Statistical Analysis: Communication Tab....82
Fig. 7.37 Statistical Analysis: Holding Tab....83
Fig. 7.38 Statistical Analysis: Emergencies Tab....83
Fig. 7.39 Statistical Analysis: Scenarios Tab....84
Fig. 7.40 AI Models and Automation Page with the Context-Switch Simulator....84
Fig. 7.41 Communication Assistant Verifying a Readback....85
Fig. 7.42 Operations: Flight Management....85
Fig. 7.43 Operations: Runway Management....86
Fig. 7.44 Operations: Runway Scheduling Calendar....86
Fig. 7.45 Operations: Priority Queue....87
Fig. 7.46 Data and Provenance Page....87
Fig. 7.47 Security and Audit Page....88
Fig. A.1 Readback Verification Result in the Communication Assistant....–
Fig. A.2 Context-Switch Simulator on the AI Models Page....–
Fig. A.3 Callsign Confusion Scenario: Wrong Aircraft Detected....–
Fig. A.4 Live Monitor of the 3D Airspace at One Moment of the Recording....–
Fig. A.5 Alert Centre....–
Fig. A.6 Statistical Analysis: Dataset Tab....–

LIST OF TABLES

Table 1.1 Course Requirements and Where They Are Met....6
Table 2.1 Summary of the Literature Reviewed....11
Table 2.2 Gaps in Current ATC and the SkyControl Fixes....12
Table 3.1 Functional Requirements....14
Table 3.2 Non-Functional Requirements....15
Table 3.3 Specification of the Real Dataset....15
Table 3.4 Hardware and Software Requirements....17
Table 3.5 Technology Stack....17
Table 3.6 Security Controls of the Login Portal and API....18
Table 3.7 Main API Endpoint Groups....19
Table 4.1 Readback Verifier Verdicts....34
Table 4.2 Callsign Similarity Rules and Scores....35
Table 4.3 Context-Switch Policy (Evaluated Top to Bottom)....36
Table 4.4 Codes and Standards Followed....38
Table 4.5 Design Alternatives and Trade-offs....39
Table 5.1 The Ten Steps Used to Handle Every Scenario....43
Table 5.2 Pages of the Application....44
Table 6.1 Scrum Roles and Ceremonies....47
Table 6.2 Epics of the Product Backlog....48
Table 6.3 Sprint Plan....48
Table 6.4 User Stories by Sprint....49
Table 6.5 Git Workflow Conventions....51
Table 6.6 Stages of the Jenkins Pipeline....52
Table 6.7 Project Milestones....53
Table 7.1 Evaluation Plan per Gap....54
Table 7.2 Mean Position Error of Straight-Line and Learned Prediction on Held-Out Traffic56
Table 7.3 Alerting Methods on the Real Recording....58
Table 7.4 Examples of Duplicate (Ghost) Targets Found in the Real Recording....59
Table 7.5 Readback Verifier Results by Error Type....60
Table 7.6 Modelled Communication Outcomes per Busy Sector-Hour....62
Table 7.7 Similar Callsigns Sharing Airspace in the Real Recording....63
Table 7.8 Confusion Matrix of the Hold Predictor on the Test Period (Threshold 0.5)....64
Table 7.9 Detection of Injected Emergencies (Detected of 27 per Kind)....65
Table 7.10 Scenario Benchmark: Comparator versus SkyControl....68
Table 7.11 Paired Comparisons: Current Practice versus SkyControl....76
Table 7.12 Current ATC and SkyControl Side by Side....77
Table 7.13 Automated Test Suite....77
Table 7.14 Development and Operating Cost (Indicative)....88

ABBREVIATIONS

Abbreviation	Expansion
ADS-B	Automatic Dependent Surveillance-Broadcast
AI	Artificial Intelligence
AMAN	Arrival Manager
API	Application Programming Interface
ASR	Automatic Speech Recognition
ATC	Air Traffic Control
ATM	Air Traffic Management
AUC	Area Under the (ROC) Curve
CI	Confidence Interval / Continuous Integration
CLAM	Cleared Level Adherence Monitoring
CPDLC	Controller-Pilot Data Link Communications
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
EGLL	ICAO code of London Heathrow airport
FL	Flight Level (hundreds of feet)
ICAO	International Civil Aviation Organization
JSON	JavaScript Object Notation
MTCD	Medium-Term Conflict Detection
NM	Nautical Mile (1,852 m)
PBKDF2	Password-Based Key Derivation Function 2
R/T	Radiotelephony
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SPA	Single-Page Application
STAR	Standard Terminal Arrival Route
STCA	Short-Term Conflict Alert
TCAS	Traffic alert and Collision Avoidance System
TMA	Terminal Control Area
UML	Unified Modeling Language
UTC	Coordinated Universal Time
WAL	Write-Ahead Logging (SQLite journal mode)
XMAN	Cross-border Arrival Management

SYMBOLS AND NOTATIONS

Symbol	Meaning
x, y	Local east and north coordinates (NM) in the projection centred on 52°N, 0.5°E
z	Barometric altitude (ft)
φ, λ	Latitude and longitude (radians)
R	Earth radius, 3,440.065 NM
v	Velocity vector (NM/s); |v| ground speed
h	Prediction horizon (s)
p(t)	Position at time t; p̂ a predicted position
$\mathbf{S_h}$**,** $\mathbf{S_v}$	Horizontal and vertical separation thresholds (NM, ft)
d(t)	Horizontal distance between two aircraft at time t
$\mathbf{E_h}$	Mean position error at horizon h (NM)
ψ	Track angle (degrees)
σ(·)	Logistic function 1/(1+e^−u)
β	Logistic-regression coefficients
s(x, n)	Isolation-forest anomaly score
c(n)	Average path length of an unsuccessful search in a binary tree
$\mathbf{p_{rb}}$	Probability of a voice readback error
$\mathbf{p_{miss}}$	Probability the controller misses a wrong readback
δ	Share of routine clearances sent by datalink
D	Verifier detection rate
T	Time to issue all instructions (s)
W	Wilcoxon signed-rank statistic
α	Significance level (0.05)
L	Automation level: AUTO, CONSENT, ADVISORY or MANUAL

CHAPTER 1
INTRODUCTION

Air traffic control (ATC) is the service that keeps aircraft safely separated from each other and from terrain while moving them efficiently between airports. A controller works a sector of airspace, watches a surveillance display, issues clearances by voice and listens to the pilot read them back. The system is extraordinarily safe, but it is also built around one scarce resource: the attention of the controller on a busy frequency. Most of its known weaknesses, from misheard levels and confused callsigns to late conflict alerts and improvised emergency handling, appear where that attention is overloaded or where a tool gives the controller information that is incomplete or wrong.

This project builds SkyControl, a hybrid human-AI air traffic management laboratory. It asks a practical question: which gaps in current ATC can artificial intelligence and well-known algorithms close today, without taking authority away from the controller, and how much do they help when measured on real traffic? The answer is delivered as a complete, deployable web application with a verified login portal, a 3D replay of real recorded traffic, a scenario lab, a communication assistant and a statistical analysis section, together with a reproducible evaluation.

1.1 Background

Modern ATC combines several layers. Surveillance comes from primary and secondary radar and, increasingly, from Automatic Dependent Surveillance-Broadcast (ADS-B), in which each aircraft broadcasts its own satellite-derived position, altitude, velocity and 24-bit ICAO address about twice per second. Controllers in a sector receive these tracks on a situation display. They communicate with pilots by radiotelephony (R/T) on a shared VHF frequency using the standard phraseology of ICAO Doc 4444 and Doc 9432, and the pilot reads back safety-related items such as levels, headings, speeds and frequencies so that the controller can hear back and correct any error (ICAO, 2016).

Ground-based safety nets sit behind the controller. Short-Term Conflict Alert (STCA) extrapolates current tracks for roughly two minutes and warns when two aircraft are predicted to lose separation. Medium-Term Conflict Detection (MTCD)

uses flight-plan intent to look further ahead where it is deployed, Cleared Level Adherence Monitoring (CLAM) compares the aircraft's altitude with the level entered by the controller, and arrival managers (AMAN and the cross-border XMAN) compute landing sequences. Airborne, the Traffic alert and Collision Avoidance System (TCAS) is the last line of defence. Controller-Pilot Data Link Communications (CPDLC) replaces some routine voice clearances with text messages, but only for equipped aircraft and mostly en route.

Traffic around London is among the densest in the world. The recording used in this project, taken from the OpenSky Network on 2026-10-15 between 15:42 and 17:12 UTC over the region 49.5°N to 54.5°N and 2°W to 3°E, contains 906 different aircraft, an average of 244.9 aircraft in view at any moment and a peak of 279. It covers Heathrow, Gatwick, Stansted, Luton, London City and Amsterdam Schiphol and the holding stacks that feed them. Such a recording makes it possible to test assistance functions on real, messy data instead of idealised simulations: real tracks turn, climb, level off, hold and occasionally disappear or appear twice.

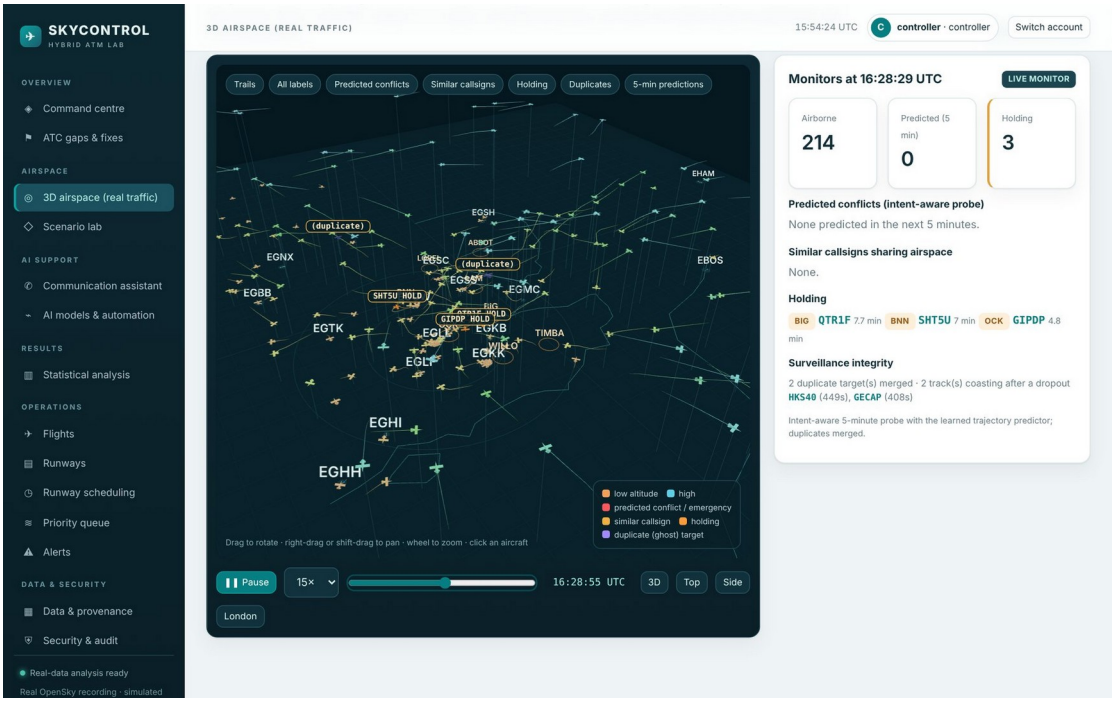


Fig. 1.1 Real London-Amsterdam Traffic Replayed in the SkyControl 3D Airspace

Fig. 1.1 shows the recording replayed in SkyControl at about 16:28 UTC: more than 200 aircraft are airborne at once, aircraft in the holding stacks around Heathrow are marked, a predicted conflict is highlighted in red and a pair of similar callsigns (HLE98 and HLE99) sharing the airspace is flagged. These are the kinds of

situation in which the controller's attention is stretched and in which an assistant can help.

1.2 Motivation

Voice communication remains the single largest source of operational error in ATC. Analyses of controller-pilot transcripts found readback errors in the order of one per cent of transmissions, a substantial share of which the controller did not catch (Cardosi, 1993; Cardosi, Falzarano, & Han, 1998). Similar callsigns on the same frequency lead the wrong aircraft to take an instruction; EUROCONTROL's air-ground communication safety study ranked call sign confusion among the leading communication hazards (EUROCONTROL, 2004). These are human-communication errors: they do not come from a lack of skill but from the limits of listening to a busy, shared voice channel.

A second motivation is the look-ahead of conflict alerting. A two-minute safety net is essential, but in a dense terminal area any straight-line probe that looks further ahead produces hundreds of nuisance alerts per hour, because aircraft turn onto arrival routes and level off at cleared levels that the probe does not know. A third is emergency handling: a fuel, medical or engine emergency is managed from memory and checklists, alternates are chosen mentally and coordination happens by telephone while the rest of the traffic keeps calling. Finally, the surveillance picture itself can be wrong. ADS-B data contains duplicate targets created by corrupted addresses and gaps in coverage, and a frozen track can look perfectly valid on a display (Strohmeier, Lenders, & Martinovic, 2015).

Artificial intelligence can help with each of these, but only if it is honest about when it can be trusted. Research on human-automation interaction warns that automation which is either fully on or fully off invites complacency and automation bias (Bainbridge, 1983; Parasuraman & Manzey, 2010). The motivation of this project is therefore not to build an autonomous controller, but to build a hybrid in which the machine handles routine work, proposes verified plans for the rest, and hands control back to the human, explicitly and with a logged reason, in critical, uncertain or degraded situations.

1.3 Objective

The objectives of the project, derived from the problem statement and the course requirements, are:

1. Collect a real, relevant dataset of air traffic (a recorded OpenSky Network capture of London-Amsterdam traffic) and document its provenance and limitations.
2. Implement the system end to end, including a login portal with verified credentials that clearly rejects wrong credentials and stores every login attempt, successful or not, in the back-end database.
3. Cover the main ATC scenarios (converging traffic, emergencies, low fuel, runway incursion, weather, readback error, similar callsigns, surveillance faults and more) with verified algorithms, and use AI with a manual context switch in critical situations where the AI may be wrong.
4. Find concrete gaps in current ATC, design a fix for each, and measure on real traffic and simulated emergencies whether the hybrid improves reliability and efficiency and reduces miscommunication and human error compared with current practice.
5. Where a first result was not significant, change the design (context switching, pattern learning, prediction from historical trends) and measure again.
6. Deliver a demonstrable 3D web application with flight animation, a step-by-step scenario lab, a statistical results section and a simulation page, and deliver it with professional engineering practice: GitHub, Jira sprints, a Jenkins pipeline and Docker deployment.

1.4 Problem Statement

Current ATC relies on the controller to detect and correct voice communication errors, to notice similar callsigns, to anticipate conflicts beyond the two-minute horizon of the safety net, to trust or distrust the surveillance picture, to decide on holding late, and to coordinate emergencies manually. Existing tools address parts of these tasks but leave gaps between them. The problem addressed in this project is to identify these gaps on real traffic, to design a hybrid human-AI system that closes them while keeping the human in authority, and to show with

statistical evidence, honestly labelled, where the hybrid improves on or complements current practice.

1.5 Scope of the Work

The work covers the following, and deliberately excludes certification and operational use:

- **In scope:** a research and teaching laboratory, the real 90-minute recording, nine gaps (G1 to G9), 21 scenarios, emergency injection into real traffic, learned models trained and tested on the recording, a Monte Carlo communication model, a full web application with authentication, a test suite, a container image, a CI/CD pipeline, a Scrum plan and this report.
- **Out of scope:** live operational use, integration with real ATC systems, certification (for example to EUROCONTROL or DGCA software assurance levels), real speech recognition of R/T audio, and real controller trials.
- **Data limits:** ADS-B carries no fuel, emergency or clearance information, so fuel states, emergencies and cleared levels are simulated or approximated and labelled as such.

1.6 Contributions

The main contributions of the project are:

- A gap catalogue of current ATC (G1 to G9), each with today's practice, the risk, the SkyControl fix, its relationship to existing tools and the evidence type.
- A learned trajectory predictor and an intent-aware five-minute conflict probe evaluated on held-out real traffic.
- A surveillance-integrity monitor that finds duplicate (ghost) targets and coasts dropped tracks.
- A readback verifier, a callsign-similarity monitor and a communication model with a sensitivity analysis.
- A holding analysis with a learned hold predictor, an anomaly detector and a cleared-level monitor, and an emergency assistant evaluated by injecting emergencies into real traffic.

- An adaptive automation policy with four levels and explicit, logged context switches, used in every scenario.
- A complete, tested and containerised web application with a secure login portal and a CI/CD pipeline.

1.7 Organisation of the Report

Chapter 2 surveys the literature and derives the gap analysis. Chapter 3 gives the technical specification, including the dataset. Chapter 4 presents the design approach, the architecture, the UML diagrams, the algorithms and equations, the codes and standards followed and the trade-offs made. Chapter 5 describes the implementation. Chapter 6 covers the schedule, the Scrum sprints in Jira and the GitHub, Jenkins and Docker workflow. Chapter 7 presents the results, statistical tests, screenshots and cost analysis, and Chapter 8 summarises the work. Chapter 9 lists the references in APA format, and Appendix A contains sample code and the user manual.

Table 1.1 Course Requirements and Where They Are Met

Requirement	How SkyControl meets it	Section
Real, relevant dataset	Recorded OpenSky capture, 906 aircraft, 86,447 reports, provenance and SHA-256 stored	3.3, 7.2
Login with verified credentials; wrong attempts stored	PBKDF2 hashes, uniform error message, every attempt stored with reason, IP and browser, lockout	3.6, 5.3, 7.3
All ATC scenarios with verified algorithms	21 scenarios, exact conflict verification of every plan, ten-step pipeline	4.4, 5.5, 7.10
AI with manual context switch	Four automation levels chosen by an explicit policy, supervisor approval for critical events	4.4.12, 7.10
Real traffic plus simulated emergencies; effectiveness vs ATC	Learned models on held-out traffic, 60 emergencies injected, paired statistical tests	7.4 to 7.9, 7.11
Improve if not significant	Intent-aware probe, duplicate merging, hold prediction and CLAM added after first results	4.6, 7.4, 7.8
3D web app, animation, results and simulation pages	three.js airspace replay and scenario lab, eight-tab statistics page	5.6, 7.15
GitHub, Jira, Jenkins, Docker	Guides, CSV backlog, validated Jenkinsfile run, multi-stage Dockerfile	Chapter 6

CHAPTER 2
LITERATURE SURVEY AND GAP ANALYSIS

This chapter reviews the literature that underpins SkyControl: how ATC works and where it fails, conflict detection, machine-learning trajectory prediction, ADS-B surveillance and its weaknesses, arrival management, anomaly detection, and the theory of levels of automation. It ends with a gap analysis that turns these findings into nine concrete gaps that the system addresses.

2.1 How Air Traffic Control Works Today

ICAO Annex 11 defines the objectives of air traffic services as preventing collisions between aircraft, expediting and maintaining an orderly flow of traffic and providing information and alerting (ICAO, 2018). In controlled airspace these objectives are met by sector controllers applying separation minima, typically 3 NM or 5 NM horizontally depending on the surveillance environment and 1,000 ft vertically, using the procedures of ICAO Doc 4444 (ICAO, 2016). A clearance is valid only after the pilot has read it back correctly; the controller is responsible for listening to the readback and correcting it, a step known as hearback.

Automation has grown around this core rather than replacing it. Safety nets such as STCA (EUROCONTROL, 2007) and TCAS (Kochenderfer, Holland, & Chryssanthacopoulos, 2012) act when separation is about to be lost. Planning tools such as MTCD and AMAN support the controller further ahead. The SESAR programme in Europe and NextGen in the United States both describe a future in which trajectory-based operations, datalink and higher levels of automation support higher capacity, but they keep the human controller responsible for separation (SESAR Joint Undertaking, 2020).

2.2 Pilot-Controller Communication and Human Error

Cardosi (1993) analysed thousands of en-route controller-pilot transmissions and found that readback errors occurred in under one per cent of transmissions but were more frequent in complex clearances, and that controllers failed to catch a meaningful share of them. A later analysis of Aviation Safety Reporting System reports showed that most communication incidents involved altitude clearances, that

similar callsigns were a frequent factor and that hearback errors were common under high workload (Cardosi, Falzarano, & Han, 1998). Prinzo and Britton (1993) surveyed the literature on voice communications and identified message length, non-standard phraseology and frequency congestion as the main contributors.

EUROCONTROL's air-ground communication safety study found that call sign confusion, readback/hearback errors and blocked transmissions were the dominant communication hazards in European airspace (EUROCONTROL, 2004). Datalink is the accepted mitigation for routine clearances: CPDLC messages are unambiguous and are addressed to a logged-on aircraft rather than to a spoken callsign. However, CPDLC response times of tens of seconds mean that urgent instructions must remain on voice. Automatic speech recognition (ASR) of controller speech has been shown to reduce controller workload when it pre-fills system inputs (Helmke, Ohneiser, Mühlhausen, & Wies, 2016), which suggests that a recogniser can also be used to check readbacks automatically.

2.3 Conflict Detection and Safety Nets

Kuchar and Yang (2000) classified conflict detection and resolution methods by how they propagate the future (nominal, worst-case or probabilistic), by detection threshold and by resolution method (geometric, optimised, force-field or manual). Nominal straight-line propagation, as used by STCA, is simple and transparent but produces false alerts when aircraft turn or level off, while worst-case propagation produces many more. Probabilistic and intent-based methods reduce nuisance alerts but need information about the aircraft's intended trajectory. Erzberger (2006) showed that automated resolution in an advisory role can solve most conflicts with a single manoeuvre when trajectories are known accurately.

STCA specifications deliberately use short look-ahead times, around two minutes, to keep nuisance alerts acceptable (EUROCONTROL, 2007). The consequence is that the safety net warns late by design; any probe that looks further ahead must be more accurate than straight-line extrapolation or it will overwhelm the controller with alerts. This trade-off between look-ahead and alert rate is the central design problem for gap G3 in this project.

2.4 Trajectory Prediction with Machine Learning

Trajectory prediction is the basis of every conflict probe. Physics-based predictors use aircraft performance models such as OpenAP (Sun, Hoekstra, & Ellerbroek, 2020) together with intent. Data-driven predictors learn from historical trajectories: Ayhan and Samet (2016) used hidden Markov models on gridded airspace, and deep-learning approaches with weather inputs have improved long-horizon predictions (Pang, Zhao, Yan, & Liu, 2021). Gradient-boosted decision trees (Friedman, 2001), in efficient histogram-based implementations (Ke et al., 2017; Pedregosa et al., 2011), are strong learners for tabular features and are fast enough to run on every aircraft at every radar update.

A common weakness of reported results is evaluation on the same period that was used for training, which overstates accuracy. SkyControl therefore learns a residual correction to straight-line extrapolation (so that it degrades gracefully to the baseline) and evaluates it only on a later, held-out part of the recording.

2.5 Surveillance Data: ADS-B and the OpenSky Network

The OpenSky Network is a crowd-sourced network of ADS-B and Mode S receivers that makes air traffic data available for research (Schäfer, Strohmeier, Lenders, Martinovic, & Wilhelm, 2014). State vectors contain the ICAO 24-bit address, callsign, position, barometric and geometric altitude, velocity, track and vertical rate. Tools such as the traffic library (Olive, 2019) and the BlueSky open-source ATC simulator (Hoekstra & Ellerbroek, 2016) have made such data a standard basis for ATM research.

ADS-B has known integrity weaknesses. Messages are unauthenticated, corrupted address bits produce duplicate tracks, receivers have coverage gaps and the reported position can be stale (Strohmeier et al., 2015). Operational trackers fuse several sources to mitigate this, but a display generally shows whatever the tracker outputs without telling the controller how far to trust it. This motivates gap G4.

2.6 Arrival Management and Airborne Holding

London's major airports absorb arrival delay in four holding stacks around Heathrow (BNN, LAM, OCK and BIG) and further stacks for Gatwick, Stansted and Luton. Holding at low level burns fuel at a high rate per minute and adds workload.

Arrival managers and cross-border arrival management (XMAN) aim to absorb part of this delay en route with small speed reductions, an approach often called linear holding (SESAR Joint Undertaking, 2020). The benefit depends on knowing early which arrivals will have to hold, which is a prediction problem that historical traffic can inform.

2.7 Anomaly Detection in Flight Data

Emergencies are rare and are not labelled in ADS-B data, so supervised learning is not possible. Unsupervised anomaly detection learns what normal flight looks like and flags departures from it. Isolation Forest isolates anomalies with random partitions and needs no distance metric, which makes it fast and robust on mixed features (Liu, Ting, & Zhou, 2008). Autoencoders have been used on Mode S data to detect controller actions and unusual trajectories (Olive, Grignard, Dubot, & Saint-Lot, 2018). Rule-based monitoring such as CLAM remains the most reliable detector of level busts, but only when the cleared level is known to the system.

2.8 Levels of Automation and Human-Automation Teaming

Parasuraman, Sheridan and Wickens (2000) proposed that automation should be designed at different levels for the four stages of information acquisition, analysis, decision selection and action implementation, and that the level should depend on the consequences of automation errors. Bainbridge (1983) described the ironies of automation: the more reliable a system, the less prepared the human is to take over when it fails. Parasuraman and Manzey (2010) reviewed evidence of complacency and automation bias, in which operators follow confident but wrong advice. Endsley (1995) linked performance to situation awareness, which automation can either support or erode.

These findings lead directly to the design principle of SkyControl: the level of automation is not fixed but chosen by an explicit policy for each situation, based on severity, confidence, data quality, disagreement between learned and rule-based components, novelty and security. A change of level is a *context switch* that is shown to the user and logged, so that the human always knows who is in charge.

2.9 Summary of the Literature

Table 2.1 Summary of the Literature Reviewed

Work	Contribution	Limitation addressed by SkyControl
Cardosi (1993); Cardosi et al. (1998)	Readback error rates and hearback misses in real R/T	Hearback depends on the controller alone
EUROCONTROL (2004)	Communication hazards in European airspace	Callsign confusion handled by vigilance
Helmke et al. (2016)	ASR reduces controller workload	ASR not used to verify readbacks
Kuchar & Yang (2000)	Taxonomy of conflict detection and resolution	Trade-off between look-ahead and false alerts
EUROCONTROL (2007)	STCA specification	Short look-ahead, no intent
Ayhan & Samet (2016); Pang et al. (2021)	Data-driven trajectory prediction	Evaluation on held-out data, real-time cost
Schäfer et al. (2014); Strohmeier et al. (2015)	OpenSky data; ADS-B weaknesses	Integrity not shown to the controller
Liu et al. (2008); Olive et al. (2018)	Unsupervised anomaly detection	Emergencies unlabelled; needs a rule partner
Parasuraman et al. (2000); Bainbridge (1983)	Levels of automation; ironies of automation	Automation level fixed, not context-dependent
Parasuraman & Manzey (2010)	Complacency and automation bias	No explicit signal of when not to trust the AI

2.10 Gap Analysis of Current ATC

Combining the literature with an analysis of the real recording gives nine gaps, listed in Table 2.2. Each gap is a place where the current system relies on a single human check, a short look-ahead or information it does not have. For each gap SkyControl adds a specific fix, states how the fix relates to existing ATC tools and states what kind of evidence supports it. SkyControl is designed to *complement* existing ATC: it never removes a safety net, and the human keeps authority over every non-routine decision.

Table 2.2 Gaps in Current ATC and the SkyControl Fixes

Gap	Problem	Today	SkyControl fix	Evidence
G1	Voice readback and hearback errors	Controller must hear the wrong readback	Structured clearances, datalink for routine items, automatic readback verification	Measured component + modelled system
G2	Similar callsigns on one frequency	Planning-stage deconfliction and vigilance	Live similarity scoring, label warnings, wrong-aircraft detection	Measured on real data
G3	Short look-ahead and nuisance alerts	STCA about 2 min, straight line	Learned prediction + cleared-level intent + duplicate merging (5 min)	Measured on real data
G4	Untrusted surveillance	Tracker output assumed correct	Duplicate detection, dead-reckoned coasting, switch to MANUAL	Measured on real data
G5	Delay absorbed in low-level holding	Holding decided near the stack	Hold prediction 30-80 NM out, linear-holding estimate	Measured + modelled
G6	Emergency handling by memory and phone	Checklists, mental alternates, telephone	Airport ranking, path checked against real traffic, re-sequencing	Real traffic + modelled workflow
G7	Level busts seen late	CLAM only if the level is entered	Cleared level always known from the digital clearance	Real tracks + injected events
G8	Automation that cannot say when not to trust it	Tools on or off	Four levels with explicit context switches	Scenario
G9	Runway safety and closures	Manual re-planning	Occupancy-aware sequencing and re-planning	Scenario

The gaps fall into four groups. G1 and G2 are human-communication gaps, the most direct answer to the request to reduce miscommunication and human error. G3, G4 and G7 are separation and surveillance gaps, where the controller acts on predictions and data that may be late or wrong. G5 and G6 are efficiency and emergency gaps, where workload and fuel are at stake. G8 and G9 concern trust in automation and runway safety. Fig. 2.1 shows the gap catalogue as it appears in the application, where each gap links to its evidence and to the scenario that demonstrates it.

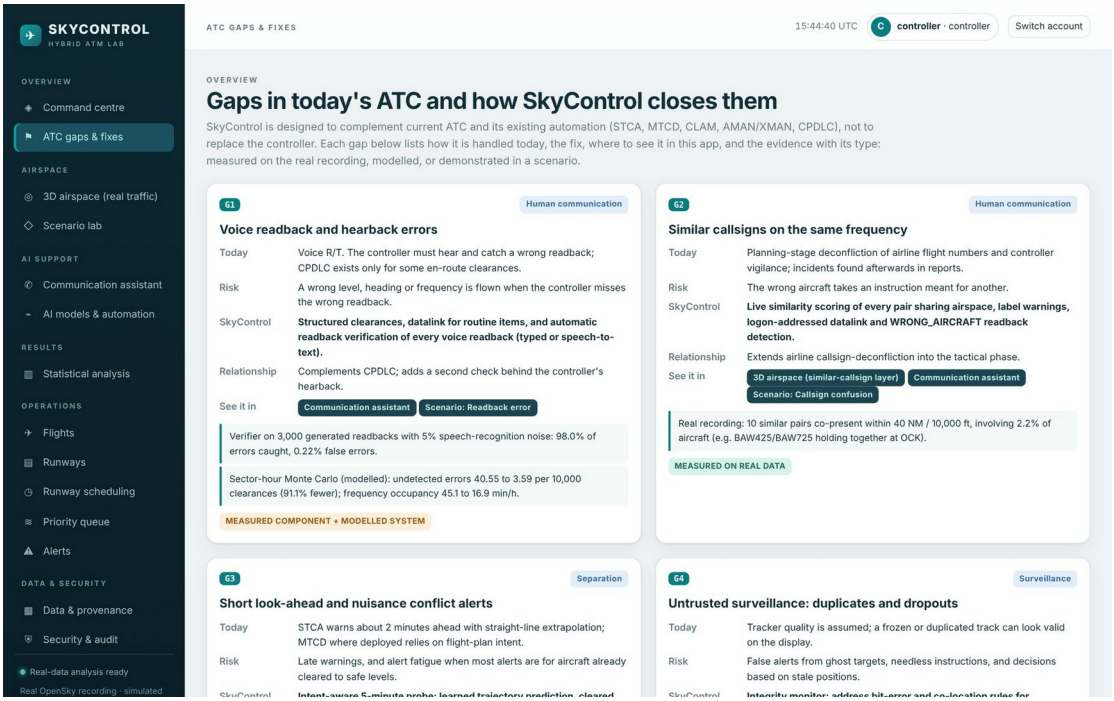


Fig. 2.1 ATC Gaps and Fixes Page of the Application

The gaps were found in two steps. First, the literature above identified candidate weaknesses. Second, each candidate was checked against the real recording: for example, the recording contains holding at every Heathrow stack, ten pairs of similar callsigns sharing airspace, 21 duplicate-target pairs and 234 dropouts of more than a minute, which confirms that G2, G4 and G5 are real in today's traffic and not only in the literature. Where the data cannot answer a question, for instance how often a controller misses a wrong readback, the report uses a stated model with a sensitivity analysis.

CHAPTER 3
TECHNICAL SPECIFICATION

This chapter specifies what SkyControl must do and under which constraints: the functional and non-functional requirements, the dataset, the hardware and software environment, the technology stack, the security policy and the programming interface between the browser and the server.

3.1 Functional Requirements

Table 3.1 Functional Requirements

ID	Requirement
FR1	Users log in with a username and password verified against a salted hash; wrong credentials are rejected with one uniform message.
FR2	Every login attempt is stored with time, username tried, IP address, browser, success flag and reason.
FR3	A username is locked for 15 minutes after 7 failures; an IP is limited after 40 failures in 15 minutes.
FR4	Two roles exist: controller and supervisor. Supervisors approve critical plans, overrides, runway closures and user management.
FR5	The real recording is replayed in 3D with trails, labels, predicted conflicts, similar callsigns, holding, duplicates and predictions.
FR6	An emergency (low fuel, medical, engine failure, emergency descent, lost communications) can be injected on any real aircraft.
FR7	The scenario lab runs 21 scenarios, each shown as a ten-step pipeline with an approval gate and a comparison with conventional ATC.
FR8	Every plan is verified conflict-free (aircraft, runway, hazard, deadline) before it can be executed.
FR9	The automation level is chosen by an explicit policy and every change is logged as a context switch.
FR10	The communication assistant builds ICAO phraseology, verifies readbacks (typed or spoken) and warns of similar callsigns.
FR11	The statistical analysis page shows measured, modelled and scenario results with tests and confidence intervals.
FR12	Operations pages manage flights, runways, runway bookings, the priority queue, alerts and incident reports.
FR13	The data page shows dataset provenance and allows importing snapshots; the audit page lists every change.
FR14	Learned models are trained on one part of the recording and evaluated on another part.

3.2 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

Quality	Requirement
Security	PBKDF2-SHA256 with 310,000 iterations; no default passwords; CSRF tokens; security headers; least-privilege container user
Reliability	All plans verified by exact geometry; automated tests for every module; health check endpoint
Performance	Interactive pages respond within about 100 ms (p50) after the one-off analysis; scenario creation under 0.5 s
Usability	Single-page app; every scenario explains itself step by step; works in current Chrome, Edge and Firefox
Portability	Runs on Windows, Linux and macOS with Python 3.12, or anywhere with Docker
Maintainability	Layered modules, lint (ruff) clean, 90% test coverage, documented algorithms
Honesty	Every result labelled measured, modelled or scenario; limitations stated; not presented as operational

3.3 Dataset Specification

The main dataset is a recorded OpenSky Network capture stored as a compressed JSON file (`data/london_amsterdam_opensky.json.gz`, format `skycontrol.opensky.recording.v1`). Each frame contains the state vectors of all aircraft in the bounding box at one time. The dataset register in the application stores the provider, period, number of records and SHA-256 hash so that every result can be traced to the exact file. Table 3.3 summarises the recording.

Table 3.3 Specification of the Real Dataset

Property	Value
Provider	OpenSky Network (ADS-B / Mode S state vectors)
Period	2026-10-15 15:42:28 to 17:12:28 UTC (90 minutes)
Region	49.5°N to 54.5°N, 2.0°W to 3.0°E (London TMA, North Sea, Amsterdam)
Frames	353, median interval 13 s
Position reports	87,155 in the file, 86,447 used after removing 708 duplicates
Aircraft	906 distinct ICAO 24-bit addresses; 81.6% of reports airborne
Fields used	ICAO address, callsign, latitude, longitude, barometric altitude, on-ground flag, time of position
Derived fields	Velocity, track, vertical rate and turn rate (finite differences), local x/y in NM
Reference data	18 airports and 8 holding fixes (data/reference/london_reference.json)
SHA-256	a1414d56cfc459bbf60f4770dd727455…

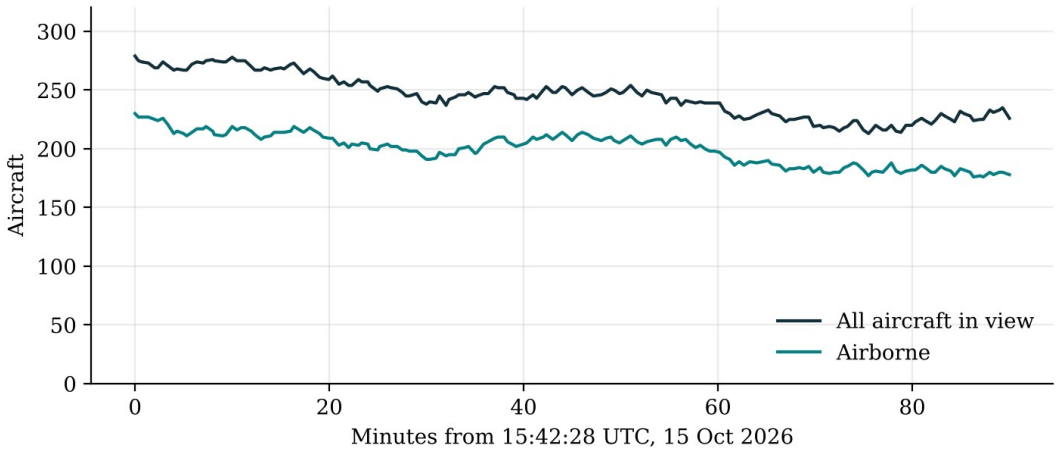


Fig. 3.1 Aircraft in View During the 90-Minute Recording

As Fig. 3.1 shows, traffic stays between about 210 and 280 aircraft throughout, of which roughly 200 are airborne. Fig. 3.2 shows the distribution of position reports by altitude band and the most frequent operators. The many reports below 5,000 ft come from arrivals and departures around the London airports, and the large share between 30,000 and 40,000 ft from overflights; Ryanair, easyJet and British Airways together operate more than a quarter of the aircraft.

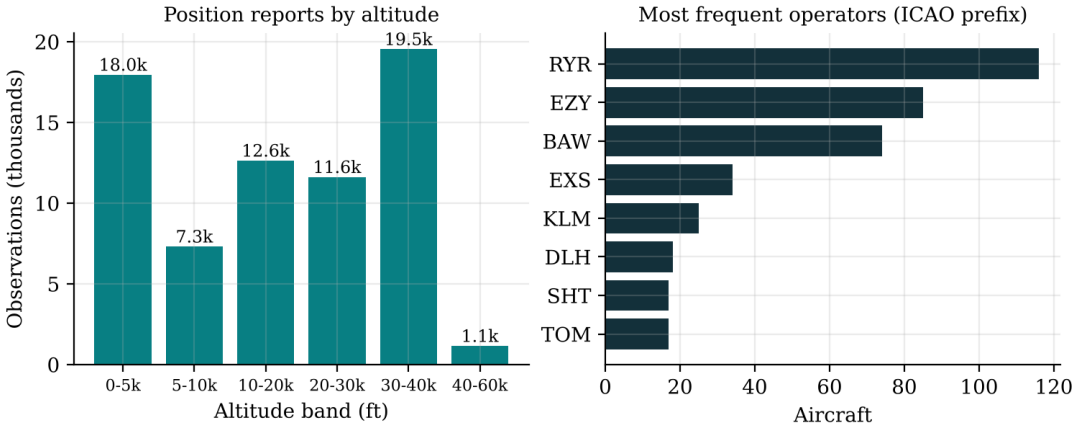


Fig. 3.2 Position Reports by Altitude Band and the Most Frequent Operators

Two additional datasets are registered: an OpenSky snapshot of Indian airspace kept from the earlier version of the project, and a historical airport traffic table (2011). They are shown on the data page with their provenance but are not used for the evaluation. Fuel states, emergencies, clearances and controller actions are not contained in any ADS-B dataset; where they are needed they are simulated and labelled.

3.4 Hardware and Software Requirements

Table 3.4 Hardware and Software Requirements

Item	Minimum	Used for development and testing
Processor	2 cores, 64-bit	4 vCPU (x86-64)
Memory	4 GB	8 GB
Disk	1 GB free	About 600 MB including the Docker image
Operating system	Windows 10/11, Ubuntu 22.04+, macOS 13+	Ubuntu (container), Windows 11 (VS Code)
Python	3.12	3.12 / 3.13
Browser	Chrome, Edge or Firefox with WebGL	Chromium (headless tests), Chrome
Docker (optional)	Docker Engine 24+ with Compose v2	Docker 27
Jenkins (optional)	Jenkins LTS with Pipeline, JUnit, Credentials Binding	jenkinsfile-runner with the same plug-ins

3.5 Technology Stack

Table 3.5 Technology Stack

Layer	Technology	Version	Reason
Web framework	FastAPI + Uvicorn	0.144.0 / 0.54.0	Typed, fast, automatic validation
Numerics	NumPy, SciPy	2.5.4 / 1.18.1	Vectorised geometry and statistics
Machine learning	scikit-learn	1.9.1	Gradient boosting, logistic regression, isolation forest
Database	SQLite (WAL mode)	Python stdlib	Zero administration, transactional
3D graphics	three.js	0.160.0	WebGL scene, vendored locally
Charts	Chart.js	4.4.4	Lightweight charts, vendored locally
Front end	HTML, CSS, ES modules	—	No build step, strict CSP
Testing	pytest, pytest-cov, httpx, Playwright	8.x	Unit, API and browser tests
Lint	ruff	0.6+	Fast static checks
Containers	Docker, Compose	27 / v2	Reproducible deployment
CI/CD	Jenkins, GitHub Actions	LTS	Automated test, build, deploy
Planning	Jira Software (Scrum)	Cloud	Backlog, sprints, reports

3.6 Security Specification

Authentication follows NIST SP 800-63B guidance for memorised secrets (Grassi et al., 2017) and the OWASP recommendations for password storage and session management (OWASP Foundation, 2021). Table 3.6 lists the controls. The

login portal deliberately returns the same message for an unknown username and a wrong password, so that it does not reveal which usernames exist, while the database records the precise reason for each failure for the security audit.

Table 3.6 Security Controls of the Login Portal and API

Control	Specification
Password storage	PBKDF2-HMAC-SHA256, 310,000 iterations, 16-byte random salt per user (RFC 8018)
Password policy	At least 12 characters with letters and digits; set at first start or from environment variables
Wrong credentials	HTTP 401 "Wrong username or password" with the number of attempts remaining
Attempt log	Every attempt stored: time, username, IP, user agent, success, reason (7 reason codes)
Lockout	7 failures per username or 40 per IP within 15 minutes → HTTP 429 until the window passes
Sessions	256-bit random token, stored only as SHA-256 hash; HTTP-only, SameSite=Lax cookie; 12-hour expiry
CSRF	Per-session token required in a header for every state-changing request
Authorisation	Role checks on every route; supervisor-only actions enforced server side
Headers	Content-Security-Policy script-src self, X-Frame-Options DENY, nosniff, no-referrer, no-store for API
Audit	Every change written to audit_events with actor, action, entity and details

3.7 Interface Specification

The browser talks to the server only through a JSON REST API under `/api`. Table 3.7 lists the main endpoint groups; the complete list has 61 routes. All routes except login and health require a session, and state-changing routes require the CSRF token.

Table 3.7 Main API Endpoint Groups

Group	Examples	Purpose
/api/auth	POST login, logout, password; GET me, attempts, users	Login portal, attempt log, user management
/api/airspace	GET meta, tracks, frame, predictions; POST emergency	3D replay of real traffic and emergency injection
/api/lab	GET catalog, benchmark, runs/{id}; POST runs, runs/{id}/control	Scenario lab, approvals and replay control
/api/analysis, /api/gaps, /api/models	GET analysis/{section}, gaps, models	Statistical results and gap catalogue
/api/comms	POST phraseology, verify, similarity; GET log	Communication assistant
/api/automation	POST decide	Context-switch policy simulator
/api (operations)	flights, runways, schedule, priority-queue, alerts, incident-reports, dashboard	Operations management
/api/data, /api/audit	datasets, snapshots, import, feed-health, audit	Data provenance and audit trail
/api/health	GET	Liveness and analytics readiness (used by Docker and Jenkins)

3.8 Acceptance Criteria

The project is accepted when the following criteria are demonstrated:

- A wrong password is rejected with a clear message and the attempt appears in the security log with its reason.
- All 21 scenarios produce a plan that passes exact verification, and critical plans cannot be executed by a controller without supervisor approval.
- Learned models improve on their baselines on held-out real data, with paired statistical tests reported.
- An emergency can be injected on a real aircraft and the response is shown step by step.
- The automated test suite passes in the Jenkins pipeline, the Docker image passes the smoke test and is deployed.

CHAPTER 4
DESIGN APPROACH AND DETAILS

This chapter explains how SkyControl is designed. It first states the design approach and the architecture, then presents the UML model of the system, then the materials and methods (the data processing, algorithms and models, with their equations), the codes and standards followed, and finally the constraints, alternatives and trade-offs that shaped the design.

4.1 Design Approach

Three principles guided the design. The first is **complement, do not replace**. SkyControl never removes an existing safety net or procedure. The STCA-style safety net, the controller's hearback and the supervisor's authority remain; SkyControl adds a second check, a longer look-ahead, an integrity signal or a verified plan beside them. This keeps the system acceptable to the people who would use it and keeps the comparison with current ATC fair.

The second principle is **verified automation with an explicit human in the loop**. Any plan that SkyControl proposes, whether it comes from a heuristic, a search algorithm or a learned model, is checked by an exact geometric verifier before it can be executed. The level of automation is decided by a transparent policy, and the human can approve, reject or override with a reason. When the learned model and the deterministic rule disagree, the rule wins and the human decides, which is the "manual context switch in critical situations where the AI can be wrong" required by the project brief.

The third principle is **evidence first**. Every function was designed together with the measurement that would show whether it works on real traffic. Learned models are always evaluated on data they did not see. Where a first design did not give a significant or useful result, the design was changed. For example, a plain five-minute probe produced 522.7 alerts per hour on the real recording, which would be useless to a controller, so duplicate merging, level-flight intent, cleared-level intent, learned trajectories and hysteresis were added until the alert rate matched the two-minute safety net.

The development itself followed an iterative Scrum approach (Chapter 6): each sprint delivered a working increment, from the login portal and data ingestion in Sprint 1 to the 3D application and DevOps pipeline in the last sprints.

4.2 System Architecture

SkyControl is a layered client-server application (Fig. 4.1). The browser runs a single-page application that renders the 3D views with three.js and the charts with Chart.js. It communicates over JSON with a FastAPI server. The server has four layers: the API routes (authentication, validation, role checks, CSRF and audit), the decision engine (`core/`: scenario planning, separation verification, communication, automation policy, workflow model and gap catalogue), the analytics layer (`analytics/`: everything computed on the real recording) and the persistence layer (SQLite in write-ahead-log mode). The analytics results and the trained predictor are cached on disk, keyed by the SHA-256 of the recording and the analysis version, so the expensive analysis runs once.

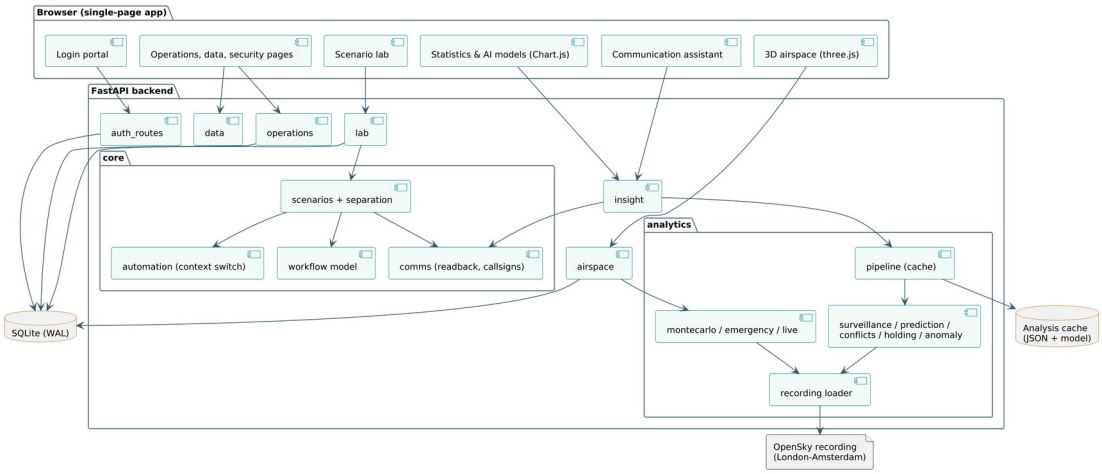


Fig. 4.1 Component Architecture of SkyControl

Fig. 4.2 shows the data flow from the raw recording to the user. The recording is loaded, cleaned and projected, then each analysis consumes the cleaned arrays. The live views used by the 3D replay are computed per frame from the same models, so what the user sees in the replay is exactly what the statistics measure.

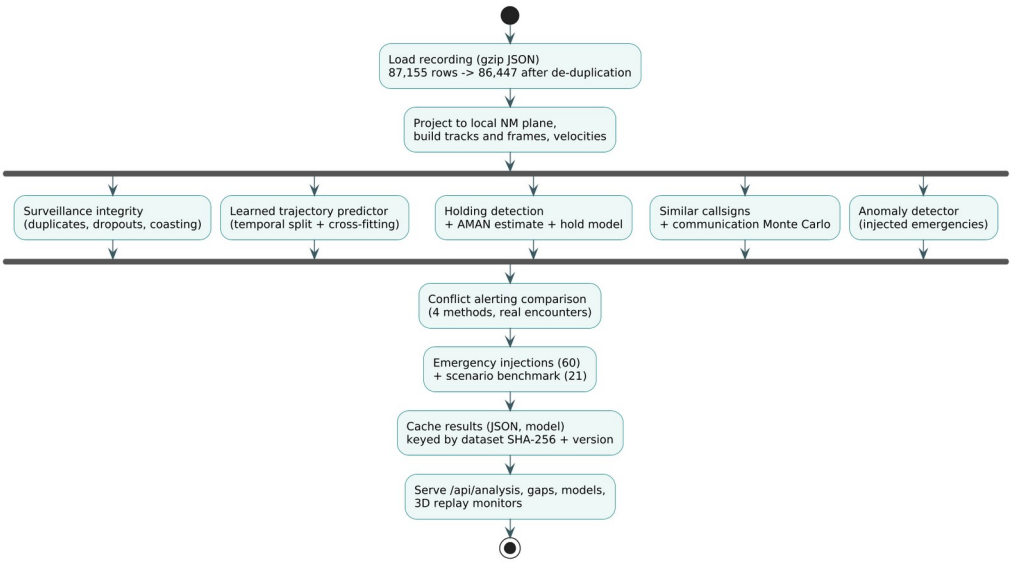


Fig. 4.2 Data Flow from the OpenSky Recording to the User Interface

The deployment view (Fig. 4.3) shows the production arrangement. The application runs in a single Docker container as an unprivileged user, with the database on a named volume. Jenkins builds, tests and deploys the container, and GitHub holds the source and runs a second CI workflow.

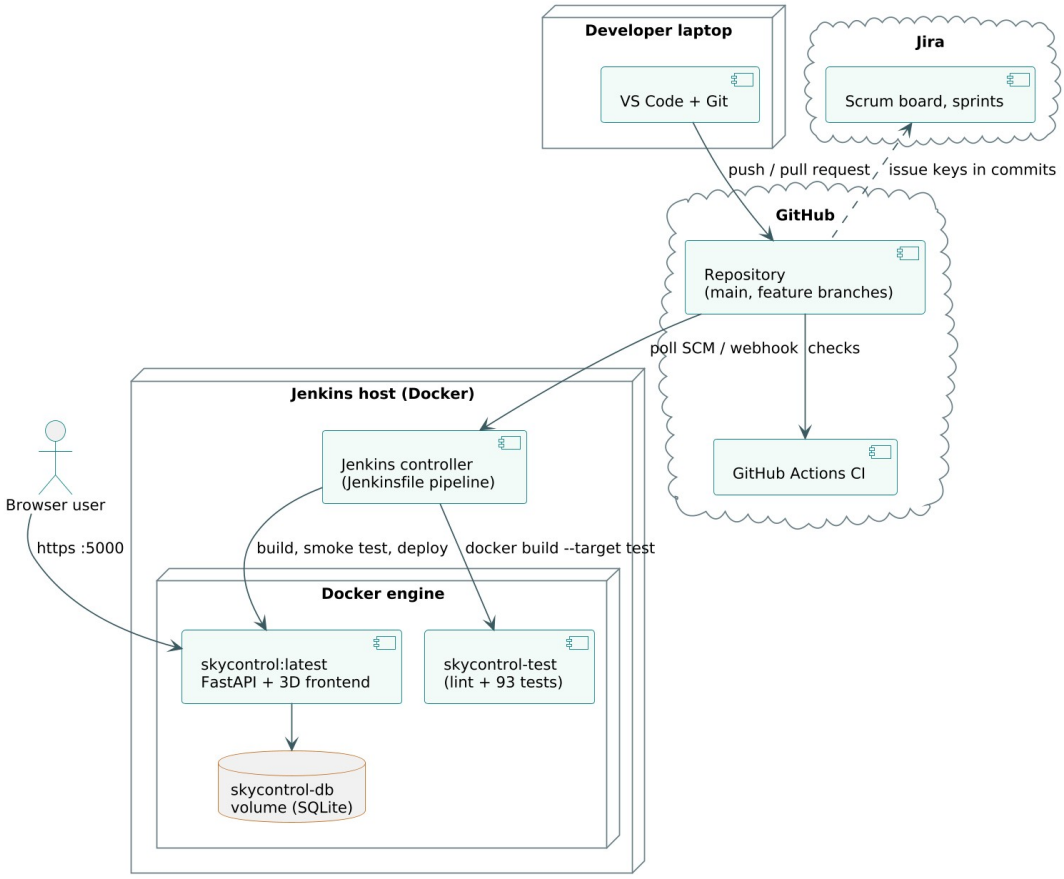


Fig. 4.3 Deployment Diagram

4.3 UML Design

4.3.1 Use Case Diagram

Three actors use SkyControl (Fig. 4.4). The controller logs in, monitors the real airspace, runs scenarios, consents to plans for high-severity events, verifies readbacks and manages flights, runways and alerts. The supervisor can do everything the controller can and additionally approves critical plans, overrides plans with a reason, reviews emergency responses, manages users and reads the audit trail. The automation (SkyControl itself) acts as a system actor that detects situations, plans, verifies and switches context.

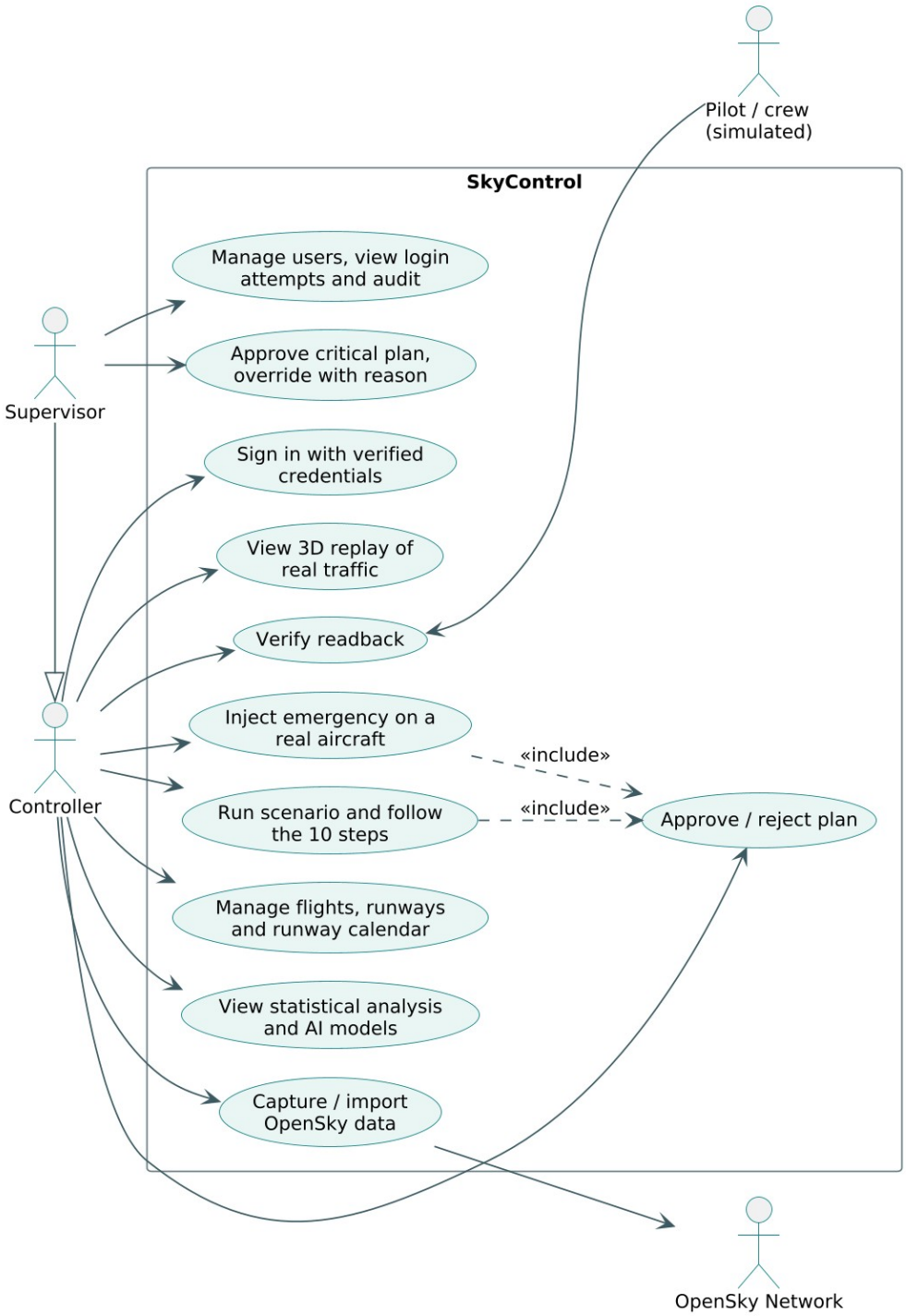


Fig. 4.4 Use Case Diagram

4.3.2 Class Diagram

The class diagram (Fig. 4.5) shows the main domain and analysis classes. A `Recording` holds the cleaned arrays of the real data. A `TrajectoryPredictor` owns one pair of gradient-boosted models per horizon. A `Scenario` is built by the scenario builder and contains `Aircraft` with routes, the comparator plan and the SkyControl plan, the pipeline `Step` list, the `ContextSwitch` log and the `Message`

list. A `Clearance` produces phraseology and is compared with a readback by the verifier.

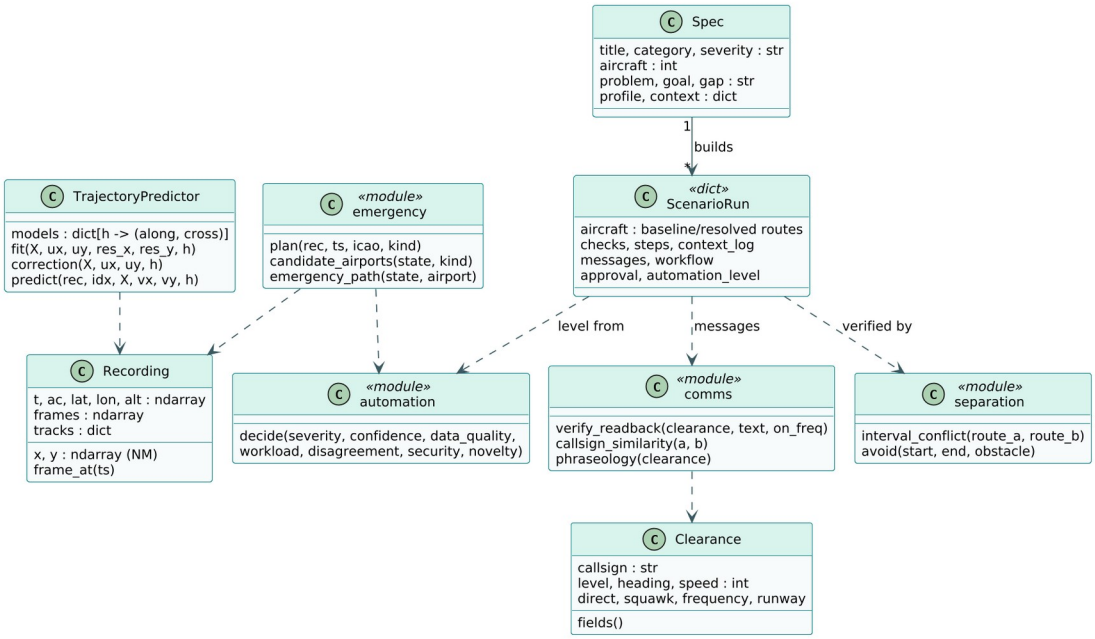


Fig. 4.5 Class Diagram of the Main Components

4.3.3 Sequence Diagrams

The login sequence (Fig. 4.6) shows how a wrong password is handled. The server counts recent failures for the username and the IP address inside one transaction, checks the PBKDF2 hash, always writes an attempt row with the reason, and either returns 401 with the remaining attempts, 429 when locked, or creates a session.

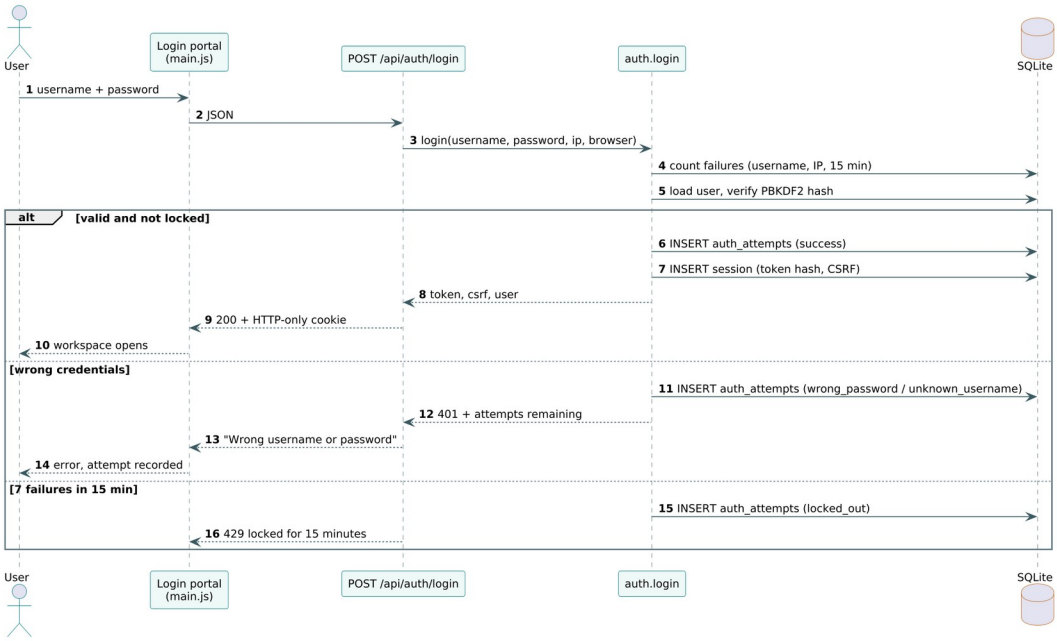


Fig. 4.6 Sequence Diagram of the Login Portal

The emergency sequence (Fig. 4.7) shows what happens when an emergency is injected on a real aircraft: the emergency planner ranks airports, builds a descent path, checks it against all recorded traffic, proposes level changes, re-sequences arrivals, drafts the messages and asks the automation policy for the level; the result is stored and a supervisor reviews it.

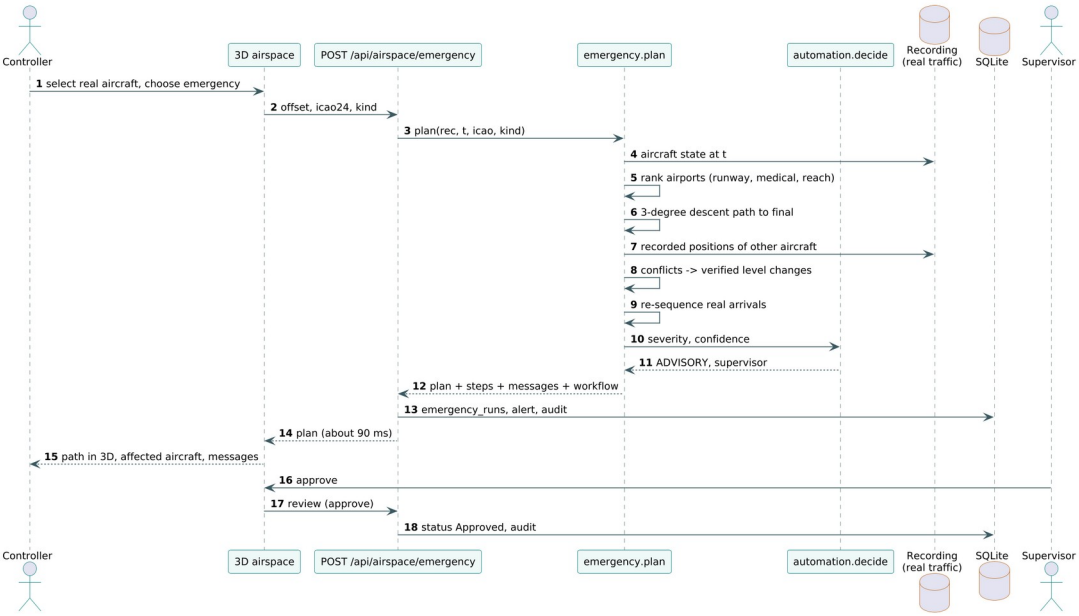


Fig. 4.7 Sequence Diagram of Emergency Injection on Real Traffic

4.3.4 Activity and State Diagrams

Every scenario and every injected emergency is handled by the same ten-step pipeline (Fig. 4.8): DETECT, ASSESS, CONTEXT SWITCH, PLAN, VERIFY,

AUTHORISE, COMMUNICATE, EXECUTE, MONITOR and CLOSE. The AUTHORISE step is a decision point: AUTO plans continue, CONSENT plans wait for one controller click, ADVISORY and MANUAL plans wait for the required role, and a rejected plan returns to the human.

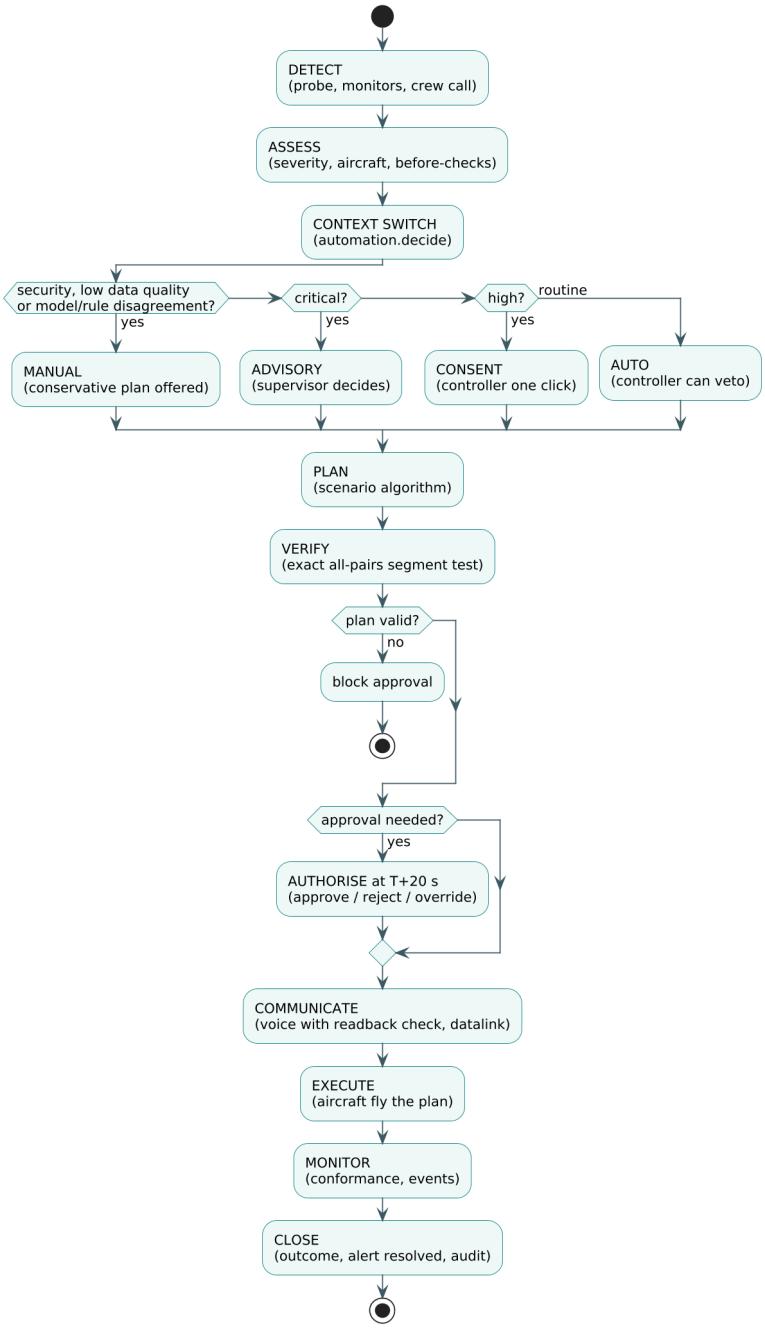


Fig. 4.8 Activity Diagram of the Ten-Step Handling Pipeline

The automation level is a state machine (Fig. 4.9). Events such as a degraded surveillance signal, a disagreement between model and rule, a security signal or a critical severity move the system to a lower level of automation; recovery of data

quality and confidence moves it back. Every transition is a context switch that is shown in the user interface and logged.

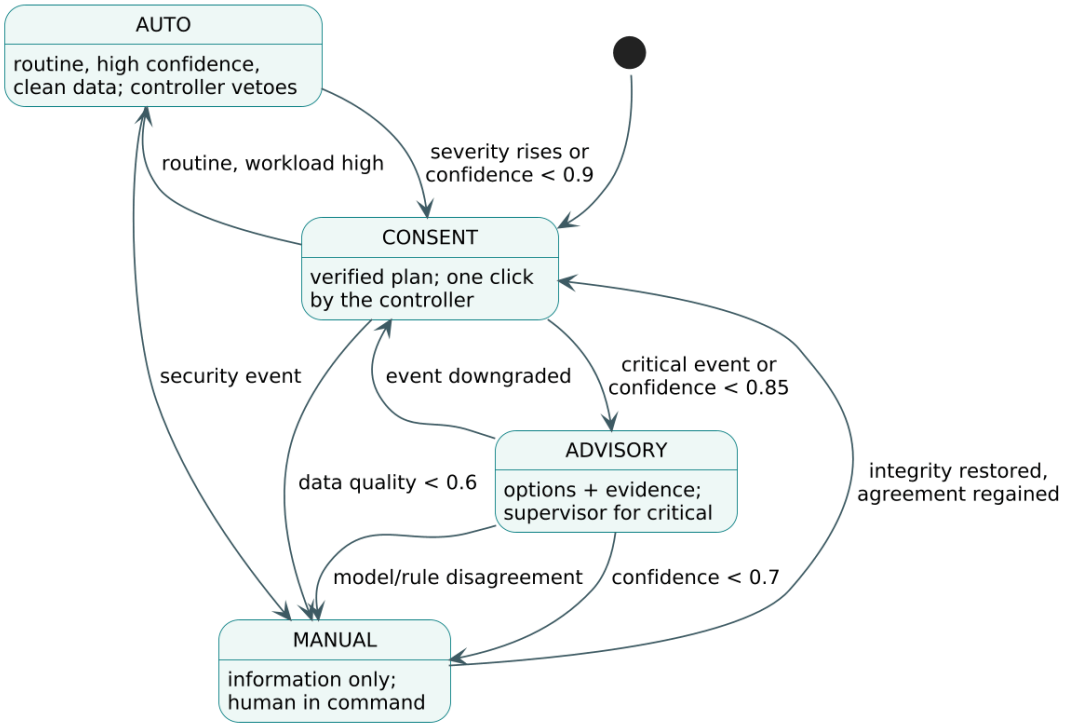


Fig. 4.9 State Diagram of the Automation Levels (Context Switches)

4.3.5 Database Design

The database (Fig. 4.10) has 16 tables. The security tables are `users`, `sessions` and `auth_attempts`; the operations tables are `flights`, `runways`, `runway_slots`, `alerts` and `incident_reports`; the lab and assistant tables are `lab_runs`, `emergency_runs` and `comm_checks`; the data tables are `datasets`, `snapshots`, `state_vectors` and `feed_events`; and `audit_events` records every change. Foreign keys and CHECK constraints enforce valid statuses, priorities and roles.

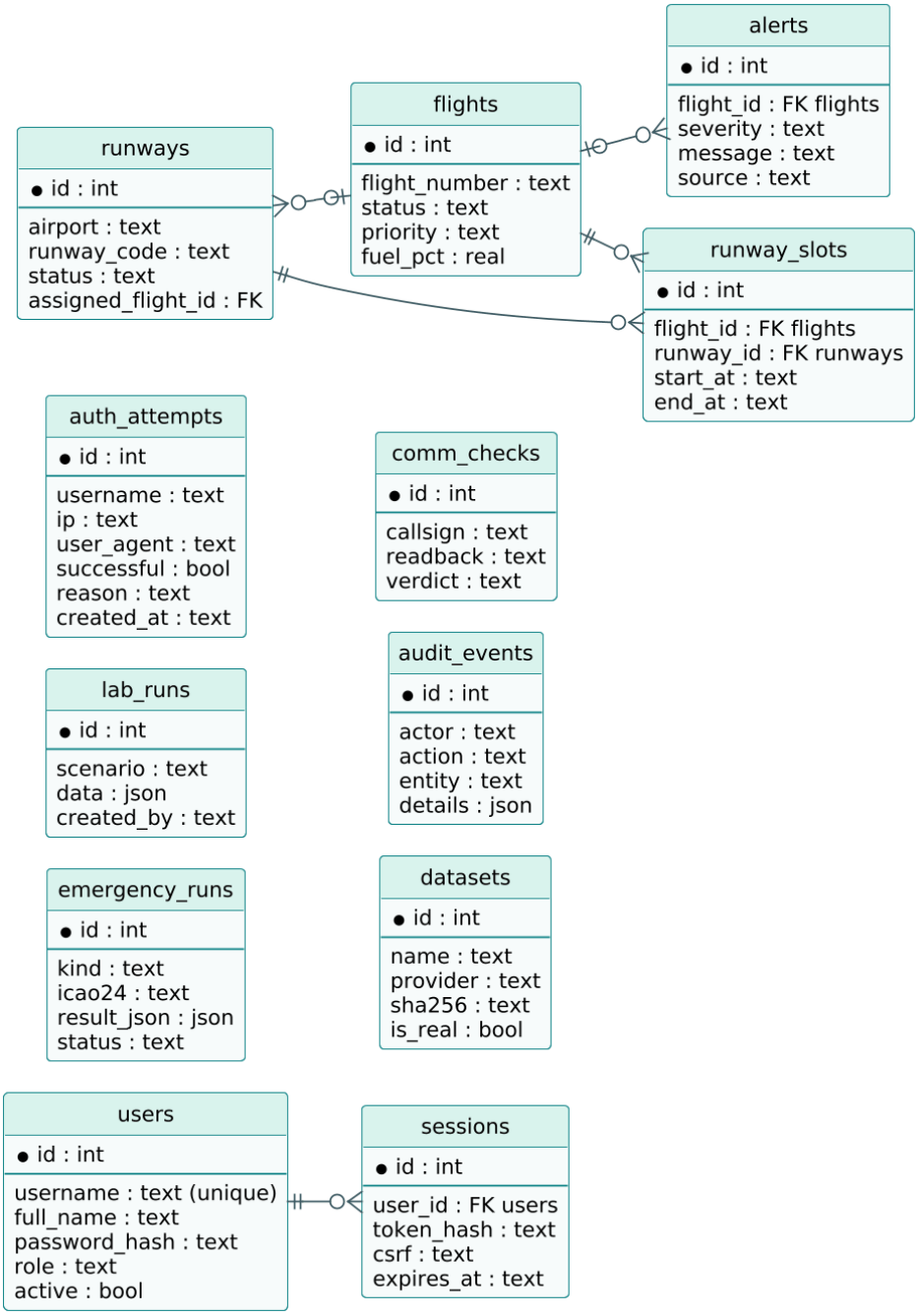


Fig. 4.10 Entity-Relationship Diagram of the Main Tables

4.4 Materials and Methods

This section describes the methods used for each gap. The material is the real recording described in Section 3.3; the methods are the algorithms implemented in `backend/analytics` and `backend/core`.

4.4.1 Data Preparation and Projection

The recording is loaded frame by frame. Reports with a missing position are dropped, and repeated reports of the same aircraft with an unchanged time of position

are removed (708 of 87,155). Positions are projected onto a local plane centred on 52°N, 0.5°E with the equirectangular approximation of Eqn. 4.1, which is accurate to well under 0.1% over the 300 NM extent of the region.

$$x = R\left(\lambda - \lambda_0\right)\cos\varphi_0 \,,\; y = R\left(\varphi - \varphi_0\right) \tag{4.1}$$

In Eqn. 4.1, R is the Earth radius in nautical miles, φ and λ are latitude and longitude, and φ_0, λ_0 the centre of the projection. Velocities are derived from consecutive reports of the same aircraft by finite differences, Eqn. 4.2, and the track angle, ground speed, vertical rate and turn rate follow from them.

$$v_i = \frac{p_i - p_{i-1}}{t_i - t_{i-1}} \tag{4.2}$$

4.4.2 Surveillance Integrity Monitor (G4)

Duplicate (ghost) targets are detected with three rules applied to aircraft within 300 ft vertically: two targets with the same callsign within 5 NM; two targets whose 24-bit addresses differ in at most two bits (a typical corruption) within 1 NM for at least two frames; and an anonymous target (no callsign) sitting within 0.5 NM and 100 ft of another aircraft. Different callsigns flying that close are treated as a formation, not a duplicate. The duplicate track is merged into the real one before any alerting.

When an aircraft stops reporting for more than 60 s, the conventional display freezes its last position. SkyControl instead coasts the track by dead reckoning, Eqn. 4.3, and shows the growing uncertainty. If the share of stale or duplicated tracks around an event is high, data quality falls and the automation policy switches to MANUAL.

$$\hat{p}\left(t\right) = p\left(t_{last}\right) + v\left(t_{last}\right)\cdot\left(t - t_{last}\right) \tag{4.3}$$

4.4.3 Learned Trajectory Prediction (G3)

The baseline used by current safety nets is straight-line extrapolation, Eqn. 4.4. SkyControl keeps this as its backbone and learns a correction. For each horizon h of 60, 120 and 300 s, two gradient-boosted regression models predict the along-track residual a_h and the cross-track residual c_h from eight features: x, y, ground speed, the sine and cosine of the track, turn rate, altitude and vertical rate. The learned prediction is Eqn. 4.5, where û is the unit vector along the current track and n̂ the unit vector to its right.

$$\hat{p}_{SL}(t+h) = p(t) + v(t)\cdot h \tag{4.4}$$

$$\hat{p}_{L}(t+h) = \hat{p}_{SL}(t+h) + a_h(f)\cdot\hat{u} + c_h(f)\cdot\hat{n} \tag{4.5}$$

The models are scikit-learn `HistGradientBoostingRegressors` with 250 iterations, learning rate 0.06 and 31 leaves. Training uses only the first 60 minutes of the recording, and a target position must also fall inside that period; testing uses the last 30 minutes. The evaluation metric is the mean horizontal position error at each horizon, Eqn. 4.6, compared pairwise with the straight-line error on the same test points.

$$E_h = \frac{1}{N}\sum_{i=1}^{N} \left\| \hat{p}_i(t+h) - p_i(t+h) \right\| \tag{4.6}$$

Learning a residual rather than the full position has two advantages. If the model knows nothing useful, it predicts a residual close to zero and the prediction falls back to the straight line; and the trees only need to learn the structure of the airspace (turns onto arrival routes, stack entries, level-offs), not basic kinematics.

4.4.4 Intent-Aware Conflict Probe (G3)

A conflict is predicted when two predicted positions come closer than the horizontal threshold S_h and the vertical threshold S_v at the same future time within the look-ahead H, Eqn. 4.7. S_h is 3 NM below FL245 and 5 NM above, S_v is 1,000 ft with a 100 ft tolerance for 25 ft altitude quantisation, and only aircraft at or above 3,000 ft are considered. The probe samples the predicted paths every 5 s.

$$\left\| \hat{p}_a(t) - \hat{p}_b(t) \right\| < S_h \wedge \left| \hat{z}_a(t) - \hat{z}_b(t) \right| < S_v, t \in [0, H] \tag{4.7}$$

Four alerting methods are compared on the same real traffic. The STCA-style safety net extrapolates straight lines for 120 s and trusts every target. The naive probe does the same for 300 s. The filtered probe adds duplicate merging, level-flight intent (an aircraft with a vertical rate below 300 ft/min is treated as level) and two-frame hysteresis. The intent-aware probe adds the learned trajectories of Eqn. 4.5 and the cleared level of each aircraft, so that a climbing or descending aircraft is predicted to stop at its cleared level instead of continuing through it. With digital clearances the ground system knows the cleared level; in the recording it is approximated by the level at which the aircraft actually levels off next, which is stated as a counterfactual in the results.

Ground truth is the set of encounters actually observed in the recording: pairs inside the screening volume of 3 NM or 5 NM and 900 ft. An alert episode is a run of alert samples for the same pair with gaps of no more than 45 s. An alert that is not followed by an encounter within the look-ahead is a nuisance alert. Warning time is the time between the first alert and the start of the encounter.

4.4.5 Exact Verification of Planned Routes

Planned routes in the scenario lab are piecewise linear in space and time. For any pair of overlapping segments the relative motion is linear, so the squared horizontal distance is a quadratic in time τ, Eqn. 4.8, where Δp and Δv are the relative position and velocity at the start of the overlap.

$$d^2(\tau) = |\Delta v|^2 \tau^2 + 2(\Delta p \cdot \Delta v)\tau + |\Delta p|^2 \tag{4.8}$$

The horizontal condition $d(\tau) < S_h$ therefore holds between the roots of Eqn. 4.9, with $A = |\Delta v|^2$, $B = 2\Delta p \cdot \Delta v$ and $C = |\Delta p|^2 - S_h^2$. The vertical condition holds in the interval of Eqn. 4.10. A conflict exists if the intersection of the two intervals with the overlap of the segments is not empty. This test is exact, not sampled, so a plan that passes it cannot hide a short loss of separation between samples.

$$\tau_{1,2} = \frac{-B \mp \sqrt{B^2 - 4AC}}{2A} \tag{4.9}$$

$$\tau \in \left[\frac{-S_v - \Delta z}{\Delta \dot{z}}, \frac{S_v - \Delta z}{\Delta \dot{z}} \right] \tag{4.10}$$

Routes around weather cells and closed airspace are found with Dijkstra's shortest-path algorithm (Dijkstra, 1959) on a visibility graph of 16 nodes placed on a ring around the buffered obstacle. Converging traffic is resolved by greedy flight-level assignment: aircraft are taken in order of priority and each receives the nearest level that is conflict-free against all aircraft already assigned, verified with Eqn. 4.9. Arrival sequencing puts priority traffic (fuel, medical and other emergencies) first and then keeps the original order, adding holding circuits until every arrival is at least 120 s after the previous one and no runway closure window is violated.

4.4.6 Holding Detection and Linear-Holding Estimate (G5)

A hold is detected when an airborne aircraft above 4,000 ft accumulates at least 330° of turn within 8 minutes, Eqn. 4.11, and it is attributed to the nearest published holding fix within 15 NM. Fuel burned in the hold is estimated with Eqn.

4.12 at a nominal 40 kg per minute (sensitivity range 30 to 100 kg per minute), and CO_2 with the standard factor of 3.16 kg per kg of fuel.

$$\sum_k \left|\Delta\psi_k\right| \geq 330\,^\circ, t_{end} - t_{start} \leq 480\,s \tag{4.11}$$

$$F = f_{hold} \sum_j m_j, CO_2 = 3.16\,F \tag{4.12}$$

The delay that could instead be absorbed en route by an arrival manager with a longer horizon is estimated per flight with Eqn. 4.13, where D is the horizon (350 NM), V the cruise speed (450 kt) and r the maximum speed reduction (10%). The absorbable delay of each hold is the smaller of the hold duration and this cap.

$$\Delta t_{absorbed} = \frac{D}{V}\left(\frac{1}{1-r} - 1\right) \tag{4.13}$$

4.4.7 Hold Prediction (G5)

To absorb delay early, the system must know early which arrivals will hold. Every stack-bound arrival is sampled at the first moment it is 30 to 80 NM from the stack, before any hold. The label is whether it later held at that stack (from the real tracks). The six features are what a ground system knows at that moment: distance, altitude, ground speed, the number of aircraft already holding at the stack, the number within 25 NM of it and the number of inbound aircraft ahead. A logistic regression with standardised features, Eqn. 4.14, is trained on the first 55 minutes and tested on the rest. $P_{hold}(x)$ is the predicted probability that an arrival with features x will hold.

$$P_{hold}(x) = \sigma\left(\beta_0 + \beta^T x\right) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}} \tag{4.14}$$

4.4.8 Anomaly Detection and Cleared-Level Monitoring (G7)

Because emergencies are not labelled in ADS-B data, an Isolation Forest is trained on normal kinematics (vertical rate, absolute turn rate, speed change, vertical acceleration, altitude and speed) from the training period. Its anomaly score is Eqn. 4.15, where E[h(x)] is the mean path length needed to isolate x in the random trees and c(n) the average path length of an unsuccessful search in a binary search tree of n points, Eqn. 4.16. The threshold is set for a 1% false-alarm rate on normal test observations.

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \tag{4.15}$$

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{4.16}$$

The conventional comparator is a fixed rule, |vertical rate| > 4,000 ft/min. The SkyControl hybrid detector is the union of the rule and the forest. Cleared-level adherence monitoring (CLAM) raises an alert when the altitude departs from the cleared level by more than 300 ft without a clearance; it is only possible when the cleared level is known, which the digital clearance record of SkyControl guarantees. Detection is measured by injecting three kinds of emergency into copies of real test-period tracks: an emergency descent at −6,000 ft/min, an engine-out drift-down with a 20% speed decay, and an unannounced turn-back.

4.4.9 Readback Verification and Callsign Similarity (G1, G2)

A clearance is represented as a structured object with optional fields: level, heading, speed, direct-to waypoint, squawk, frequency and runway. The phraseology generator turns it into ICAO-style words ("climb flight level one two zero"). The verifier tokenises the readback, reads numbers spoken as digits, NATO letters or "thousand/hundred" forms, identifies the answering callsign among those on frequency, and compares each required field. The result is one of four verdicts (Table 4.1). If digits were evidently lost, as happens with clipped transmissions or speech recognition, the field is marked unclear and the crew is asked to confirm rather than being told they are wrong.

Table 4.1 Readback Verifier Verdicts

Verdict	Condition	Action shown to the controller
CORRECT	All required fields match	None
INCOMPLETE	A required field is missing or unclear	Ask the crew to confirm (soft alert)
MISMATCH	A field has a different value	Correct immediately: "negative, I say again ..."
WRONG_AIRCRAFT	Another aircraft on frequency answered	Cancel and re-issue to both aircraft

Callsign similarity follows the patterns known to cause confusion (Table 4.2). Every pair of aircraft within 40 NM and 10,000 ft of each other, a proxy for sharing a sector frequency, is scored continuously; pairs scoring 0.7 or more are flagged on the display and in the communication assistant.

Table 4.2 Callsign Similarity Rules and Scores

Pattern	Example	Score
Same operator, same characters in a different order	BAW425 / BAW452	0.95
Same operator, one character differs	HLE98 / HLE99	0.90
Same operator, same final characters	BAW425 / BAW725	0.75
Same operator, same leading characters	EZY71A / EZY71Q	0.70
Same operator, one character added or dropped	RYR52 / RYR522	0.65
Different operator, same flight number	KLM1234 / DLH1234	0.60

4.4.10 Communication Model (G1, G2)

The effect of the communication functions on a whole sector cannot be measured from ADS-B data, so it is modelled by Monte Carlo simulation of 4,000 busy sector-hours with 300 clearances per hour (Poisson). In the voice baseline a readback error (probability p_{rb}) or a callsign confusion goes undetected when the controller misses it (p_{miss}). In the hybrid, a share δ of routine clearances goes by datalink with a small residual error p_{dl}, similar-callsign warnings prevent a share a of confusions, and an error on voice goes undetected only if both the controller and the verifier (detection rate D, measured) miss it. The expected numbers of undetected errors per hour are approximately Eqn. 4.17 and Eqn. 4.18, where e is the similar-callsign exposure measured on the real recording and p_{cc} the confusion probability when similar callsigns share a frequency.

$$U_{voice} = N\left(p_{rb} + p_{cc}\, e\right) p_{miss} \tag{4.17}$$

$$U_{hyb} = N\left[(1-\delta)\left(p_{rb} + p_{cc}\, e\,(1-a)\right) p_{miss}\,(1-D) + \delta\, p_{dl}\right] \tag{4.18}$$

Because p_{rb} and p_{miss} come from the literature rather than from measurement, the model is run over a grid of values (p_{rb} from 0.5% to 3%, p_{miss} from 20% to 60%) and the datalink share is varied from 0 to 90%. The verifier detection rate D is measured on a corpus of 3,000 readbacks generated from real callsigns, both clean and with 5% simulated speech-recognition word noise.

4.4.11 Emergency Planning on Real Traffic (G6)

When an emergency is declared for a real aircraft, candidate airports are filtered by runway length, medical support and reachability within the endurance of the emergency type, and ranked by distance. The emergency trajectory goes direct to a 10 NM final with a 3° descent profile, Eqn. 4.19; if the aircraft is too high for the distance, a dog-leg is inserted to lengthen the track. The trajectory is checked every 10

s for 15 minutes against the recorded future positions of every other aircraft (5 NM, 1,000 ft). For each conflicting aircraft, level changes between 3,000 ft and 41,000 ft are tried in order of size and re-checked against all other traffic; aircraft close to an airfield at low level are re-sequenced instead. The emergency receives the first landing slot, with 120 s behind it and 90 s between following arrivals, and only arrivals whose original slot is affected are delayed.

$$d_{descent} = \frac{z_{now} - z_{aerodrome}}{318\,ft/NM} \tag{4.19}$$

4.4.12 Adaptive Automation and Context Switching (G8)

The automation level is chosen by an explicit rule policy inspired by the levels-of-automation model of Parasuraman et al. (2000). Its inputs are the severity of the event, the plan confidence (agreement between the planner's checks and the learned models), the surveillance data quality, the controller workload, disagreement between learned and deterministic components, a novelty (anomaly) score and a security flag. Table 4.3 gives the rules in order of precedence. Each decision returns the level, the role that must approve and the reasons, which are displayed as a context switch.

Table 4.3 Context-Switch Policy (Evaluated Top to Bottom)

Condition	Level	Approver
Security event (e.g. unlawful interference)	MANUAL	Supervisor
Surveillance data quality below 60%	MANUAL	Controller
Learned model disagrees with the safety rule	MANUAL	Controller
Novelty score above 0.8 (situation unlike training data)	ADVISORY	Supervisor if critical
Critical severity and confidence below 70%	MANUAL	Supervisor
Critical severity, verified plan	ADVISORY	Supervisor
High severity and confidence of at least 85%	CONSENT	Controller
High severity, lower confidence	ADVISORY	Controller
Routine, confidence ≥ 90% and data quality ≥ 90%	AUTO	None (controller can veto)
Routine, otherwise	CONSENT	Controller

4.4.13 Controller Workflow Model

To compare handling time with conventional ATC, every scenario and every emergency is also expressed as a controller workflow. In the conventional workflow the controller detects, decides, performs manual tasks (checklist, alternate search), telephones adjacent units and then issues every instruction by voice with a readback, Eqn. 4.20. In the hybrid workflow detection comes from the monitors, the plan is

computed and verified automatically, one review is needed if the level is not AUTO, coordination is digital, and urgent voice instructions run in parallel with routine datalink messages, Eqn. 4.21.

$$T_{conv} = t_{det} + t_{dec} + \sum_{k} t_k + n_c\, t_{phone} + n\left(t_i + t_{rb}\right) + n\, p_{rb}\, t_{corr} \tag{4.20}$$

$$T_{hyb} = t_{det}' + t_{plan} + t_{rev} + t_{dig} + max\left(n_u\left(t_i + t_{rb}\right), t_{dl}\right) \tag{4.21}$$

The task durations (5 s per voice instruction, 4 s per readback, 25 s per telephone call, 30 s for a checklist, 60 s for an alternate search, 30 s datalink response) are modelling assumptions informed by Cardosi (1993) and CPDLC experience. They are stored in one place (`core/workflow.py`) so that they can be changed, and the results are reported as modelled.

4.4.14 Statistical Evaluation

Wherever the baseline and SkyControl are evaluated on the same cases (the same test points, the same dropouts, the same encounters, the same injected emergencies or the same scenarios), the comparison is paired. The one-sided Wilcoxon signed-rank test (Wilcoxon, 1945) is used because the differences are not normally distributed; its statistic is the sum of the ranks of the positive differences, Eqn. 4.22. The uncertainty of the mean paired improvement is reported as a 95% bootstrap confidence interval (Efron & Tibshirani, 1993) from 2,000 resamples, Eqn. 4.23. Classifier performance is reported as the area under the ROC curve (Fawcett, 2006) with a bootstrap interval, and detection rates with Wilson intervals. The significance level is α = 0.05.

$$W^{+} = \sum_{d_i > 0} R_i \tag{4.22}$$

$$CI_{95} = \left[Q_{0.025}\left(\bar{d}^{\,*}\right), Q_{0.975}\left(\bar{d}^{\,*}\right)\right] \tag{4.23}$$

4.4.15 Password Hashing

Passwords are never stored. The stored value is the output of PBKDF2 with HMAC-SHA256 as the pseudo-random function (Moriarty, Kaliski, & Rusch, 2017), Eqn. 4.24, where P is the password, s a random 16-byte salt per user, c the iteration count and ℓ the key length in bytes; the salt and the iteration count are stored with the key. Verification recomputes the key and compares it in constant time.

$$DK = PBKDF2\left(P, s, c, \ell\right), c = 310\,000, \ell = 32 \tag{4.24}$$

4.5 Codes and Standards

The design follows aviation, software-engineering and security standards where they apply to a laboratory system. Table 4.4 lists them and how they are used.

Table 4.4 Codes and Standards Followed

Standard	Use in SkyControl
ICAO Annex 11, Air Traffic Services	Objectives of ATC used to define the gaps
ICAO Doc 4444, PANS-ATM	Separation minima (3/5 NM, 1,000 ft), readback requirements, emergency handling
ICAO Doc 9432, Manual of Radiotelephony	Phraseology generated by the communication assistant
ICAO Annex 10 Vol. II	Communication procedures, numbers and NATO alphabet
EUROCONTROL STCA specification	Look-ahead and alerting behaviour of the safety-net baseline
NIST SP 800-63B	Password length, no composition tricks beyond letters+digits, throttling
RFC 8018 (PKCS #5 v2.1)	PBKDF2 key derivation for password storage
OWASP Top 10 (2021) and ASVS	Session management, CSRF, security headers, access control, logging
ISO/IEC 25010	Quality attributes used for the non-functional requirements
PEP 8 and ruff	Python code style and static checks
Semantic versioning	Application version 6.0.0
OpenSky Network terms of use	Data used for non-commercial research and teaching with attribution

4.6 Constraints, Alternatives and Trade-offs

The main constraints were the data (one region, 90 minutes, no clearances, no fuel or emergency fields), the absence of controllers for user trials, the need to run on an ordinary laptop, and the requirement that the result be demonstrable in a browser. Table 4.5 summarises the main design alternatives and why the chosen option was preferred.

Table 4.5 Design Alternatives and Trade-offs

Decision	Alternatives considered	Choice and reason
Trajectory predictor	LSTM / transformer; physics model with intent	Gradient-boosted residuals: accurate on tabular features, fast on CPU, degrades to straight line
Conflict probe	Naive 5-min probe; probabilistic probe	Intent-aware probe: the naive probe gave 522.7 alerts/h, unusable
Anomaly detection	Supervised classifier; autoencoder	Isolation forest + rule: no labels exist, fast, transparent with the rule
Hold prediction	Gradient boosting	Logistic regression: only 174 samples, interpretable coefficients
Automation control	Learned policy	Rule policy: auditable, predictable, matches the levels-of-automation literature
Verification	Sampled checks	Exact interval geometry: no gaps between samples
Database	PostgreSQL	SQLite WAL: zero administration for a lab; migration path documented
Front end	React / Angular with a build step	Vanilla ES modules: no build, strict CSP, easy to read
3D library	CesiumJS; deck.gl	three.js: small, flexible, vendored locally
Speech	Cloud ASR	Browser speech API + typed input: no external service or keys
Deployment	Virtual machine; Kubernetes	Single Docker container + Compose: right size for the project

Two trade-offs deserve emphasis. First, accuracy against honesty: the intent-aware probe uses the next observed level-off as a stand-in for the cleared level. This makes the probe look ahead realistically, but it is optimistic, so the report presents it as what a digital clearance record would make possible rather than as a measured operational result. Second, automation against authority: a higher automation level would always be faster in the workflow model, but SkyControl deliberately drops to ADVISORY or MANUAL in critical, uncertain or degraded situations and accepts the extra time, because the literature on automation bias shows that this is where blind automation is most dangerous.

CHAPTER 5
IMPLEMENTATION

This chapter describes how the design was implemented: the project structure, the back end, the login portal, the analytics pipeline, the scenario lab, the 3D front end, the communication assistant, the tests and the deployment artefacts. Short code excerpts illustrate the key parts; longer samples are in Appendix A.

5.1 Project Structure

The repository is organised by layer, as shown in the listing below.

```
SkyControl/
├── backend/
│   ├── app.py              FastAPI app, security headers, static files, health
│   ├── auth.py             PBKDF2 hashing, login, lockout, sessions, CSRF, roles
│   ├── database.py         SQLite connection (WAL), schema.sql, audit helper
│   ├── seed.py             accounts from environment, runways, dataset register
│   ├── routes/             auth, operations, lab, airspace, insight, data
│   ├── core/               scenarios, separation, comms, automation, workflow, gaps
│   └── analytics/          recording, surveillance, prediction, conflicts, holding,
│                           hold_model, anomaly, callsigns, montecarlo, emergency,
│                           stats, live, pipeline
├── frontend/               index.html, css/, js/ (pages/, scene3d.js), vendor/
├── data/                   OpenSky recording (.json.gz), reference data, snapshots
├── tests/                  93 pytest tests
├── scripts/                build_analysis.py, smoke_test.sh, load_test.py
├── jenkins/                Jenkins controller image with Docker CLI
├── jira/                   backlog CSV, sprint plan
├── docs/                   architecture, gap analysis, guides, evidence, report
├── Dockerfile, docker-compose.yml, Jenkinsfile, .github/workflows/ci.yml
└── run.py, setup_accounts.py, start_windows.bat, start_linux_mac.sh
```

5.2 Back End

The back end is a FastAPI application. On start-up a lifespan handler creates the schema, seeds the accounts from environment variables (there are no default passwords), the runways and the dataset register, and starts a background thread that builds or loads the cached real-data analysis, so that the login page is available immediately. Middleware adds gzip compression and the security headers. The static front end is served from the same origin, which allows a strict Content-Security-Policy without any cross-origin requests.

Routes are grouped into six routers. Each route declares its input with type hints and Pydantic validation, obtains the caller's identity with `require(request, supervisor, mutation)`, which checks the session, the CSRF token for mutations and the supervisor role where needed, performs its work inside a database transaction, and writes an audit event.

5.3 Login Portal and Attempt Storage

The login function is the heart of requirement 2. It runs inside an immediate transaction so that concurrent attempts cannot bypass the lockout, determines a precise reason for the outcome, always stores the attempt and only then decides the response. The excerpt below is taken from `backend/auth.py`.

```
def login(username, password, ip, user_agent=''):
    with connect() as db:
        db.execute('BEGIN IMMEDIATE')
        failures = db.execute('SELECT COUNT(*) FROM auth_attempts WHERE username=? '
                              'AND successful=0 AND created_at>=?',
                              (username, cutoff)).fetchone()[0]
        ip_failures = ...                       # same count per IP address
        user = one(db.execute('SELECT * FROM users WHERE username=?', (username,)))
        valid = bool(user and user['active'] and check_password(password,
user['password_hash']))
        if not username:                    reason = 'missing_username'
        elif ip_failures >= LOGIN_IP_LIMIT: reason = 'ip_rate_limited'
        elif failures >= LOGIN_LOCKOUT_FAILURES: reason = 'locked_out'
        elif valid:                         reason = 'success'
        elif user and not user['active']:   reason = 'account_disabled'
        elif user:                          reason = 'wrong_password'
        else:                               reason = 'unknown_username'
        db.execute('INSERT INTO auth_attempts (username, ip, user_agent, successful, '
                   'reason, created_at) VALUES (?,?,?,?,?,?)',
                   (username, ip, user_agent[:200], int(reason == 'success'), reason, utcnow()))
        if reason != 'success':
            db.commit()
            raise HTTPException(429 if locked else 401,
                                {'message': 'Wrong username or password',
                                 'attempts_remaining': remaining})
        ...                   # session token (stored as SHA-256) and CSRF token
```

In the browser, the login page shows the server message and the number of attempts remaining (Fig. 5.1). The security page lists every attempt with the username tried, the IP address, the browser, the outcome and the reason, can filter to failed attempts only, and charts the outcomes; it is shown in Chapter 7.

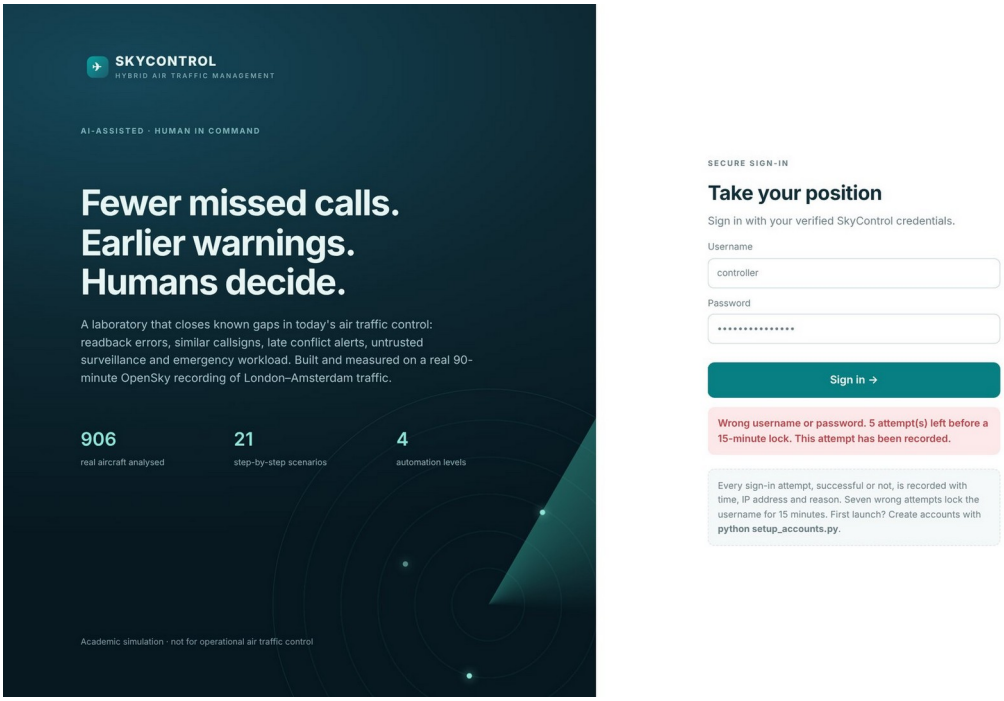


Fig. 5.1 Login Portal Rejecting a Wrong Password and Showing the Attempts Remaining

5.4 Analytics Pipeline

The module `analytics/pipeline.py` runs all analyses in order: recording integrity, surveillance (ghosts, gaps and coasting), trajectory prediction (training and held-out evaluation), conflict alerting with the four methods, holding and the AMAN estimate, hold prediction, anomaly detection with injected emergencies, callsign similarity, the communication model, 60 emergency injections and the scenario benchmark. The complete analysis takes about 45 s on four CPU cores. The result is a JSON document of about 170 KB plus the trained predictor (about 4 MB); both are cached by dataset hash and analysis version. In the Docker image the cache is built during the image build, so a container starts with the analysis ready.

The live views of the 3D replay (`analytics/live.py`) reuse the same models. For a chosen moment they compute the intent-aware predicted conflicts, duplicates, similar callsigns sharing airspace, aircraft holding and tracks being coasted, and the five-minute straight-line and learned predicted positions of every aircraft. Results are cached per recorded frame with a least-recently-used cache, which brings the median response time of a frame to 23 ms.

The core of the vectorised conflict probe is shown below; it compares all nearby pairs at all future sample times at once.

```
px, py, pz = _predicted_paths(rec, eligible, ts, cfg, levels, learned)   # shape (aircraft, samples)
```

```
i, j = np.triu_indices(len(eligible), 1)                                  # all pairs
near = np.hypot(rec.x[eligible[i]] - rec.x[eligible[j]],
                rec.y[eligible[i]] - rec.y[eligible[j]]) < 80          # pre-filter 80 NM
i, j = i[near], j[near]
thr = horizontal_threshold(rec.alt[eligible[i]], rec.alt[eligible[j]])[:, None]   # 3 or 5 NM
inside = (np.hypot(px[i] - px[j], py[i] - py[j]) < thr) & (np.abs(pz[i] - pz[j]) < V_THRESHOLD)
for k in np.flatnonzero(inside.any(axis=1)):
    first = int(np.argmax(inside[k])) * SAMPLE_S                          # seconds to predicted conflict
```

5.5 Scenario Lab Engine

The scenario engine (`core/scenarios.py`) holds the specification of 21 scenarios in eight categories. A builder creates the aircraft (with callsigns taken from aircraft airborne in the real recording when the user chooses that source), the hazard, the comparator plan and the SkyControl plan. Both plans are verified with the exact interval method, outcome checks are computed (aircraft conflicts, runway conflicts, hazard or closure violations and fuel deadline misses), and the ten pipeline steps, the context-switch log, the message log with readback verification and the workflow comparison are generated. Table 5.1 shows what each step does.

Table 5.1 The Ten Steps Used to Handle Every Scenario

Step	Actor	What happens
1 DETECT	Monitors	Probe, integrity monitor, readback verifier or emergency monitor detects the situation
2 ASSESS	SkyControl	Severity, aircraft involved and the comparator's outcome are computed
3 CONTEXT SWITCH	Automation policy	Level and approver chosen; reasons shown and logged
4 PLAN	Planner	Levels, reroutes, sequence or holding computed
5 VERIFY	Safety checker	Exact all-pairs separation, runway, hazard and deadline checks
6 AUTHORISE	Human	Run pauses at T+20 s for approval, rejection or override with a reason
7 COMMUNICATE	Controller + datalink	Urgent voice with verified readback; routine by datalink
8 EXECUTE	Aircraft	Manoeuvres flown in the replay
9 MONITOR	SkyControl	Conformance monitored until all aircraft reach their final state
10 CLOSE	SkyControl	Outcome recorded, alert resolved and audit trail written

Runs are stored in the database so that a run started by a controller is kept when a supervisor logs in to approve it. The replay clock advances on the server and stops at the approval gate until a user with the required role acts; the browser animates the aircraft between server updates.

5.6 Three-Dimensional Front End

The front end is a single-page application written as ES modules without a build step. `main.js` handles login, navigation and the session; each page is a module under `js/pages/`. The 3D views use a shared `Airspace3D` class (`js/scene3d.js`) built on three.js: aircraft are instanced meshes coloured by altitude or state, with trails, vertical stems to the ground, text labels, airports, holding fixes, a coastline, predicted conflict lines and prediction segments. Orbit, pan and zoom controls and preset views (3D, top, side) are provided. Chart.js renders the statistics. three.js and Chart.js are vendored locally with their licences so that the CSP can forbid external scripts.

Table 5.2 Pages of the Application

Page	Content
Command centre	Key results, live counts, failed logins in the last 24 hours, open alerts
ATC gaps and fixes	Gap catalogue G1-G9 with evidence and links to scenarios
3D airspace	Real traffic replay with layers, live monitor and emergency injection
Scenario lab	21 scenarios with 3D replay, ten steps, approvals, messages and comparison
Communication assistant	Clearance builder, phraseology, readback verification, similar callsigns
AI models and automation	Model cards and the context-switch simulator
Statistical analysis	Eight tabs: summary, conflicts, surveillance, communication, holding, emergencies, scenarios, dataset
Operations	Flights, runways, runway scheduling, priority queue, alerts
Data and provenance	Dataset register, snapshots, import, feed health
Security and audit	Login attempts, outcomes chart, users, password change, audit trail

5.7 Communication Assistant

The communication assistant lets the user build a clearance from structured fields, shows the ICAO phraseology and the frequency occupancy it would need, and verifies a readback typed by the user or spoken through the browser's speech-recognition interface. The verdict is shown with the field-by-field comparison and the advice for the controller. A log of checks is stored in `comm_checks`, and a similarity panel warns of similar callsigns on the frequency.

5.8 Testing

Testing used three levels. Unit and integration tests (pytest) cover every module and route: authentication and attempt storage, operations, all 21 scenarios, the lab API and approval gates, communication and automation, the real-data analytics

and the application shell. API tests use FastAPI's test client with a temporary database. Browser tests with Playwright and headless Chromium exercise every page and the interactive flows, and a container smoke test checks the deployed image. The test fixture sets the account passwords through environment variables, exactly as in production.

```
def test_wrong_password_is_rejected_and_stored(api, supervisor):
    r = api.post('/api/auth/login',
                 json={'username': 'controller', 'password': 'not-the-password1'})
    assert r.status_code == 401
    assert r.json()['error'] == 'Wrong username or password'
    assert r.json()['details']['attempts_remaining'] == 6
    attempts = api.get('/api/auth/attempts', headers=supervisor).json()['attempts']
    failed = [a for a in attempts
              if a['username'] == 'controller' and not a['successful']]
    assert len(failed) == 1 and failed[0]['reason'] == 'wrong_password'


def test_lockout_after_seven_failures_even_with_correct_password(api, supervisor):
    for _ in range(7):
        r = api.post('/api/auth/login', json={'username': 'controller', 'password': 'WRONG-pass-1'})
        assert r.status_code == 401
    blocked = api.post('/api/auth/login',
                       json={'username': 'controller', 'password': CONTROLLER_PASSWORD})
    assert blocked.status_code == 429
```

5.9 Containerisation

The Dockerfile has three stages. The `base` stage installs the runtime dependencies and copies the application and data. The `test` stage adds the development dependencies and the tests and runs ruff and pytest with JUnit and coverage reports; Jenkins builds this stage, so the build agent needs only Docker. The `runtime` stage pre-computes the analysis, creates an unprivileged user, declares the database volume and a health check, and starts the server.

```
FROM python:3.12-slim AS base
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY backend backend
COPY frontend frontend
...
FROM base AS test
RUN pip install -r requirements-dev.txt
COPY tests tests
CMD ["sh", "-c", "ruff check . && python -m pytest --junitxml=reports/junit.xml ..."]

FROM base AS runtime
ENV SKYCONTROL_CACHE_DIR=/app/cache
RUN python scripts/build_analysis.py          # analysis ready at start-up
RUN useradd --create-home --uid 10001 skycontrol \
    && mkdir -p /app/data/db && chown -R skycontrol /app/data /app/cache
USER skycontrol
EXPOSE 5000
VOLUME ["/app/data/db"]
HEALTHCHECK CMD python -c "import urllib.request; \
    urllib.request.urlopen('http://127.0.0.1:5000/api/health')"
CMD ["python", "run.py"]
```

`docker-compose.yml` runs the image with the passwords taken from an `.env` file (an example file is provided and the real one is excluded from Git), maps port 5000 and mounts a named volume for the database.

5.10 Continuous Integration and Delivery

The `Jenkinsfile` is a declarative pipeline with five stages: Checkout, Lint and tests (in the test image, with JUnit and coverage reports published), Build image (tagged with the build number and `latest`), Smoke test (a temporary container on port 5099 checked by `scripts/smoke_test.sh` using credentials from the Jenkins credentials store) and Deploy (replace the running container, keep the database volume, wait for the health check). The pipeline is described in Chapter 6. A GitHub Actions workflow (`.github/workflows/ci.yml`) runs lint, tests and a Docker build on every push and pull request.

CHAPTER 6
SCHEDULE, TASKS AND MILESTONES

The project was run as a Scrum project of six two-week sprints between 3 August and 23 October 2026. This chapter describes the method, the backlog and sprints as they are set up in Jira, the version-control workflow on GitHub, the Jenkins pipeline, the Docker deployment and the milestones.

6.1 Agile Methodology

Scrum (Schwaber & Sutherland, 2020) was chosen because the requirements were expected to change as results on real data came in, which is exactly what happened: the conflict probe, the emergency planner and the readback verifier were all redesigned after their first evaluation. Each sprint started with planning, in which stories were pulled from the ordered backlog up to the team's velocity, had short daily stand-ups, and ended with a review, in which the increment was demonstrated in the running application, and a retrospective that produced one improvement action. The definition of done required that the code was merged through a pull request, that lint and all tests passed in CI and Jenkins, that the acceptance criteria were shown in the application and that the documentation was updated.

Table 6.1 Scrum Roles and Ceremonies

Item	How it was used
Product owner	Owns the order of the backlog: login and data first, then analytics, scenarios, UI, DevOps
Scrum master	Runs the ceremonies and removes blockers (dataset access, Docker set-up)
Development team	Designs, implements and tests the stories; everyone writes tests for their own stories
Guide	Reviews the increment at sprint reviews and advises on scope and evaluation
Sprint planning	Stories pulled up to the average velocity of about 34 points
Daily stand-up	15 minutes; Jira board updated
Sprint review	Increment demonstrated in the running app
Retrospective	One improvement per sprint, tracked as an issue labelled retro

6.2 Product Backlog and Epics

The backlog contains 47 user stories grouped into nine epics with a total of 205 story points (Table 6.2). Story points were estimated with planning poker on the

Fibonacci scale. The largest epics are the real-data analytics and the AI models, which reflects the emphasis of the project on evidence.

Table 6.2 Epics of the Product Backlog

Epic	Title	Story points
E1	Secure login portal and user management	17
E2	Real data pipeline (OpenSky recording)	13
E3	Operations management	15
E4	Real-data analytics and statistics	34
E5	AI models and adaptive automation	34
E6	Scenario lab (21 step-by-step scenarios)	32
E7	3D web application and UX	27
E8	DevOps and quality	23
E9	Documentation, report and demonstration	10
	Total	**205**

6.3 Sprint Plan

Table 6.3 gives the six sprints with their dates, goals and committed points.

Table 6.3 Sprint Plan

Sprint	Dates	Sprint goal	Points
Sprint 1	08-03 to 08-14	Secure login portal, project skeleton and real data ingestion	25
Sprint 2	08-17 to 08-28	Operations management and surveillance-integrity analytics	26
Sprint 3	08-31 to 09-11	AI models: learned trajectory prediction, conflict probe, holding, anomaly detection	32
Sprint 4	09-14 to 09-25	Communication safety, context-switch policy and the scenario engine	47
Sprint 5	09-28 to 10-09	3D web app: airspace replay, scenario lab, statistics and emergency injection	43
Sprint 6	10-12 to 10-23	DevOps (Docker, Jenkins, GitHub Actions), testing, report and release	32

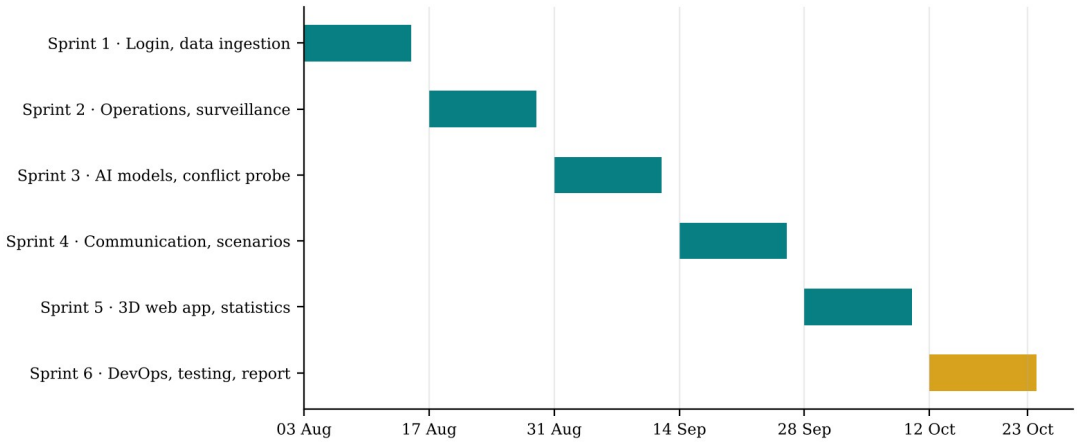


Fig. 6.1 Project Timeline of the Six Sprints

Fig. 6.1 shows the timeline. Sprints 1 to 5 delivered the application increment; Sprint 6 delivers the DevOps pipeline, the test suite, this report and the release, and is the sprint that is run live on the Jira board.

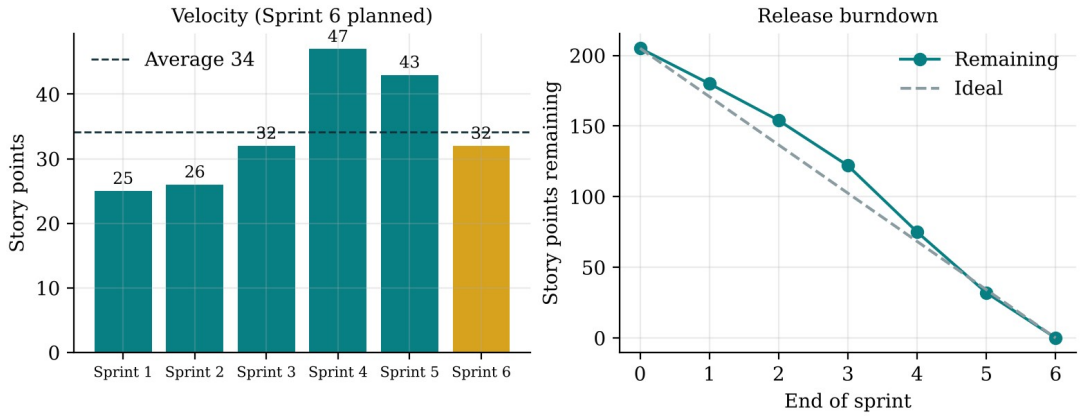


Fig. 6.2 Velocity per Sprint and Release Burndown (from the Sprint Plan)

The velocity chart in Fig. 6.2 shows that the team committed 25 and 26 points in the first two sprints, while the stack and the data pipeline were being established, and then 32 to 47 points once the architecture was stable. Sprint 4 was the largest because the communication functions and the scenario engine were closely coupled and were planned together. The release burndown reaches zero at the end of Sprint 6.

6.4 Sprint Backlogs

Table 6.4 lists every story with its epic, sprint and estimate, as imported into Jira.

Table 6.4 User Stories by Sprint

Sprint	Epic	User story	Pts
S1	E1	Login with verified credentials	5
S1	E1	Store every login attempt in the backend	3
S1	E1	Lock a username after 7 failures in 15 minutes	3
S1	E1	Sessions, CSRF protection and logout	3
S1	E2	Load the London-Amsterdam OpenSky recording	5
S1	E2	Dataset register with provenance	3
S1	E8	Project skeleton, linting and test harness	3
S2	E2	Import and live-capture OpenSky snapshots	3
S2	E3	Flight CRUD, search and cancellation	5
S2	E3	Runway allocation with conflict blocking	5
S2	E3	Runway calendar with alternatives	3
S2	E3	Priority queue and alert centre	2
S2	E4	Detect duplicate (ghost) targets	5
S2	E4	Coast dropped tracks and measure the error	3
S3	E4	Compare STCA-style alerting with the SkyControl probe	8
S3	E4	Detect holding and estimate AMAN linear holding	5
S3	E4	Similar callsigns sharing airspace	3

Sprint	Epic	User story	Pts
S3	E5	Learned trajectory predictor	8
S3	E5	Hold predictor (30-80 NM before the stack)	3
S3	E5	Emergency anomaly detector	5
S4	E4	Communication Monte Carlo with sensitivity analysis	5
S4	E5	Adaptive automation (context switch) policy	5
S4	E5	Readback verifier	8
S4	E6	Scenario engine with exact conflict verification	8
S4	E6	Communication scenarios	5
S4	E6	Surveillance and automation scenarios	5
S4	E6	Emergency and runway scenarios	5
S4	E6	Ten-step handling pipeline per scenario	3
S4	E6	Workflow model: conventional vs hybrid	3
S5	E1	Supervisor user management and attempt log page	3
S5	E4	Statistical analysis page	5
S5	E5	Emergency injection into real traffic	5
S5	E6	Approval gate and role rules	3
S5	E7	3D airspace replay of the recording	8
S5	E7	Scenario lab page with step-by-step view	8
S5	E7	Communication assistant page	5
S5	E7	Command centre and gap analysis pages	3
S5	E7	AI models page with policy simulator	3
S6	E2	OpenSky multi-frame recorder script	2
S6	E8	Automated test suite	5
S6	E8	Multi-stage Dockerfile and compose	5
S6	E8	Jenkins pipeline	5
S6	E8	GitHub repository and Actions CI	3
S6	E8	Smoke and load tests	2
S6	E9	Project report (VIT template)	5
S6	E9	Guides: GitHub, Jenkins, Jira, Docker	3
S6	E9	Faculty demonstration script	2

6.5 Jira Configuration

The backlog is provided as a CSV file (`jira/skycontrol_jira_import.csv`, 56 issues: 9 epics and 47 stories) for Jira Cloud's external-system import, together with a step-by-step guide (`docs/JIRA_GUIDE.md`). A company-managed Scrum project with key SKY is created, the six sprints are created first with exactly the names in the CSV, and the import maps Issue Id, Parent Id (linking stories to epics), Issue Type, Summary, Description, Priority, Story Points, Sprint, Labels and Status. Sprints 1 to 5 are imported as Done, so velocity reports are meaningful, and Sprint 6 is imported as To Do so that it can be started and run on the board to produce a genuine burndown chart.

- **Board workflow:** To Do → In Progress → In Review → Done.

- **Traceability:** branches and commits start with the issue key (for example `feature/SKY-48-jenkins-pipeline`, `SKY-48 Add container smoke test`); the GitHub for Jira app shows branches, commits, pull requests and builds in each issue's Development panel.
- **Reports used:** backlog, active sprint board, sprint burndown, velocity chart, timeline of the nine epics.

6.6 Version Control with GitHub

The source is kept in a GitHub repository. `docs/GITHUB_GUIDE.md` explains how to create the repository, push the project for the first time and work with branches and pull requests. The `.gitignore` excludes the virtual environment, caches, the database and the `.env` file, so that no password or generated data is ever committed.

Table 6.5 Git Workflow Conventions

Convention	Rule
Main branch	`main` is always releasable; protected, changes only through pull requests
Feature branches	`feature/SKY-<n>-short-name`, one per story
Commits	Start with the Jira key, imperative mood: `SKY-12 Store every login attempt`
Pull requests	Reviewed by the other team member; CI (GitHub Actions) must be green
Releases	Tagged with semantic versions, e.g. `v6.0.0`
Secrets	Never committed; `.env.example` documents the variables

```
git init
git add .
git commit -m "SKY-1 Initial SkyControl 6.0.0"
git branch -M main
git remote add origin https://github.com/<user>/skycontrol.git
git push -u origin main
git checkout -b feature/SKY-48-jenkins-pipeline        # one branch per story
```

6.7 Jenkins Pipeline

The `Jenkinsfile` defines the continuous integration and delivery pipeline (Table 6.6). Python runs only inside containers, so the Jenkins agent needs nothing but the Docker CLI and curl. The account passwords used by the smoke test and the deployment are stored as Jenkins secret-text credentials and are masked in the log. A ready-to-run Jenkins controller with the Docker CLI is provided in `jenkins/`.

Table 6.6 Stages of the Jenkins Pipeline

Stage	What it does	Fails when
Checkout	Checks out the repository and locates the application folder	Repository unreachable
Lint and tests	Builds the test image, runs ruff and 93 pytest tests, publishes JUnit and coverage	Any lint error or failing test
Build image	Builds the runtime image tagged with the build number, Git hash and latest	Docker build error
Smoke test container	Starts the image on port 5099 and runs scripts/smoke_test.sh	Health, login, wrong-credential or attempt-log check fails
Deploy	Replaces the running container on port 5000, keeps the database volume, waits for health	Container not healthy within 60 s

The pipeline was executed end to end with Jenkins (jenkinsfile-runner with the Pipeline, JUnit and Credentials Binding plug-ins). Build 9 passed every stage. The excerpt below is from the console log kept in `docs/evidence/jenkins_console_build9.txt`; the JUnit and coverage reports of the same build are stored next to it.

```
+ docker build --target test -t skycontrol-test:9 .
+ docker run --name skycontrol-tests skycontrol-test:9
All checks passed!
TOTAL                              3722    390    90%
93 passed, 1 warning in 122.21s (0:02:02)
[Pipeline] { (Build image)
[Pipeline] { (Smoke test container)
Masking supported pattern matches of $SKYCONTROL_CONTROLLER_PASSWORD
  or $SKYCONTROL_SUPERVISOR_PASSWORD
+ sh scripts/smoke_test.sh http://127.0.0.1:5099 supervisor ****
Frontend: OK
Wrong credentials rejected: OK (401)
Login with verified credentials: OK
Scenario lab: readback_error plan verified
Failed login stored in the backend: OK
Smoke test passed
[Pipeline] { (Deploy)
+ docker run -d --name skycontrol --restart unless-stopped -p 5000:5000
    -v skycontrol-db:/app/data/db ... skycontrol:latest
CONTAINER ID  IMAGE              STATUS                              PORTS
e8f3155d83ae  skycontrol:latest  Up 4 seconds (health: starting)  0.0.0.0:5000->5000/tcp
SkyControl build 9 passed every stage.
Finished: SUCCESS
```

6.8 Deployment with Docker

Deployment uses the multi-stage Dockerfile of Section 5.9. For a manual deployment, the team copies `.env.example` to `.env`, sets the two account passwords and runs Docker Compose. The database lives on the named volume `skycontrol-db`, so updating the application (rebuilding and recreating the container) keeps users, attempts, runs and the audit trail. `docs/DOCKER_GUIDE.md` also covers logs, backup and restore of the volume, and troubleshooting.

```
cp .env.example .env                # then set the two account passwords in .env
docker compose up -d --build        # build and start on http://localhost:5000
```

```
docker compose ps                    # STATUS shows (healthy) after the health check
docker compose logs -f skycontrol    # follow the logs
docker compose down                  # stop (the database volume is kept)
```

6.9 Milestones

Table 6.7 Project Milestones

Milestone	Date	Deliverable	Status
M1 Secure portal and data	14 Aug 2026	Login portal with attempt log; recording loaded and registered	Done
M2 Operations and integrity	28 Aug 2026	Flights, runways, alerts; ghost targets and coasting measured	Done
M3 AI models	11 Sep 2026	Trajectory predictor, conflict probe, holding, anomaly detector	Done
M4 Communication and scenarios	25 Sep 2026	Readback verifier, context-switch policy, 21 scenarios	Done
M5 3D application	09 Oct 2026	Airspace replay, scenario lab, statistics, emergency injection	Done
M6 Pipeline and release	23 Oct 2026	Docker, Jenkins, GitHub, tests, report, demonstration	Pipeline validated; release at sprint end

CHAPTER 7
RESULTS AND DISCUSSION

This chapter presents the results of SkyControl on the real recording, the injected emergencies and the scenarios, compares them with current ATC practice, reports the software tests and performance, shows the user interface and gives a cost analysis. It closes with a discussion of what the results mean and of their limits.

7.1 Evaluation Design

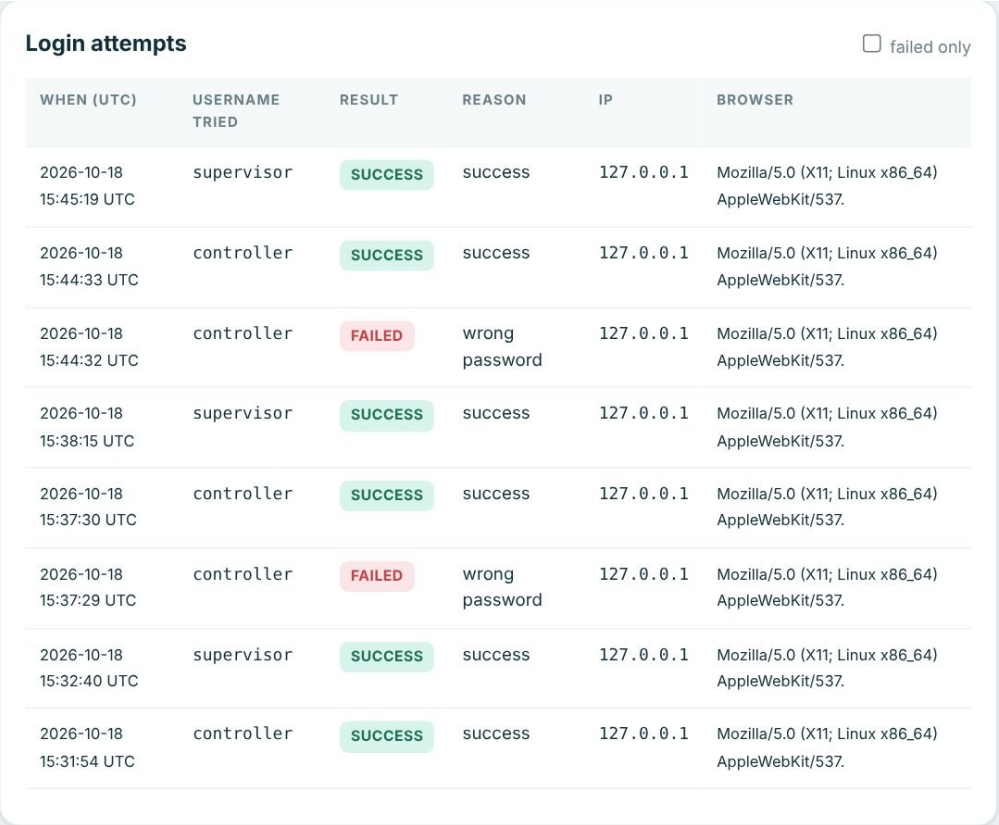
Each result in this chapter carries one of three labels. **Measured** results are computed from the real recording; learned models are always evaluated on data they did not see (the last 30 minutes for the trajectory predictor and the anomaly detector, the last 35 minutes for the hold predictor). **Modelled** results depend on stated assumptions (error rates, task times, fuel burn) and come with a sensitivity analysis. **Scenario** results come from constructed exercises; they demonstrate behaviour and are not statistical evidence. Paired comparisons use the one-sided Wilcoxon signed-rank test and a 95% bootstrap confidence interval of the mean improvement (Section 4.4.14).

Table 7.1 Evaluation Plan per Gap

Gap	Comparison	Cases	Label
G3	Straight line vs learned predictor (60/120/300 s)	48,632 held-out points	Measured
G3	STCA-style, naive, filtered and intent-aware alerting	90 min, 8 real encounters	Measured
G4	Frozen vs dead-reckoned track after dropouts	196 dropouts	Measured
G1	Readback verifier detection and false alarms	3,000 readbacks, clean and noisy	Measured (generated corpus)
G1, G2	Voice-only vs hybrid communication	4,000 sector-hours	Modelled
G2	Similar callsigns sharing airspace	906 aircraft	Measured
G5	Holding, linear-holding estimate, hold predictor	27 holds, 174 arrivals	Measured + modelled
G7	Rule vs ML vs hybrid vs CLAM detection	81 injected events	Measured on real tracks
G6	Emergency planning in real traffic; workflow	60 injections	Real traffic + modelled
G8, G9	Comparator vs SkyControl in every scenario	21 scenarios	Scenario + modelled

7.2 Login Portal and Security

The login portal meets requirement 2. A wrong password or an unknown username returns HTTP 401 with the message "Wrong username or password" and the number of attempts remaining (Fig. 5.1). Every attempt is stored with the username tried, the IP address, the browser, the success flag and one of seven reasons (success, wrong_password, unknown_username, locked_out, account_disabled, ip_rate_limited, missing_username); passwords are never stored. Fig. 7.1 shows the attempt log on the security page during testing, including the failures created by the demonstration and the automated browser runs.



Login attempts

☐ failed only

WHEN (UTC)	USERNAME TRIED	RESULT	REASON	IP	BROWSER
2026-10-18 15:45:19 UTC	supervisor	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:44:33 UTC	controller	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:44:32 UTC	controller	FAILED	wrong password	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:38:15 UTC	supervisor	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:37:30 UTC	controller	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:37:29 UTC	controller	FAILED	wrong password	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:32:40 UTC	supervisor	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.
2026-10-18 15:31:54 UTC	controller	SUCCESS	success	127.0.0.1	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.

Fig. 7.1 Login Attempt Log with Outcome, Reason, IP Address and Browser

Nine automated tests verify this behaviour: the uniform message, the stored reason for wrong passwords and unknown usernames, that no password text appears in any stored data, the lockout after seven failures even when the eighth attempt uses the correct password, that only supervisors can read the log, CSRF protection, user management, password change and logout. The container smoke test in Jenkins repeats the essential check against the deployed image: a wrong password is rejected with 401 and the failed attempt is found in the back end.

7.3 Trajectory Prediction (G3)

Table 7.2 and Fig. 7.2 compare the learned predictor with straight-line extrapolation on held-out traffic. The improvement grows with the horizon, because the further ahead the prediction, the more the aircraft's route structure matters. At five minutes the mean error falls from 6.845 NM to 4.482 NM (34.5% lower, 95% CI of the improvement 2.26 to 2.47 NM), and the 95th-percentile error from 23.452 NM to 14.233 NM. All three differences are significant ($p < 0.001$).

Table 7.2 Mean Position Error of Straight-Line and Learned Prediction on Held-Out Traffic

Horizon	n	Straight line (NM)	Learned (NM)	Improvement	95% CI (NM)	p
60 s	17,994	0.555	0.449	19.1%	0.09 to 0.12	< 0.0001
120 s	16,873	1.635	1.267	22.5%	0.34 to 0.40	< 0.0001
300 s	13,765	6.845	4.482	34.5%	2.26 to 2.47	< 0.0001

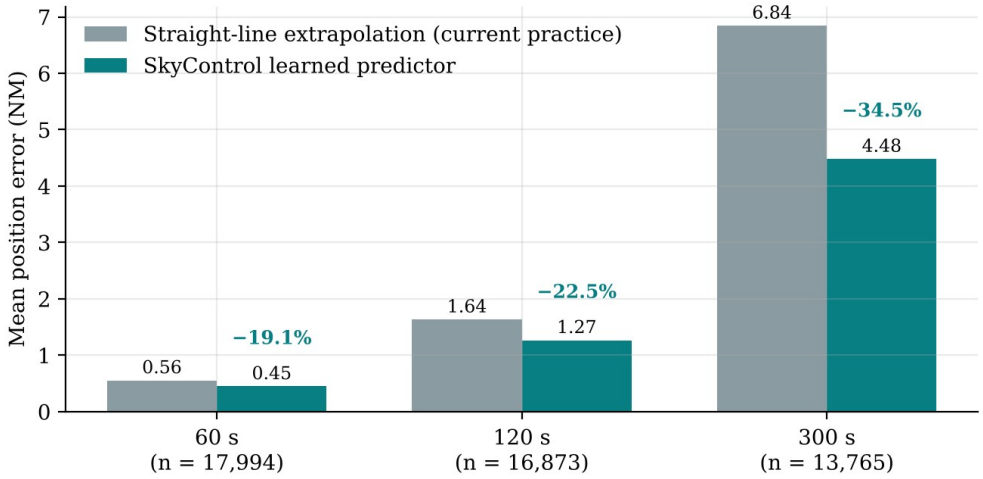


Fig. 7.2 Prediction Error by Horizon: Straight Line versus Learned Predictor

The gain is concentrated where straight lines fail. For the 1,683 test points where the aircraft was turning, the five-minute error falls from 15.283 NM to 7.452 NM, roughly half. The median error at short horizons is almost unchanged (0.192 NM against 0.215 NM at 60 s): for an aircraft flying straight and level the straight line is already right, and the learned correction correctly leaves it alone. Fig. 7.3 shows that the improvement holds in every altitude band, and is largest in the terminal area below 20,000 ft where arrivals turn onto the stacks and final approach.

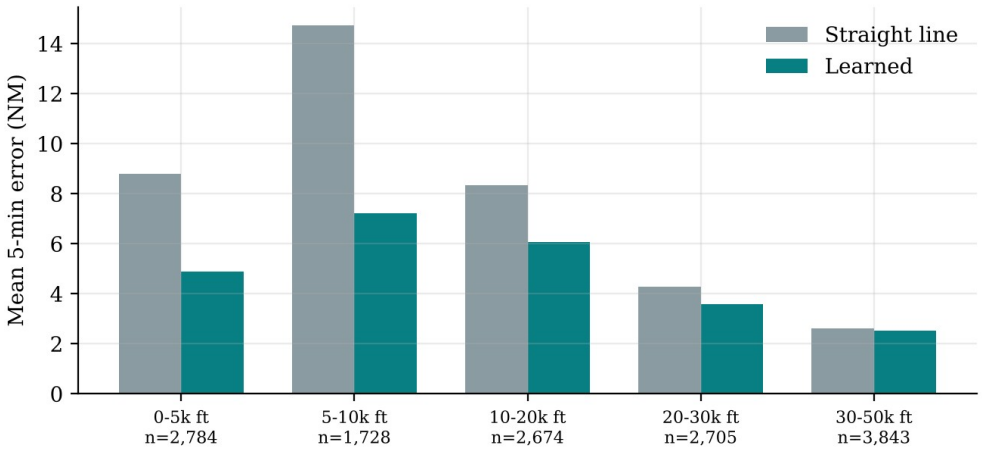


Fig. 7.3 Five-Minute Prediction Error by Altitude Band

Permutation importance (Fig. 7.4) shows that altitude and speed carry most of the information: together they tell the model whether an aircraft is an arrival about to turn, a departure still accelerating or an overflight in cruise. Position (x, y) adds the local route structure.

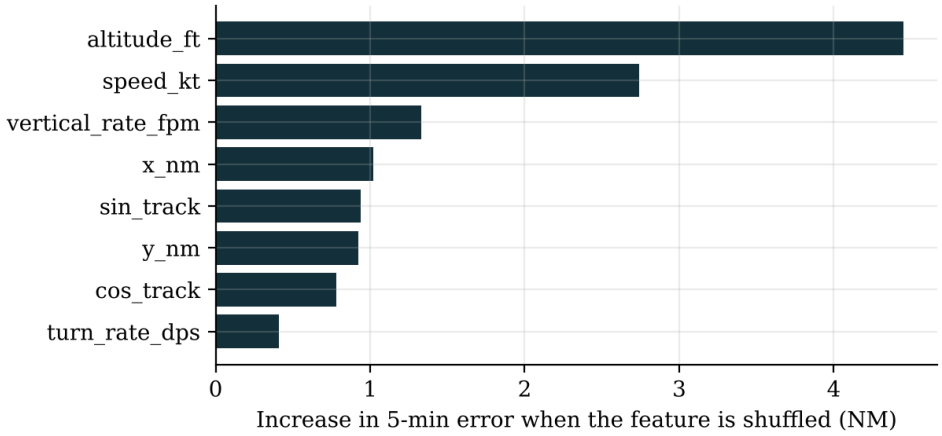


Fig. 7.4 Permutation Importance of the Predictor Features (Five-Minute Horizon)

7.4 Conflict Alerting (G3)

In the 90 minutes there were 9 real encounters inside the screening volume, of which 8 occurred after the five-minute warm-up and could be evaluated; most are legitimate proximity (in-trail sequencing, climbs and descents through traffic, arrival sequencing to Heathrow). Table 7.3 compares the four alerting methods on the same traffic.

Table 7.3 Alerting Methods on the Real Recording

Method	Look-ahead	Alerts per hour	Encounters alerted	Alerted ≥30 s ahead	Mean warning (s)	Nuisance share
STCA-style safety net	120 s	148.7	5 of 8	4	33.4	96.0%
Naive 5-min probe	300 s	522.7	5 of 8	4	43.4	98.5%
SkyControl filtered probe	300 s	204.7	5 of 8	4	43.4	97.1%
SkyControl intent-aware probe	300 s	145.3	5 of 8	5	47.1	95.0%

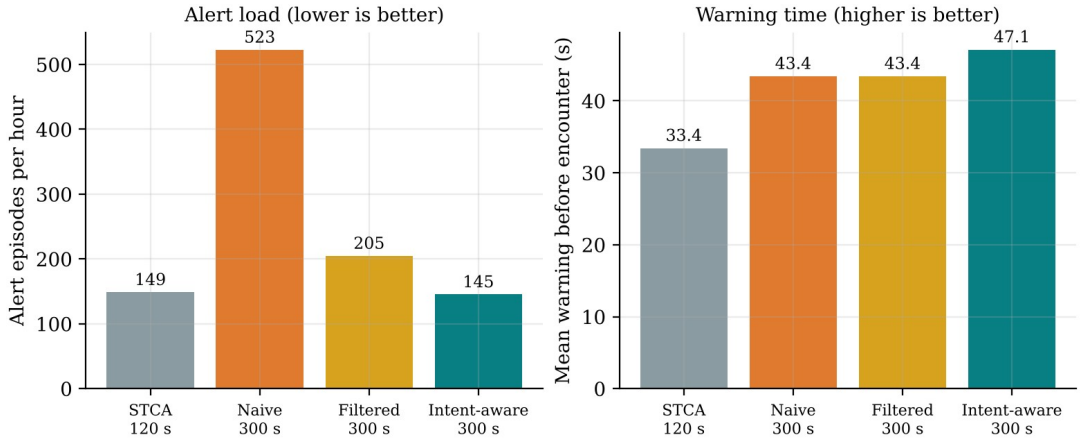


Fig. 7.5 Alert Load and Warning Time of the Four Alerting Methods

The result shows why simply looking further ahead does not work, and how the design was improved. The naive five-minute probe raised 522.7 alerts per hour, 3.5 times the safety net, almost all of them nuisance alerts. Merging duplicate targets, recognising level flight and adding hysteresis brought this down to 204.7. The intent-aware probe, with the learned trajectories and cleared levels, brought it to 145.3 per hour (72.2% fewer than the naive probe), slightly below the two-minute safety net, while looking 2.5 times further ahead. It alerted 5 of 8 encounters at least 30 s ahead, against 4 for the safety net, with a mean warning of 47.1 s against 33.4 s.

The warning-time gain of 13.75 s on average (95% CI 1.25 to 30.5 s) is not statistically significant (Wilcoxon $p = 0.125$), because only eight encounters were available and three of them improved. The honest conclusion is that SkyControl achieves a longer look-ahead *at the same alert load* as the safety net, which is measured with confidence, while the earlier warning is indicative and needs more data. The cleared level used here is approximated from the recording; with a real digital clearance record the probe would know it directly. The probe complements STCA, which stays in place as the last ground-based safety net.

7.5 Surveillance Integrity (G4)

The integrity monitor found 21 duplicate-target pairs involving 19 ghost tracks and 665 reports (0.77% of all reports). Table 7.4 shows examples. A typical case is RYR52ZT, whose address 4cafc1 appears for 436 s as a second target 4caf41, one bit different, 0.1 NM away with the same callsign. A conventional safety net would see two aircraft 0.1 NM apart and raise an alert; SkyControl merges them before alerting.

Table 7.4 Examples of Duplicate (Ghost) Targets Found in the Real Recording

Real target	Ghost	Bit errors	Same callsign	Min. distance (NM)	Duration (s)
RYR52ZT (4cafc1)	4caf41	1	yes	0.1	436
4063CE (4063ce)	4273ce	2	no	0.22	201
EJU16JD (4403bb)	44033b	1	yes	0.0	325
LOG34JD (407d46)	406d46	1	yes	0.33	197
PRF65 (43c8c4)	43c8d5	2	no	0.6	83
RYR2VG (4cadc1)	4caf41	2	no	0.68	69
RYR55FV (4cafd0)	4caf50	1	yes	0.08	148
SHT9P (40097f)	40097b	1	yes	0.0	62

Position reports were missing for more than 60 s 234 times on 217 of 906 tracks (Fig. 7.6); most gaps are short, but 36 exceed ten minutes. For the 196 dropouts after which the aircraft reappeared, the frozen last position was on average 6.24 NM from the true position at reappearance, while the dead-reckoned coast was 1.48 NM away, a 76.2% reduction (95% CI of the improvement 4.11 to 5.34 NM, $p < 0.001$; Fig. 7.7). The median error falls from 6.36 NM to 0.26 NM.

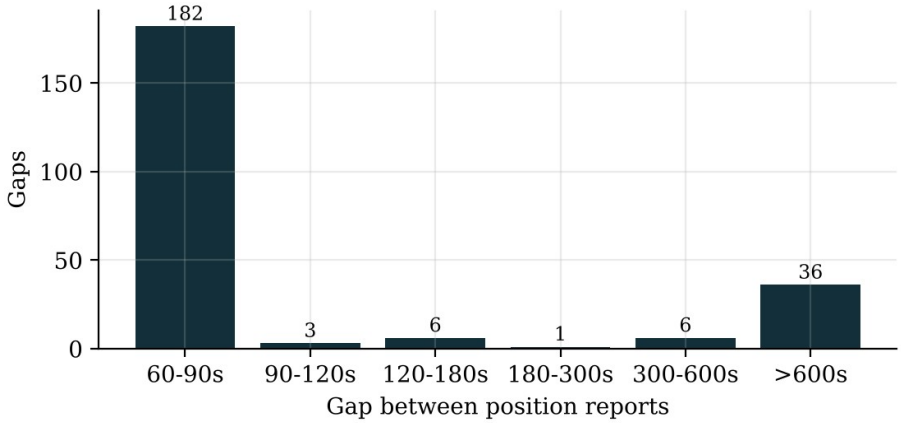


Fig. 7.6 Gaps Between Position Reports in the Real Recording

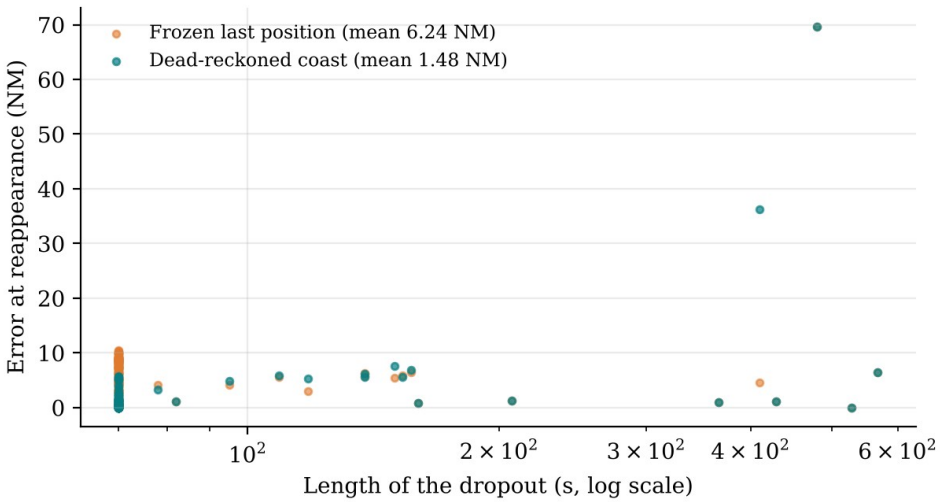


Fig. 7.7 Position Error at Reappearance: Frozen Track versus Dead-Reckoned Coast

Coasting is not always better: for long gaps in which the aircraft turned, a straight coast can be further off than the frozen position, which is why the display shows the coasted track with a growing uncertainty and why low data quality makes the automation policy switch to MANUAL rather than act on the coasted position.

7.6 Communication Safety (G1, G2)

7.6.1 Readback Verifier

The verifier was evaluated on 3,000 readbacks generated from real callsigns of the recording with random clearances and injected errors (wrong digit, swapped digits, omitted item, wrong aircraft answering), once as clean text and once with 5% speech-recognition word noise. On clean text it detected 96.2% of the erroneous readbacks with no false alarm; with ASR noise it detected 98.0% with a hard false-alarm rate (a correct readback called wrong) of 0.22% (Table 7.5, Fig. 7.8).

Table 7.5 Readback Verifier Results by Error Type

Readback type	Clean: n	Clean: flagged	ASR noise: n	ASR noise: flagged	ASR: hard / soft
Wrong value	753	94.2%	781	97.8%	577 / 187
Wrong aircraft	61	100.0%	61	90.2%	54 / 1
Omitted item	336	100.0%	323	100.0%	1 / 322
Correct readback	1850	0.0%	1835	27.2%	4 / 495

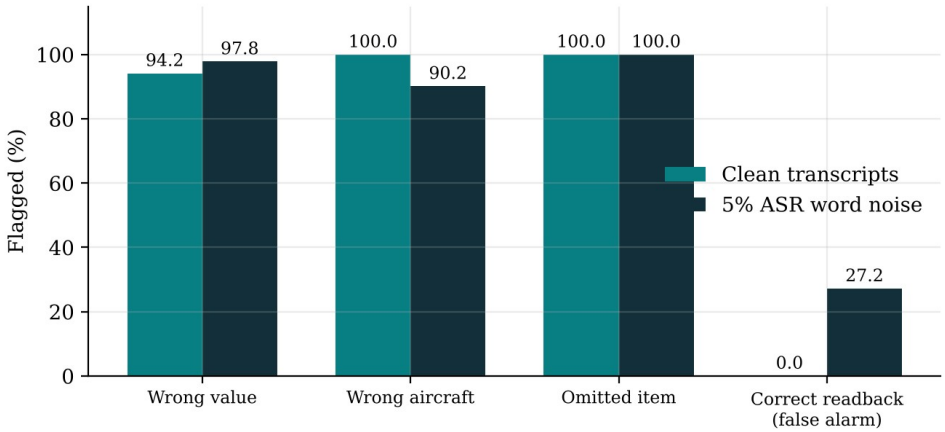


Fig. 7.8 Share of Readbacks Flagged by the Verifier, by Error Type

The cost of noise is visible in the last row: with ASR noise, 27.2% of correct readbacks receive a *soft* "please confirm" request because a digit may have been lost, although only 4 of 1,835 are wrongly called errors. This is a deliberate design choice: an uncertain field is never silently accepted, but it is also not called wrong. In practice it means that the verifier should be used with a good recogniser or with typed or datalink readbacks, and that its confirmation requests add some frequency time, which the model below includes. The corpus was generated by the same team that wrote the verifier, so the detection rates are an upper bound until the verifier is tested on real R/T recordings.

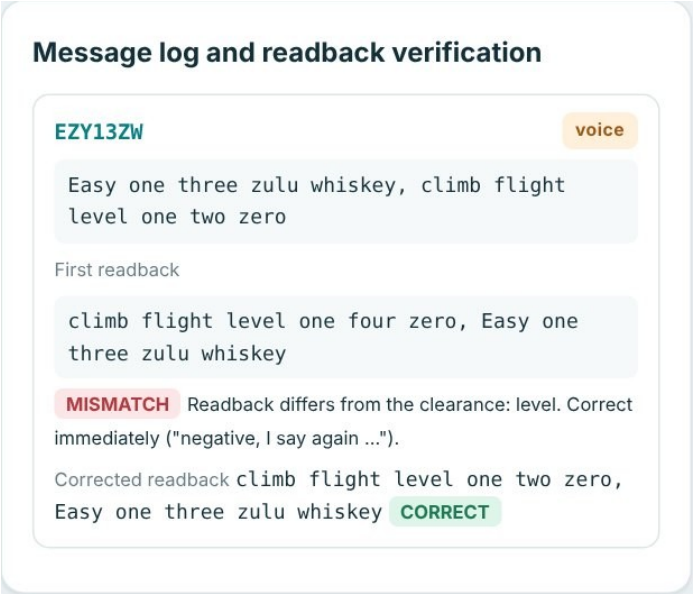


Fig. 7.9 Readback Error Scenario: the Verifier Catches FL140 Instead of FL120

7.6.2 Sector Communication Model

In the Monte Carlo model of 4,000 busy sector-hours, undetected communication errors fell from 40.55 to 3.59 per 10,000 clearances (91.1% fewer;

95% CI of the reduction 35.8 to 38.22 per 10,000), and frequency occupancy fell from 45.1 to 16.9 minutes per hour because 70% of routine clearances move to datalink (Table 7.6).

Table 7.6 Modelled Communication Outcomes per Busy Sector-Hour

Measure	Voice only	SkyControl hybrid	Change
Undetected errors per 10,000 clearances	40.55	3.59	−91.1%
Frequency occupancy (min per hour)	45.1	16.9	−62.5%
Voice transmissions per hour	598.8	180.0	−69.9%

Because the error rates come from the literature, the sensitivity analysis in Fig. 7.10 matters more than the single headline number. Across all combinations of readback error rate (0.5% to 3%) and controller miss rate (20% to 60%), the hybrid stays at about 3.6 undetected errors per 10,000 while the voice baseline ranges from 10 to 180; the reduction is therefore between 65% and 98%. The right-hand panel shows a real trade-off: moving more clearances to datalink frees the frequency, but the residual datalink error slowly increases the undetected errors, so the best datalink share is a design decision rather than "as much as possible".

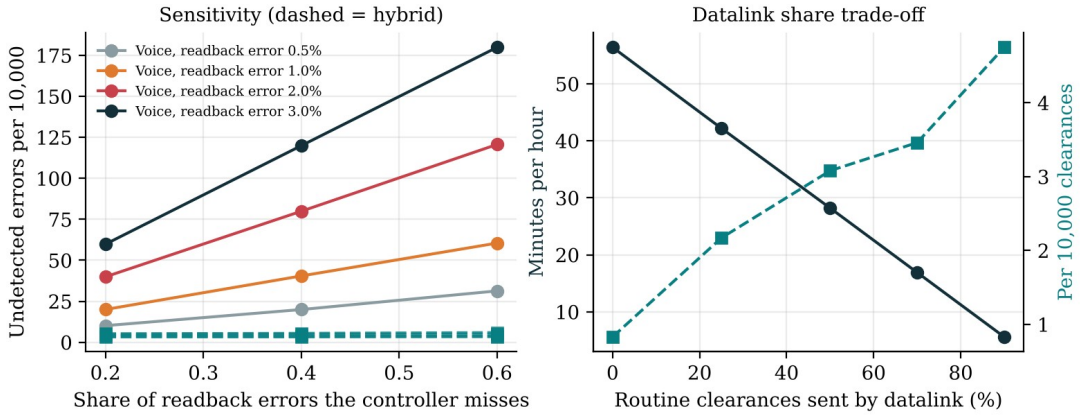


Fig. 7.10 Sensitivity of the Communication Model and the Datalink Trade-off

7.6.3 Similar Callsigns

In the real recording, 10 pairs of similar callsigns shared airspace (within 40 NM and 10,000 ft), involving 20 aircraft (2.2% of aircraft with a callsign), with on average 2.1 aircraft exposed at any moment (Table 7.7). LDG33 and LDG39 came within 0.3 NM of each other, and BAW425 and BAW725 held together in the OCK stack, exactly the situation in which one aircraft takes the other's descent. Today such pairs are found by planning-stage deconfliction and controller vigilance; SkyControl flags them live and detects a wrong aircraft answering.

Table 7.7 Similar Callsigns Sharing Airspace in the Real Recording

Pair	Pattern	Score	Co-present (min)	Closest (NM)
HLE98 / HLE99	one character differs	0.90	19.1	22.4
BAW332 / BAW334	one character differs	0.90	17.7	10.7
LDG33 / LDG39	one character differs	0.90	16.7	0.3
BAW425 / BAW725	one character differs	0.90	9.4	4.8
BAW575 / BAW875	one character differs	0.90	9.0	2.5
BAW935 / BAW938	one character differs	0.90	7.6	13.5
CPA238 / CPA239	one character differs	0.90	5.8	22.4
BAW293 / BAW29G	one character differs	0.90	5.8	13.6
BAW168 / BAW968	one character differs	0.90	1.0	28.1
BAW751 / BAW754	one character differs	0.90	0.8	3.5

7.7 Holding and Hold Prediction (G5)

The recording contains 27 holds at published stacks (plus 4 other orbits), totalling 268.2 aircraft-minutes in 90 minutes, at times with several aircraft holding in each Heathrow stack (Fig. 7.11, Fig. 7.12). At the nominal 40 kg per minute this is about 10,728 kg of fuel (range 8,046 to 26,820 kg) and 33,900 kg of CO_2.

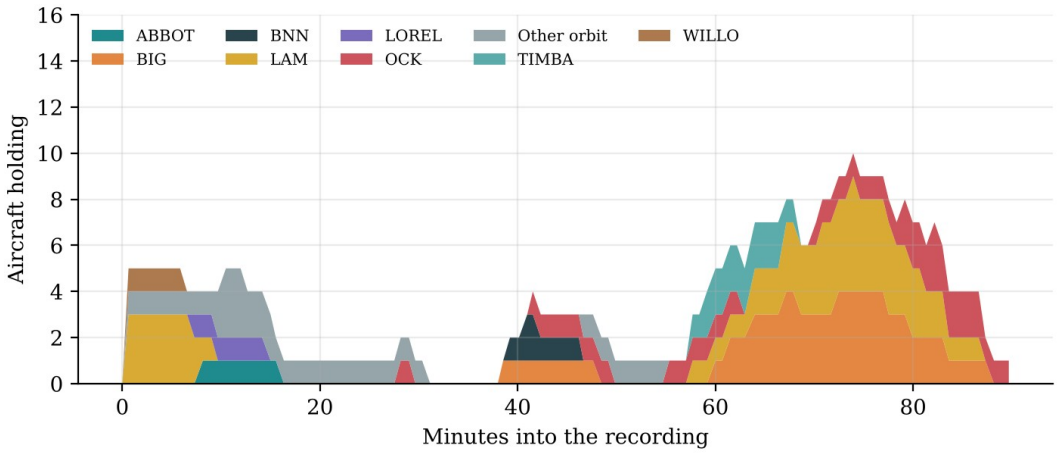


Fig. 7.11 Aircraft Holding per Stack During the Recording

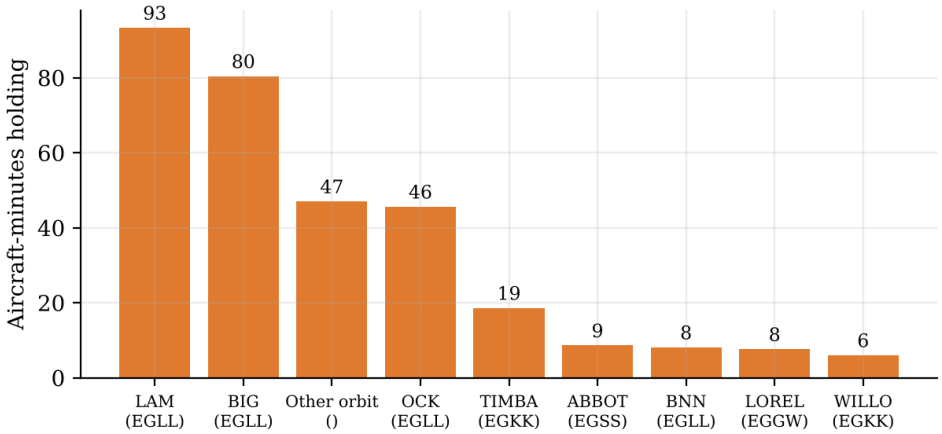


Fig. 7.12 Total Holding Time per Stack

With the speed-control horizon actually observed in the recording, only 39.6 minutes could have been absorbed en route, because many arrivals entered the region shortly before holding. With an extended arrival-management horizon of 350 NM and at most 10% speed reduction, 131.4 minutes (49.0%) could be absorbed, saving about 3,943 kg of fuel and 12,461 kg of CO_2 in these 90 minutes (modelled). Linear holding of this kind is the aim of extended AMAN; SkyControl's contribution is the prediction that makes it possible early enough.

The hold predictor was trained on 135 arrivals and tested on 39 later arrivals (7 of which held). It reached a test AUC of 0.759 (95% CI 0.589 to 0.915) and a Brier score of 0.168; the hold was predicted on average 11.5 minutes before it began (Fig. 7.13, Table 7.8). Higher altitude at 30 to 80 NM (still far from the descent) and more inbound aircraft ahead raise the probability; a greater distance lowers it. The sample is small and the confidence interval wide, so the result is indicative: the model ranks arrivals better than chance and gives useful lead time, but it needs more days of data before its threshold can be relied on.

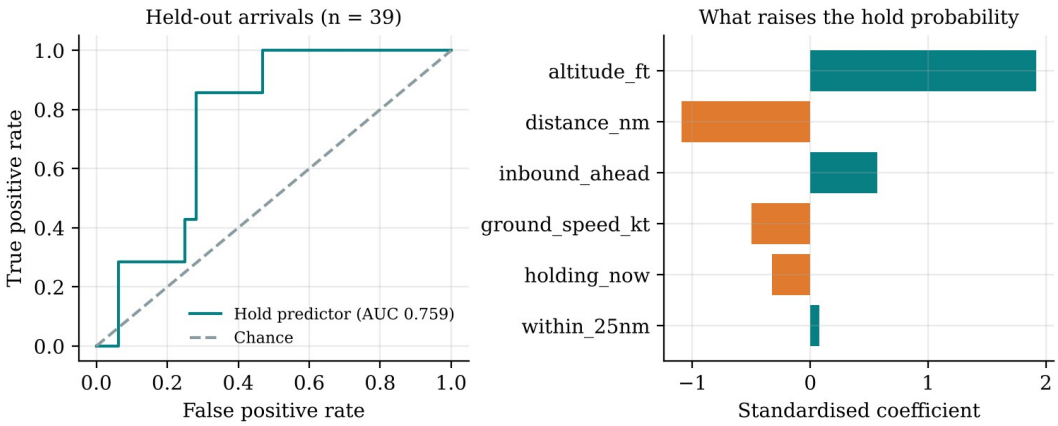


Fig. 7.13 Hold Predictor: ROC Curve on Held-Out Arrivals and Model Coefficients

Table 7.8 Confusion Matrix of the Hold Predictor on the Test Period (Threshold 0.5)

	Predicted hold	Predicted no hold
Held	3	4
Did not hold	9	23

7.8 Detecting Emergency-Like Behaviour and Level Busts (G7)

Eighty-one emergencies were injected into real test-period tracks: 27 emergency descents, 27 engine-out drift-downs and 27 unannounced turn-backs. The vertical-rate rule, the kind of single threshold a display can show, detected only the

descents (27 of 81). The Isolation Forest, at a 1% false-alarm rate on normal traffic, detected 50, and the hybrid (rule or forest) 52. CLAM with the cleared level from the digital clearance record detected all 81 (Fig. 7.14, Table 7.9).

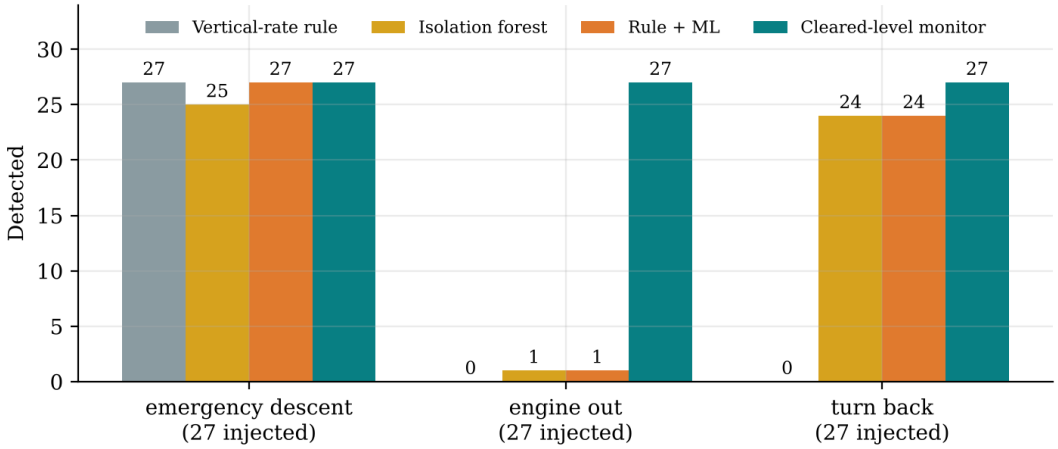


Fig. 7.14 Detection of Injected Emergencies by the Four Detectors

Table 7.9 Detection of Injected Emergencies (Detected of 27 per Kind)

Emergency	Rule	Isolation forest	Hybrid	CLAM	Median delay (s)
emergency descent	27	25	27	27	18.0
engine out	0	1	1	27	21.0
turn back	0	24	24	27	22.0
False alarms on normal traffic	0.25%	1.00%	1.16%	—	

The engine-out drift-down shows the value of combining methods: its descent rate is ordinary and its speed decay gradual, so neither the rule nor the forest sees it, but it departs from the cleared level, which CLAM catches within about 21 s. This was one of the design changes made after the first results, in which the learned detector alone missed most drift-downs. Lost communications have no kinematic signature at all and are therefore handled by the communication monitor (an aircraft that does not acknowledge), which is stated as a limitation.

7.9 Emergencies Injected into Real Traffic (G6)

Sixty emergencies (twelve of each of the five kinds) were injected on real aircraft at different moments of the recording. Each emergency path was checked against the recorded positions of on average 330 other aircraft. In 63.3% of cases the emergency path conflicted with real traffic, with 2.73 conflicts per case on average and 164 in total. SkyControl resolved 150 of them (91.5%) with level changes that were re-verified against all other traffic; the rest were handed to the controller with

the automation level lowered. The emergency got the first landing slot and on average 4.92 real arrivals were delayed by a total of 584 s. The median planning time was 90 ms.

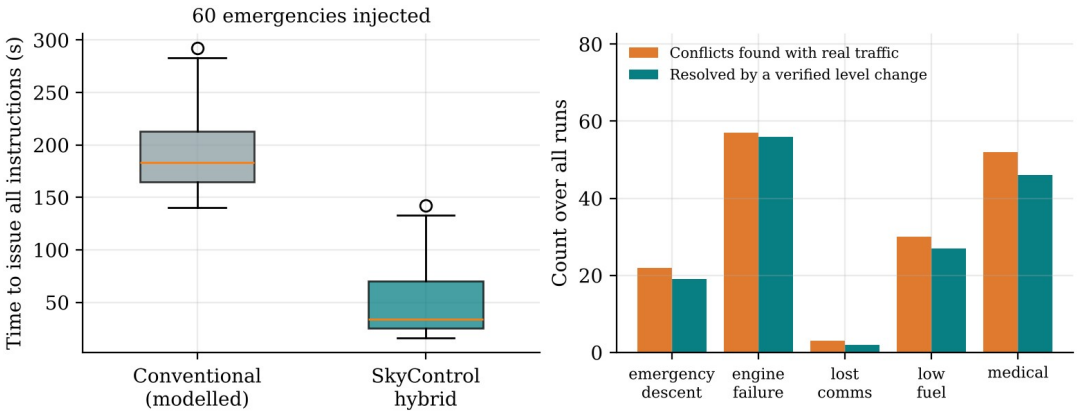


Fig. 7.15 Emergencies Injected into Real Traffic: Handling Time and Conflicts Resolved

In the workflow model the time to issue all instructions fell from 190.8 s (median 183.2 s) for the conventional workflow to 47.8 s (median 34.0 s) for the hybrid, a 75.0% reduction (95% CI 140.1 to 145.6 s, Wilcoxon $p < 0.0001$), mainly because the checklist, the alternate search and the telephone calls are replaced by an automatic ranking and digital coordination. The number of voice transmissions is almost the same (7.1 against 7.07) because emergency instructions stay on voice, as they should.

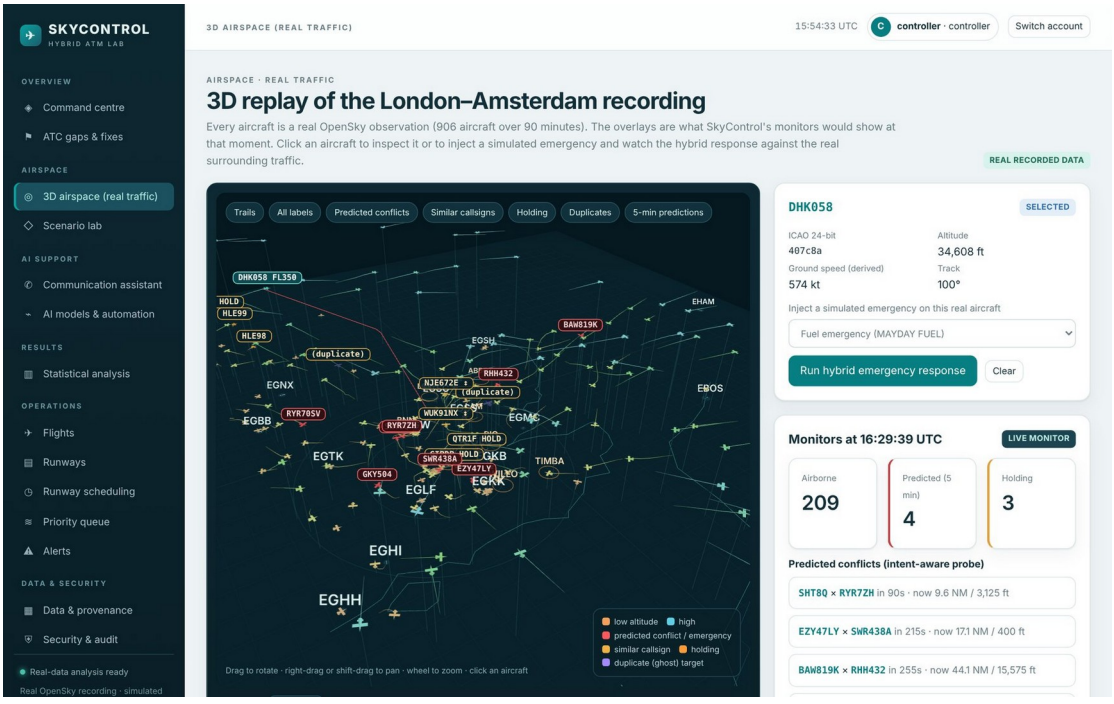


Fig. 7.16 Selecting a Real Aircraft in the 3D Airspace to Inject an Emergency

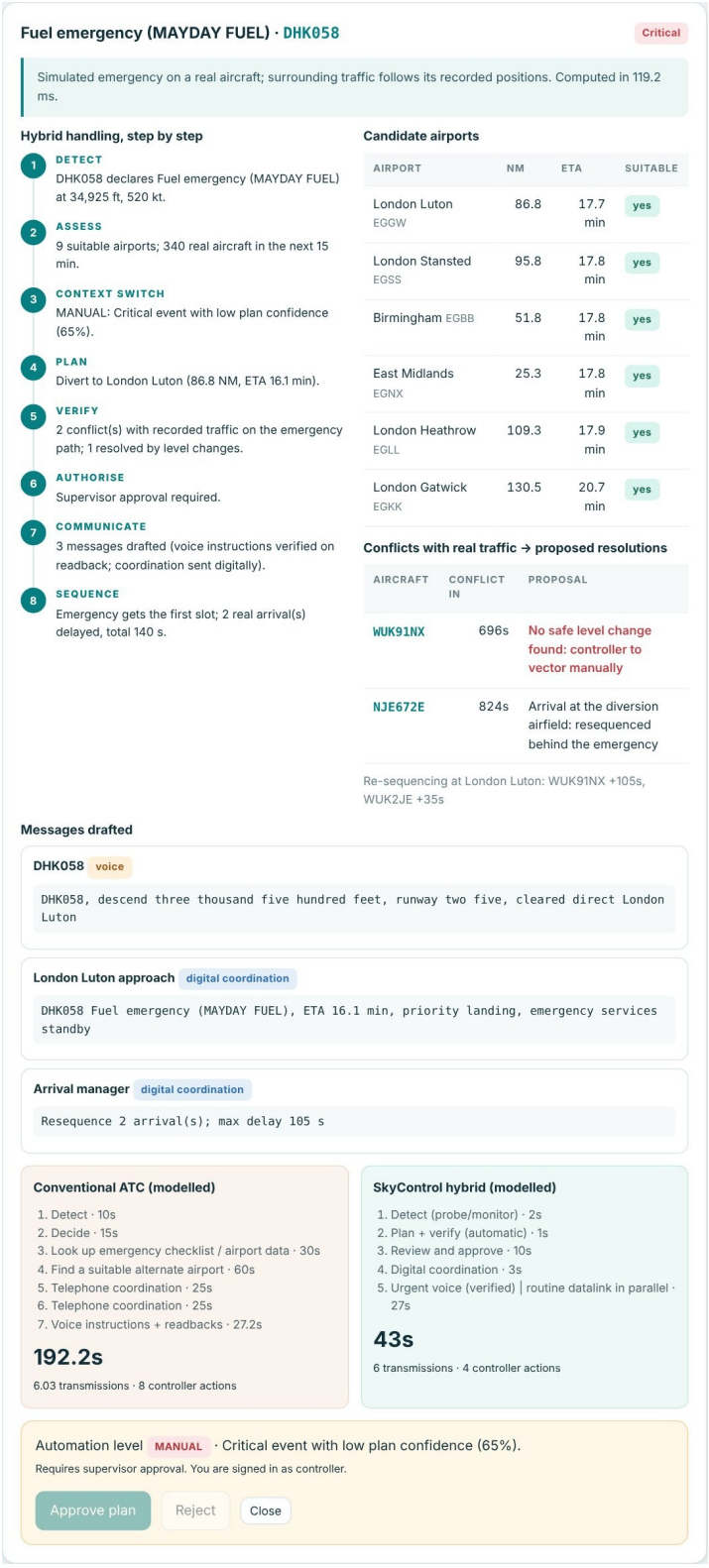


Fig. 7.17 Hybrid Emergency Response for a Fuel Emergency on a Real Aircraft

In Fig. 7.17 the airports are ranked by arrival time rather than distance: from FL350 a 3° descent needs about 110 NM of track, so even East Midlands, 25 NM away, would need a long descent dog-leg and is reached no sooner than Luton. The example is also useful because the AI could not solve everything. For DHK058 the

planner found two conflicts with real traffic; one arrival was re-sequenced behind the emergency, but no safe level change existed for WUK91NX. The plan confidence was therefore only 65%, and the policy switched to MANUAL with supervisor approval: the system shows the controller what it found, drafts the messages and the coordination, and leaves the unresolved conflict to the human. This is the manual context switch required by the project brief, triggered by the data rather than hard-coded.

7.10 Scenario Lab Results

All 21 scenarios produced a SkyControl plan that passed exact verification. Table 7.10 lists each scenario with its category, severity, the automation level chosen, the problems left by the comparator (conflicts, runway conflicts, hazard or closure breaches and fuel-deadline misses) and the modelled time to issue all instructions. The comparator is the conventional handling for each scenario, for example first-come-first-served sequencing without emergency priority or a voice-only readback without automatic verification.

Table 7.10 Scenario Benchmark: Comparator versus SkyControl

Scenario	Category	Severity	Level	Problems	Time (s)
Normal approach	Capacity	Low	AUTO	0 → 0	52 → 32
Converging traffic	Separation	Critical	ADVISORY	15 → 0	120 → 58
Traffic overload	Capacity	High	CONSENT	0 → 0	161 → 46
Fuel emergency	Emergency	Critical	ADVISORY	1 → 0	121 → 46
Medical priority	Emergency	Critical	ADVISORY	0 → 0	137 → 46
Engine failure	Emergency	Critical	ADVISORY	1 → 0	163 → 34
Emergency descent	Emergency	Critical	ADVISORY	2 → 0	99 → 26
Bird strike	Emergency	High	CONSENT	3 → 0	142 → 47
Runway incursion	Runway	Critical	ADVISORY	3 → 0	92 → 43
Runway closure	Runway	High	CONSENT	11 → 0	171 → 45
Go-around	Runway	High	CONSENT	3 → 0	52 → 43
Diversion	Capacity	High	CONSENT	6 → 0	167 → 46
Severe weather	Hazard	High	CONSENT	5 → 0	115 → 46
Radio failure	Communication	High	CONSENT	6 → 0	112 → 51
Readback error	Communication	High	CONSENT	1 → 0	144 → 21
Callsign confusion	Communication	High	CONSENT	1 → 0	132 → 21
Level bust	Separation	Critical	ADVISORY	1 → 0	143 → 32
Ghost target	Surveillance	High	CONSENT	1 → 0	29 → 12
Surveillance gap	Surveillance	High	MANUAL	3 → 0	127 → 49
AI disagreement	Automation	High	MANUAL	1 → 0	54 → 22
Security signal	Emergency	Critical	MANUAL	2 → 0	123 → 46

Over all scenarios the comparator left 34 aircraft conflicts, 10 runway conflicts, 21 hazard or closure breaches and 1 fuel-deadline miss; SkyControl left

none (Fig. 7.18). The price is a longer total route length (3,951 NM against 4,550 NM, +15%) from reroutes around weather and holding circuits. The modelled time to issue all instructions fell from 117.1 s to 38.7 s (67.0% less, Wilcoxon p = 3.0e-05), radio transmissions from 130.7 to 74 and controller actions from 113 to 57 (Fig. 7.19).

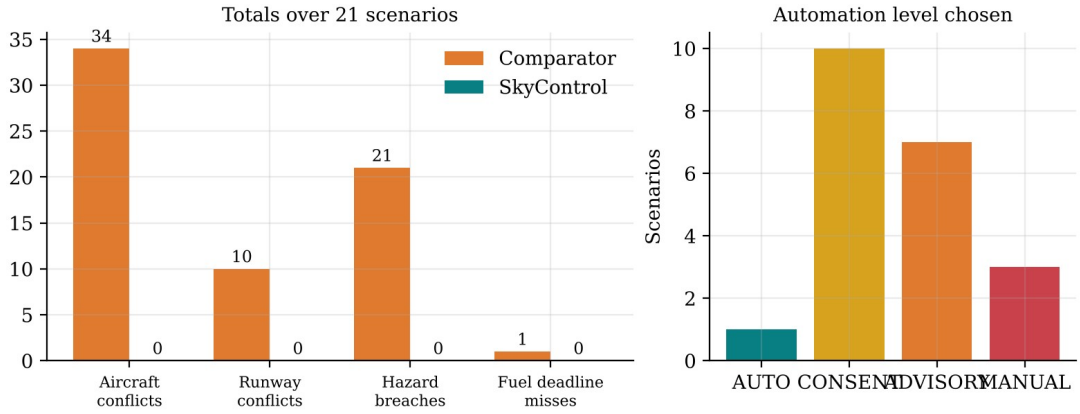


Fig. 7.18 Problems Left by the Comparator and by SkyControl, and Automation Levels Chosen

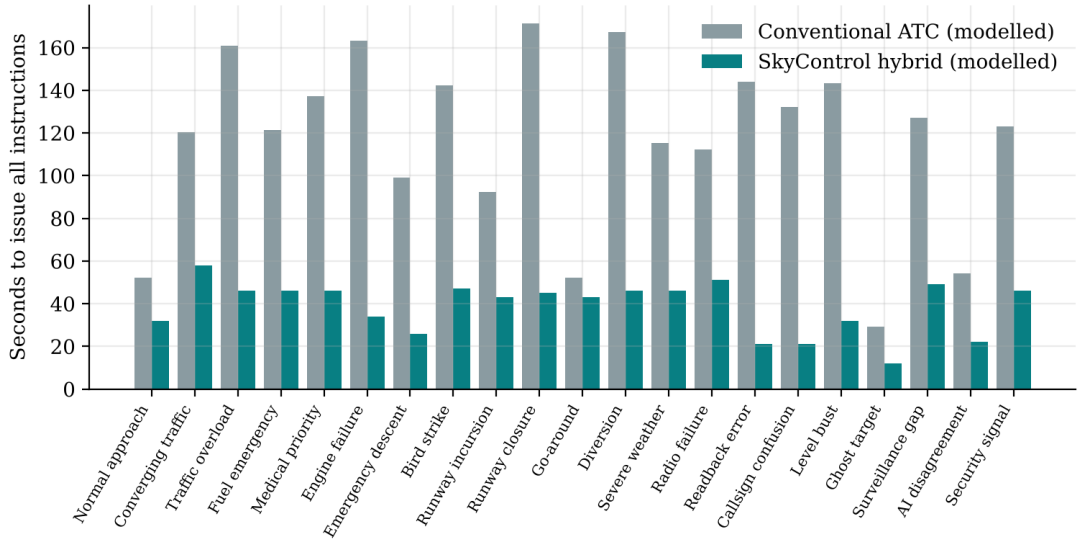


Fig. 7.19 Modelled Time to Issue All Instructions in Each Scenario

The automation levels chosen (1 AUTO, 10 CONSENT, 7 ADVISORY and 3 MANUAL) show that SkyControl acts autonomously only in routine traffic. All critical emergencies require a supervisor; the security signal, the AI disagreement and the surveillance gap put the human fully in charge. The following subsections walk through five scenarios step by step, as they appear in the scenario lab.

7.10.1 Fuel Emergency: Approval Gate and Ten Steps

A low-fuel arrival is last in a queue of four. The comparator (first-come-first-served without emergency priority) lands it after its simulated fuel deadline.

SkyControl detects the emergency, assesses it as critical, and the policy chooses ADVISORY with supervisor approval. The run pauses at T+20 s; a controller cannot approve it (Fig. 7.20). After a supervisor approves, the plan (emergency first, holding circuits for the others, 120 s spacing) is executed and all ten steps complete (Fig. 7.21). Fig. 7.22 compares the workflows: 121.3 s and about eight radio transmissions conventionally, 46 s with SkyControl.

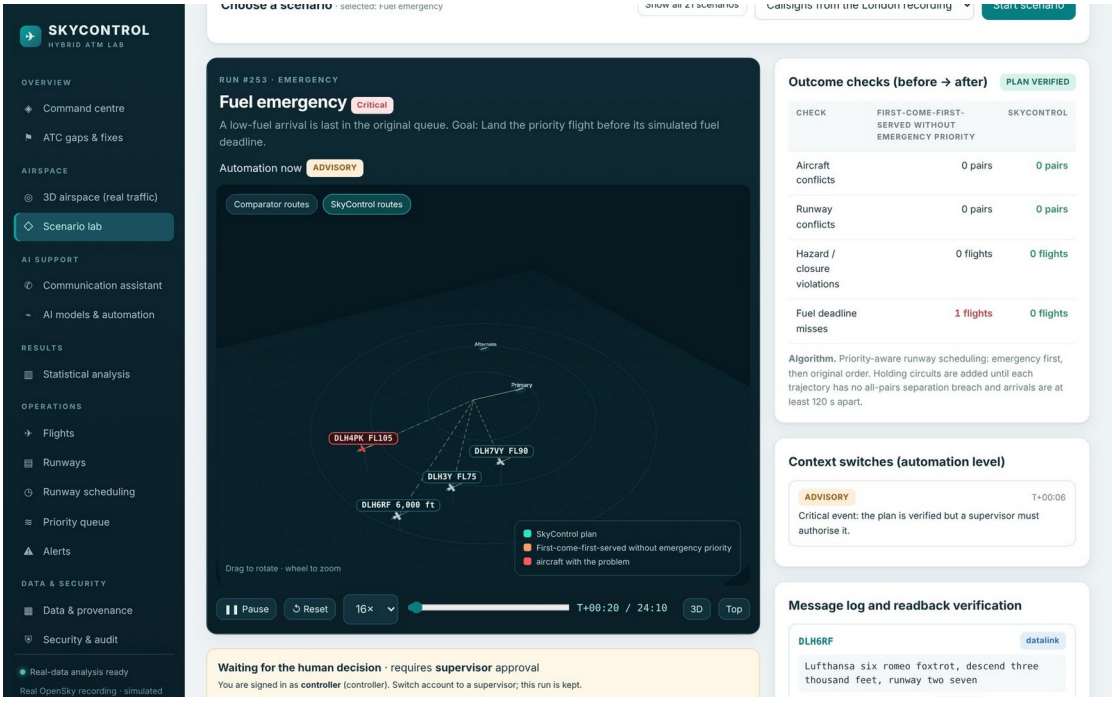


Fig. 7.20 Fuel Emergency Paused at the Approval Gate for a Controller



Fig. 7.21 Fuel Emergency: the Ten Handling Steps after Supervisor Approval

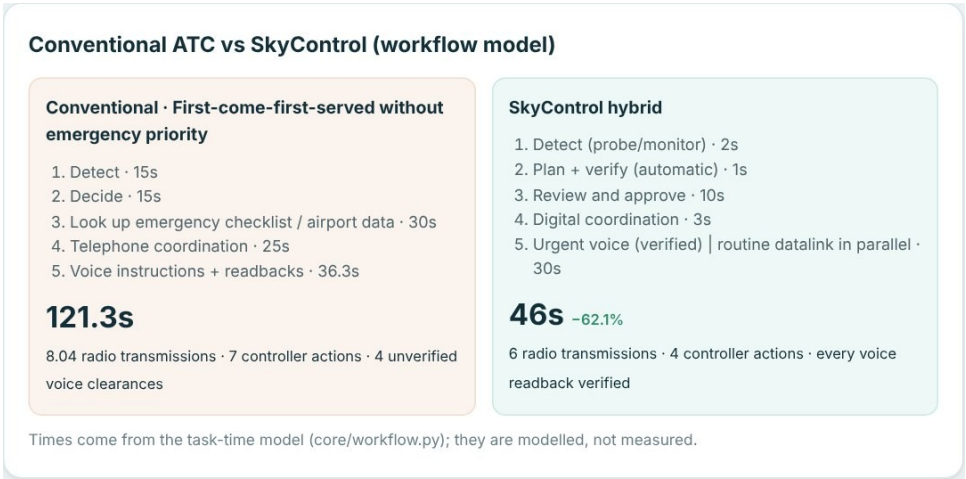


Fig. 7.22 Fuel Emergency: Conventional and SkyControl Workflows (Modelled)

7.10.2 Readback Error

EZY13ZW is cleared to FL120 and reads back FL140. In the comparator the controller must hear the wrong number; if it is missed, the aircraft climbs into the level of conflicting traffic, which the scenario shows as a conflict. SkyControl's verifier flags MISMATCH on the level field immediately, the correction is issued, and the corrected readback is verified CORRECT (Fig. 7.9). Fig. 7.23 shows the steps and Fig. 7.24 the outcome checks.

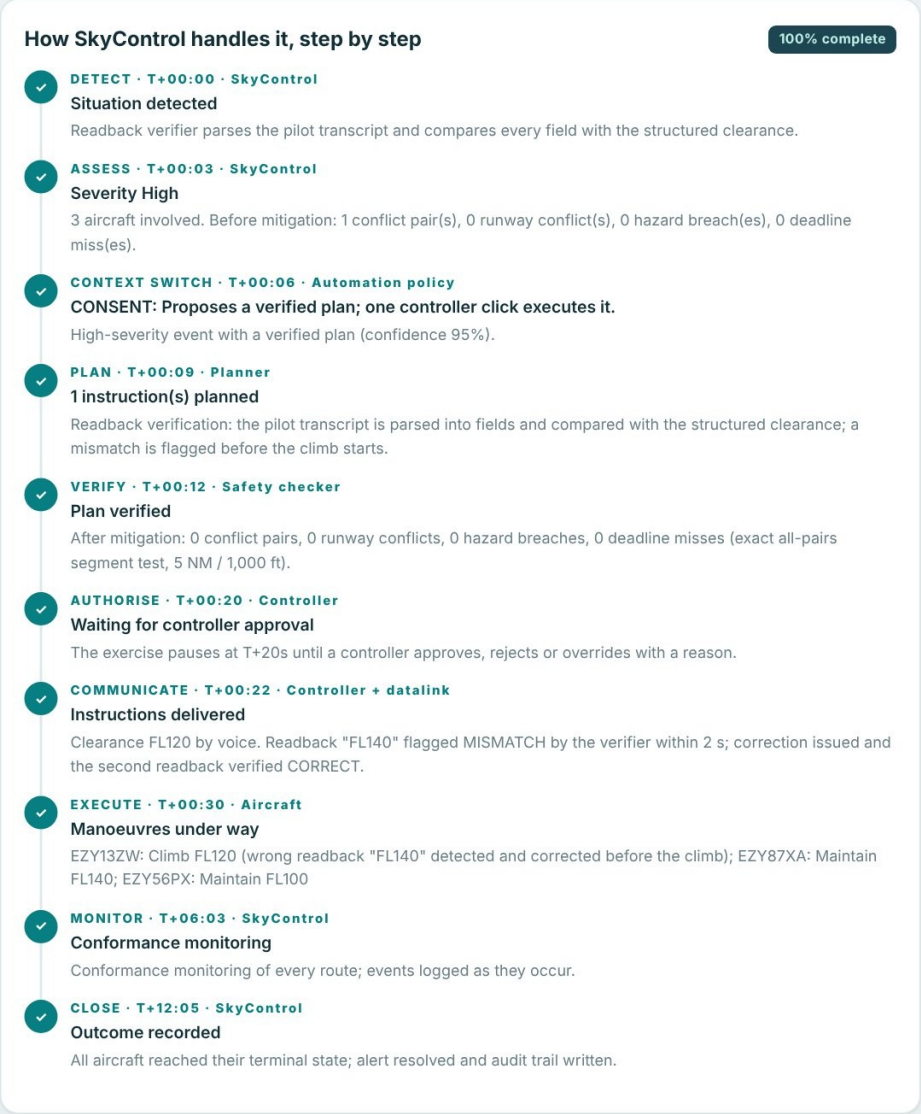


Fig. 7.23 Readback Error Scenario: Handling Steps

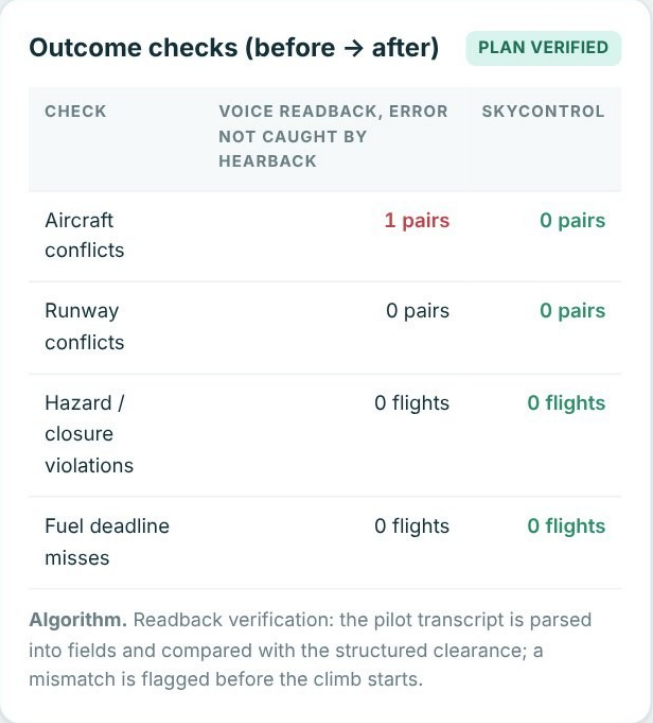


Fig. 7.24 Readback Error Scenario: Outcome Checks

7.10.3 Converging Traffic

Six aircraft converge on one crossing point at similar levels. Without intervention, 15 pairs would lose separation; serial voice resolution takes the controller a long time. SkyControl assigns flight levels greedily in priority order and verifies all 15 pairs exactly, removing every conflict with level changes only (Fig. 7.25, Fig. 7.26). Because the event is critical, a supervisor authorises the plan.

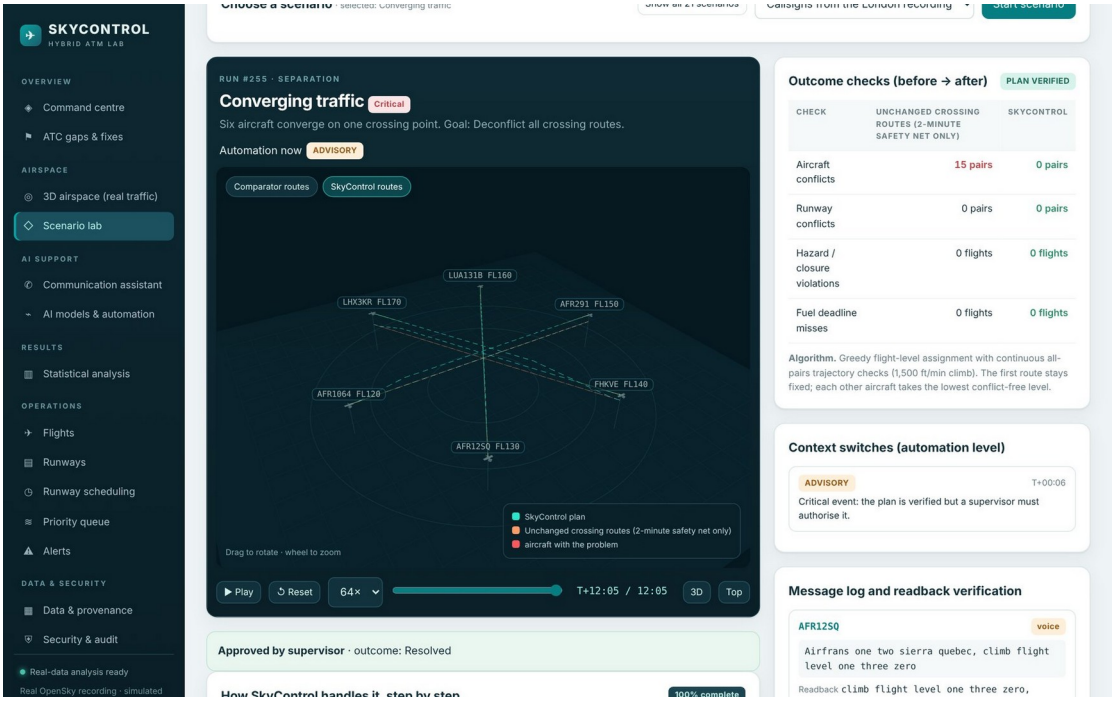


Fig. 7.25 Converging Traffic Scenario in the 3D Scenario Lab

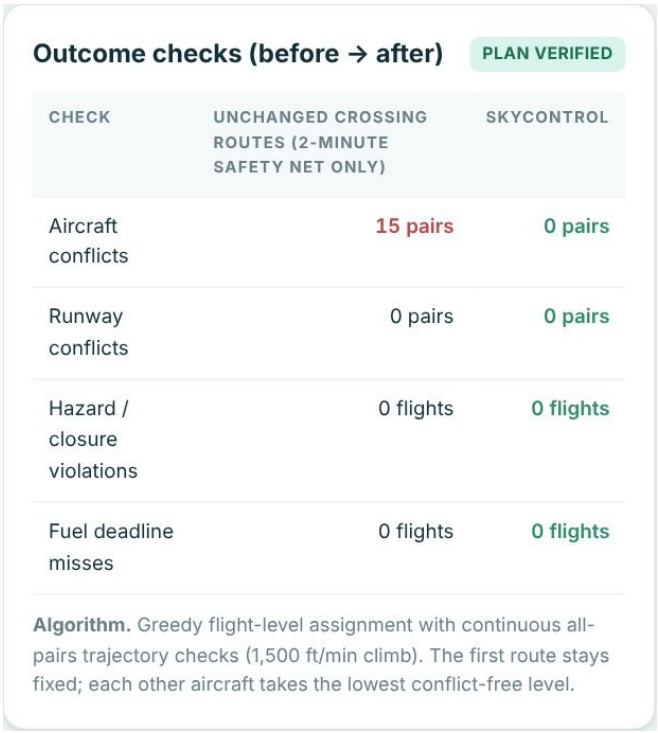


Fig. 7.26 Converging Traffic: Outcome Checks Before and After

7.10.4 Ghost Target

A duplicate radar target appears on top of a real aircraft. A comparator that trusts every target sees a conflict and issues an unnecessary avoidance instruction. SkyControl's integrity monitor identifies the duplicate from the address bit error; the automation level first drops to ADVISORY while the integrity check runs, then

returns to CONSENT once the duplicate is confirmed and merged and data quality is restored (Fig. 7.27); no instruction is needed.

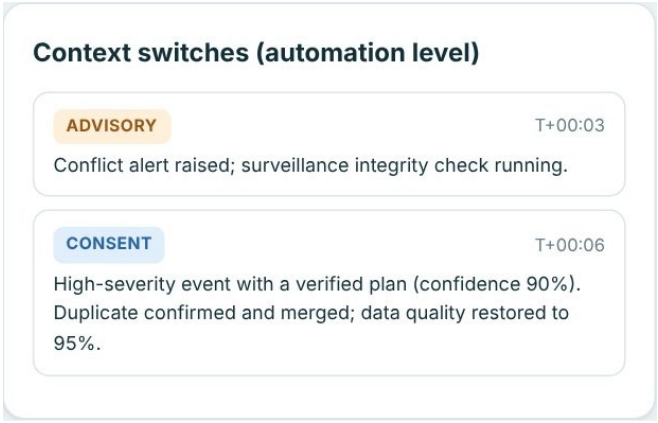


Fig. 7.27 Ghost Target: Context Switches Logged During the Event

7.10.5 AI Disagreement: When the AI Is Wrong

This scenario tests the central safety idea of the project. The learned model predicts that an aircraft will turn away with 82% confidence, so trusting it would leave the aircraft on course. The deterministic rule, which extrapolates the current motion, predicts a conflict. Because the two disagree, the policy drops from CONSENT to MANUAL: the rule wins and the human decides (Fig. 7.28). In the replay the aircraft does not turn, so the comparator that trusted the AI loses separation (one pair), while the conservative plan keeps all aircraft separated (Fig. 7.29).

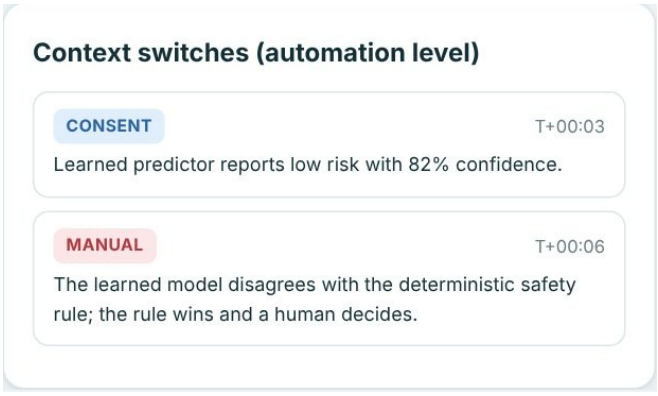


Fig. 7.28 AI Disagreement: Context Switch from CONSENT to MANUAL

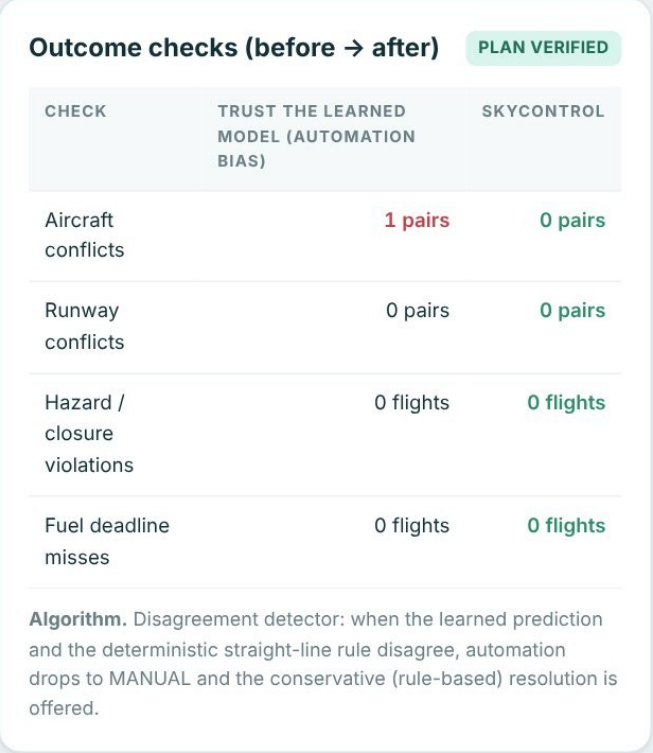


Fig. 7.29 AI Disagreement: Trusting the Learned Model versus the Conservative Plan

7.11 Summary of Statistical Results

Table 7.11 summarises the paired comparisons. Six of the seven comparisons are significant at $\alpha = 0.05$; the warning time of the conflict probe is not, because only eight real encounters were available.

Table 7.11 Paired Comparisons: Current Practice versus SkyControl

Comparison	n	Current	SkyControl	Change	p	Evidence
Prediction error, 60 s (NM)	17,994	0.555	0.449	19.1%	< 0.0001	Measured
Prediction error, 120 s (NM)	16,873	1.635	1.267	22.5%	< 0.0001	Measured
Prediction error, 300 s (NM)	13,765	6.845	4.482	34.5%	< 0.0001	Measured
Track error after dropout (NM)	196	6.24	1.48	76.2%	< 0.0001	Measured
Conflict warning time (s, higher better)	8	33.4	47.1	+41.2%	0.125	Measured (small n)
Emergency handling time (s)	60	190.8	47.8	75.0%	< 0.0001	Real traffic, modelled time
Scenario handling time (s)	21	117.1	38.7	67.0%	< 0.0001	Scenario, modelled

7.12 How SkyControl Complements Current ATC

SkyControl was not built to win against current ATC but to strengthen it where it is weak. Table 7.12 summarises, for each function, what current ATC does, what SkyControl adds and how the two work together.

Table 7.12 Current ATC and SkyControl Side by Side

Function	Current ATC	SkyControl adds	Effect (evidence)
Readback	Controller hearback	Automatic verification of every voice readback; datalink for routine items	Undetected errors −91% (modelled)
Callsigns	Planning deconfliction, vigilance	Live similarity alerts; wrong-aircraft detection	10 real pairs flagged (measured)
Conflict alerting	STCA, ~2 min	Intent-aware 5-min probe; STCA kept	2.5× look-ahead at the same alert rate (measured)
Surveillance	Tracker output trusted	Ghost detection, coasting, integrity-driven context switch	21 ghosts; track error −76% (measured)
Holding	Decided near the stack	Hold prediction 30-80 NM out; linear-holding estimate	AUC 0.76, 11.5 min lead; 49% absorbable (modelled)
Level busts	CLAM if the level is entered	Cleared level always known	81 of 81 injected events (measured)
Emergencies	Checklists, phone, memory	Airport ranking, path checked against real traffic, digital coordination	91.5% conflicts resolved; time −75% (modelled)
Automation	On or off	Four levels with logged context switches	Humans approve all critical plans (scenario)
Runways	Manual re-planning	Occupancy-aware sequencing	All runway conflicts removed (scenario)

7.13 Software Testing and Performance

The automated test suite has 93 tests, all passing locally and in the Jenkins pipeline, with 90% statement coverage of the back end (Table 7.13). ruff reports no lint errors. Browser tests with headless Chromium loaded all 14 pages without JavaScript errors and exercised the interactive flows: the wrong-login message, the blocked critical plan for a controller and its approval by a supervisor, a readback-error scenario through all ten steps, emergency injection on a real aircraft, a MISMATCH readback check and all eight statistics tabs.

Table 7.13 Automated Test Suite

Test file	Tests	What is verified
test_auth.py	9	Uniform wrong-credential message, stored attempts with reasons, no passwords stored, lockout, CSRF, roles, users
test_operations.py	6	Flight CRUD and validation, runway conflicts, calendar, priority queue, alerts, dashboard
test_scenarios.py	47	All 21 scenarios verified conflict-free, comparator problems, ten steps, context switches, readback, fuel deadline
test_lab_api.py	5	Supervisor gate, controller consent, AUTO runs, replay controls, override reason, data sources
test_comms_and_automation.py	10	Readback verdicts, NATO letters, phraseology, callsign similarity, policy, exact geometry
test_analytics.py	12	Recording integrity, known ghost pair, coasting, predictor on held-out data, alerts, holding, emergencies
test_app.py	4	Health, static front end, security headers, dataset register, audit access
Total	**93**	**90% statement coverage**

A load test with eight concurrent clients and 120 requests per endpoint measured the latencies in Fig. 7.30. Interactive endpoints answer in 18 to 50 ms at the median. Creating a scenario, which plans and verifies all aircraft pairs and builds the pipeline, takes about 343 ms. The first request for a new frame of the 3D replay computes the intent-aware probe for all aircraft (up to 2.5 s); later requests for the same frame are served from the cache in about 23 ms.

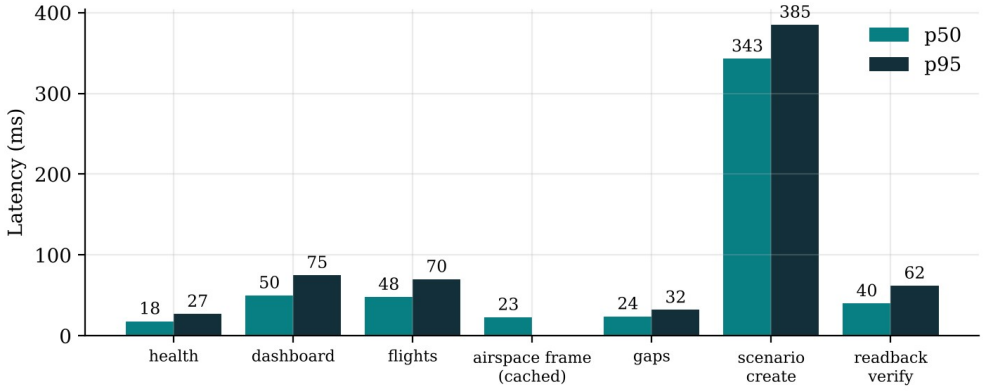


Fig. 7.30 Median and 95th-Percentile Latency under Load (8 Concurrent Clients)

7.14 User Interface

The following screenshots show the main pages of the application as used in the faculty demonstration. The command centre (Fig. 7.31) summarises the key results and the live state, including failed logins in the last 24 hours. The scenario lab catalogue (Fig. 7.32) groups the 21 scenarios by category and states the gap each one demonstrates. The statistical analysis page has eight tabs (Fig. 7.33 to Fig. 7.39). The AI models page (Fig. 7.40) documents each model and contains the context-switch simulator, and the communication assistant (Fig. 7.41) verifies readbacks. The operations, data and security pages complete the application.

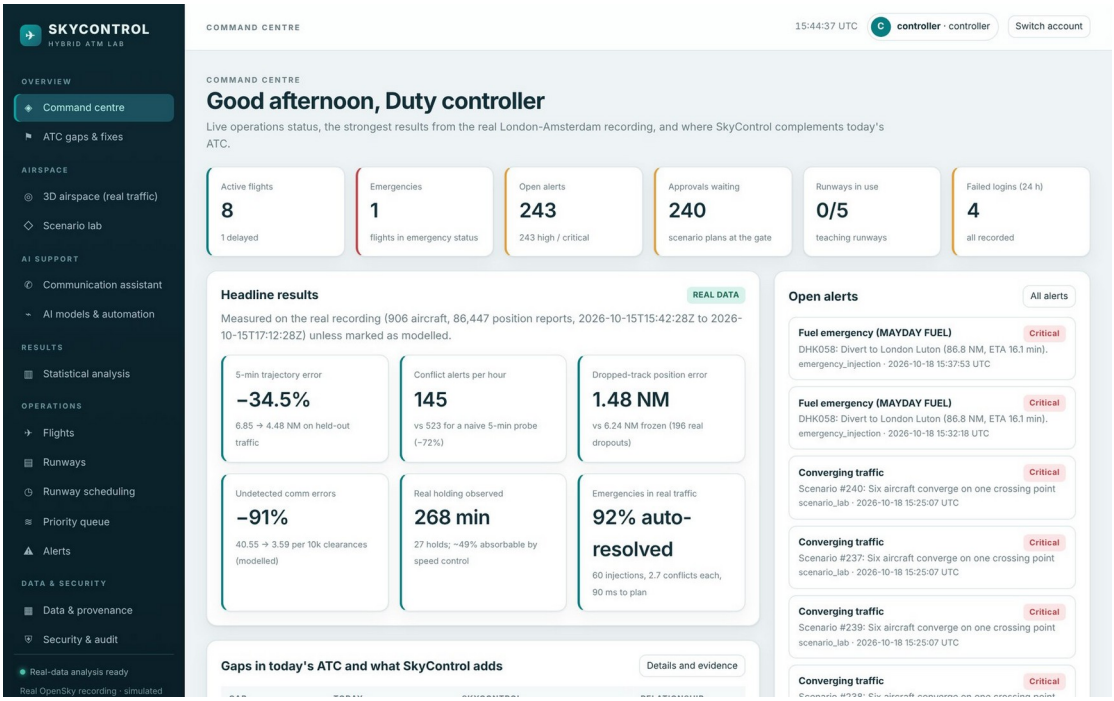


Fig. 7.31 Command Centre (Dashboard)

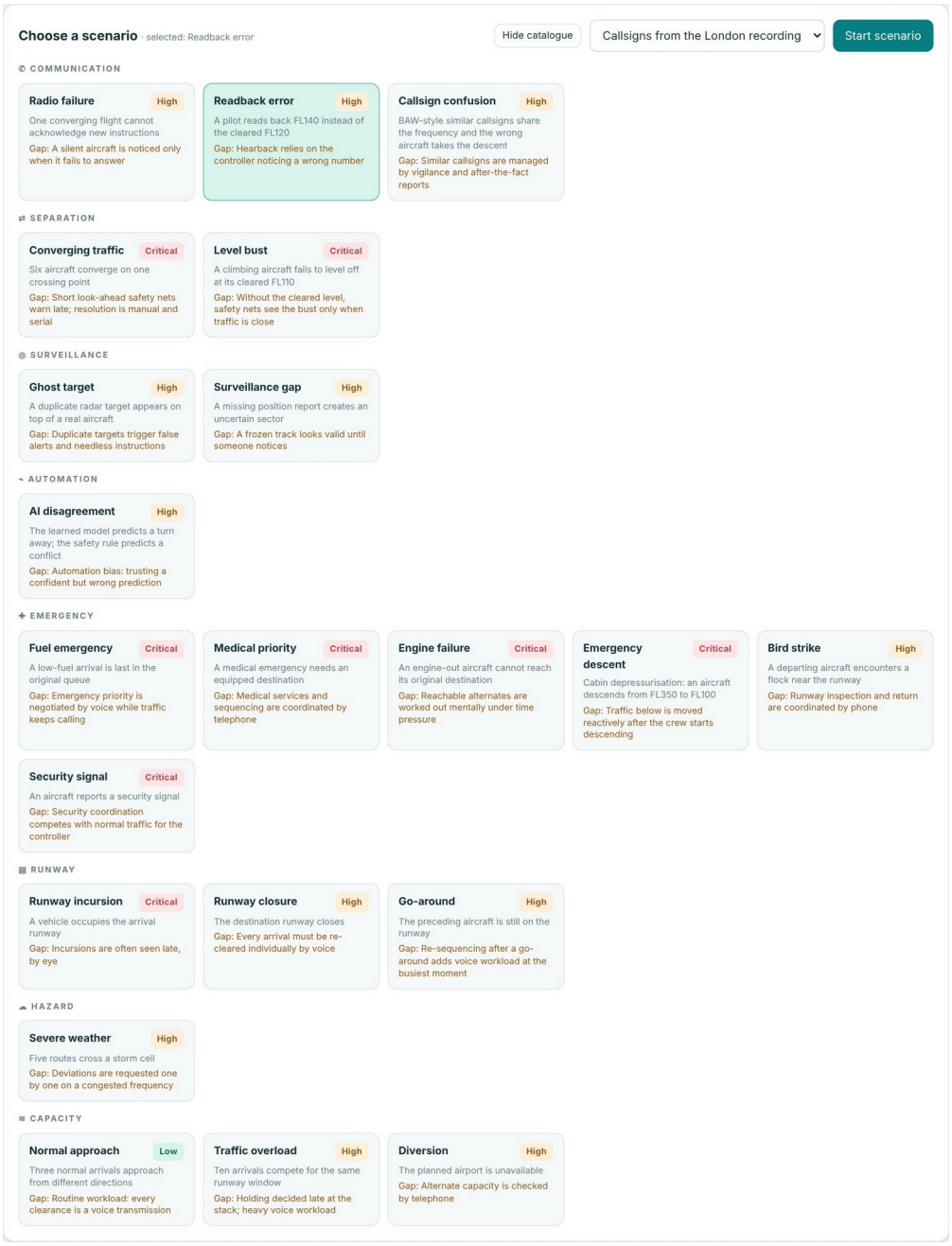


Fig. 7.32 Scenario Lab Catalogue of 21 Scenarios

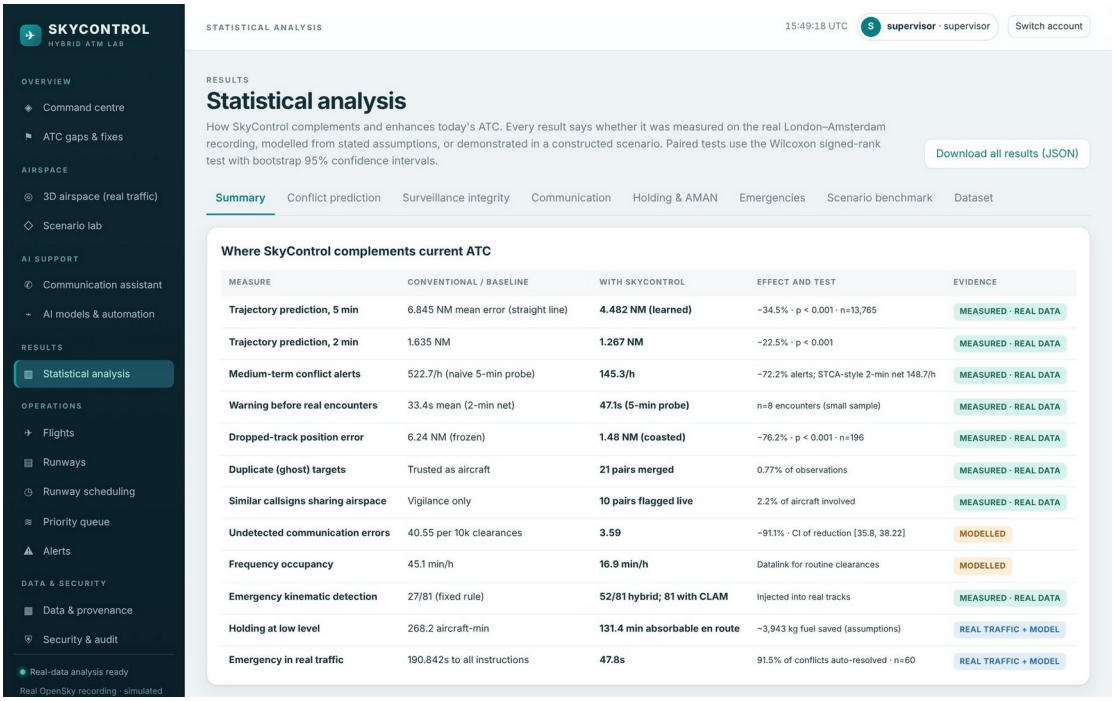


Fig. 7.33 Statistical Analysis: Summary Tab

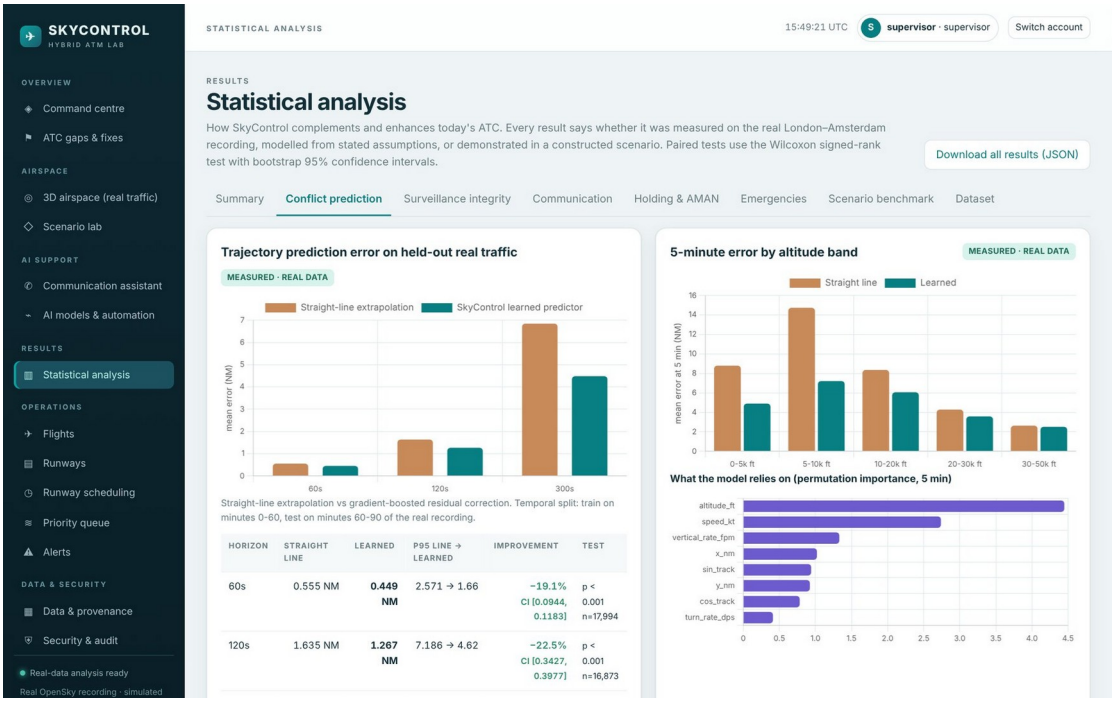


Fig. 7.34 Statistical Analysis: Conflict Alerting Tab

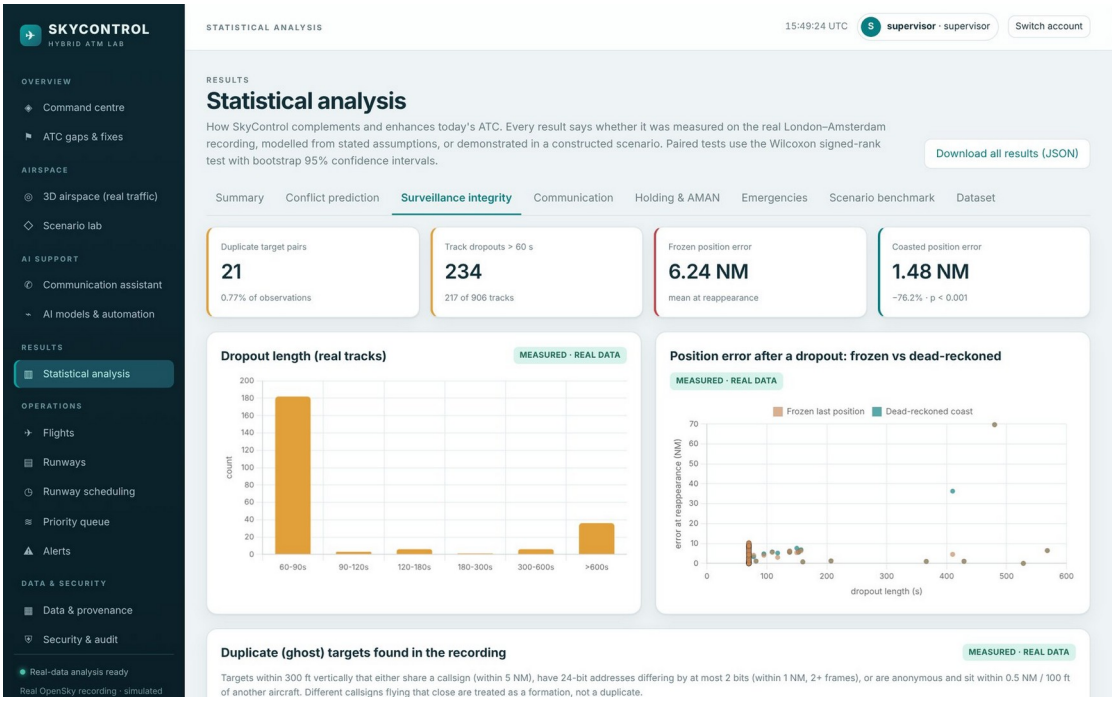


Fig. 7.35 Statistical Analysis: Surveillance Integrity Tab

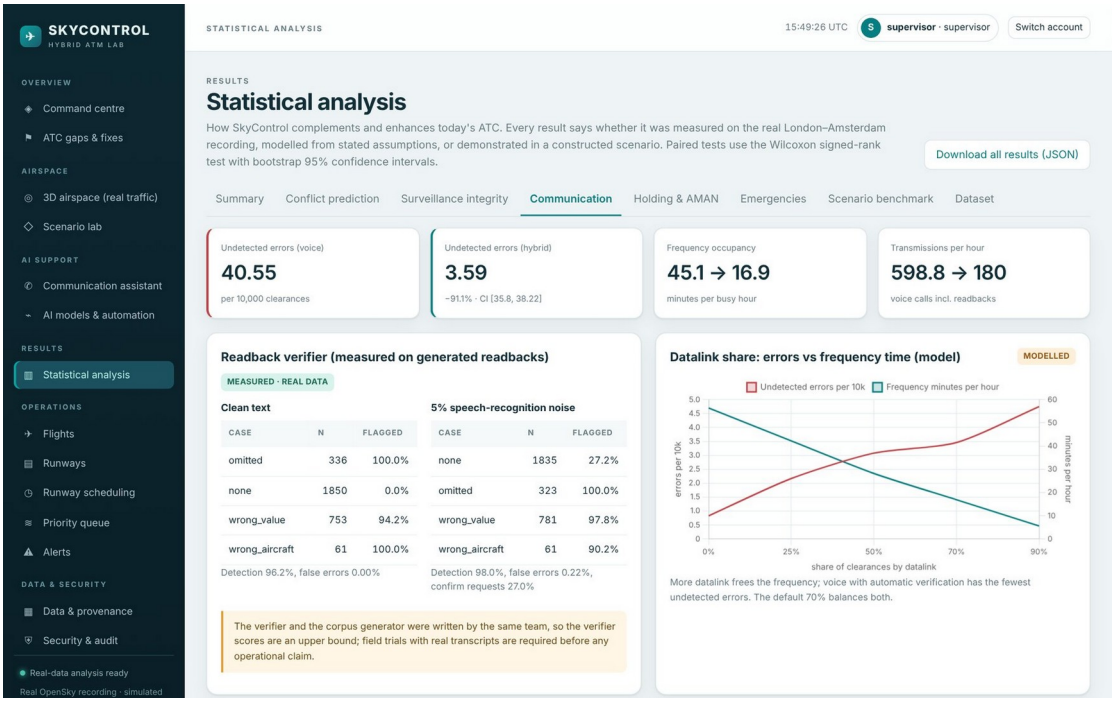


Fig. 7.36 Statistical Analysis: Communication Tab

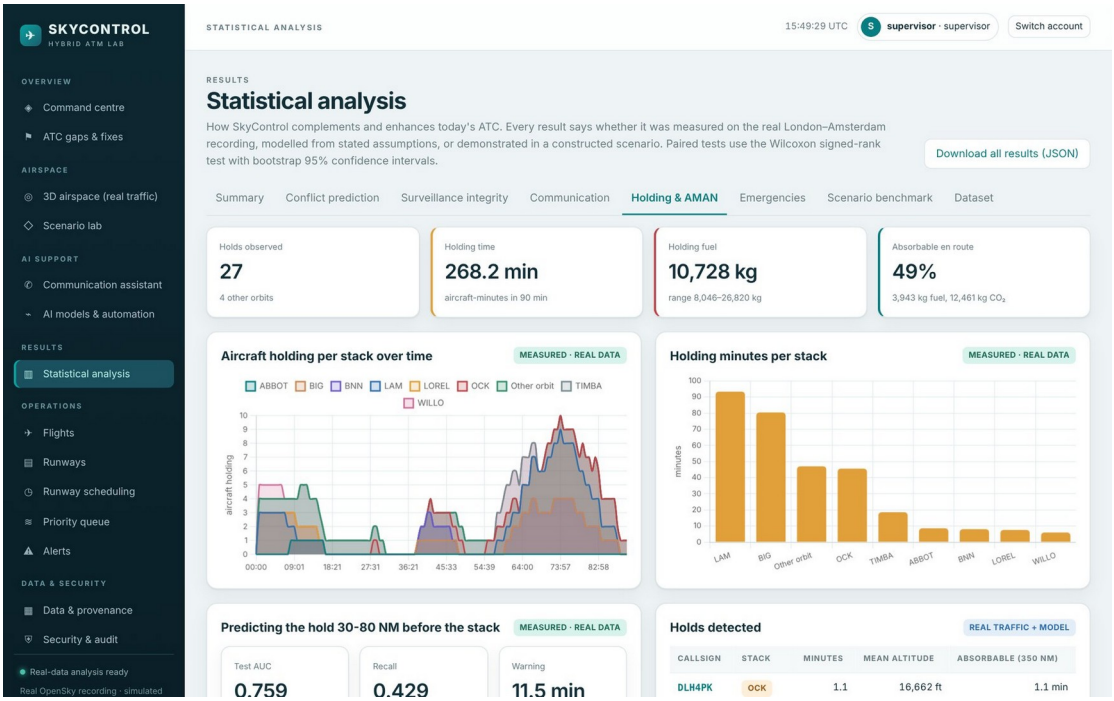


Fig. 7.37 Statistical Analysis: Holding Tab

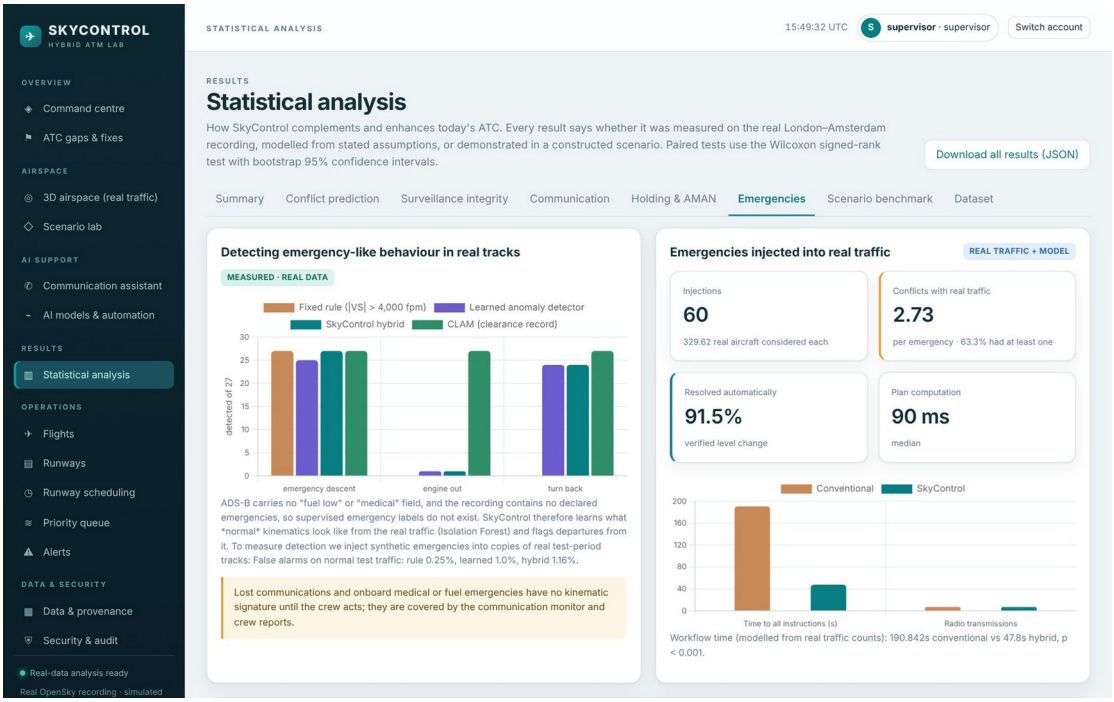


Fig. 7.38 Statistical Analysis: Emergencies Tab

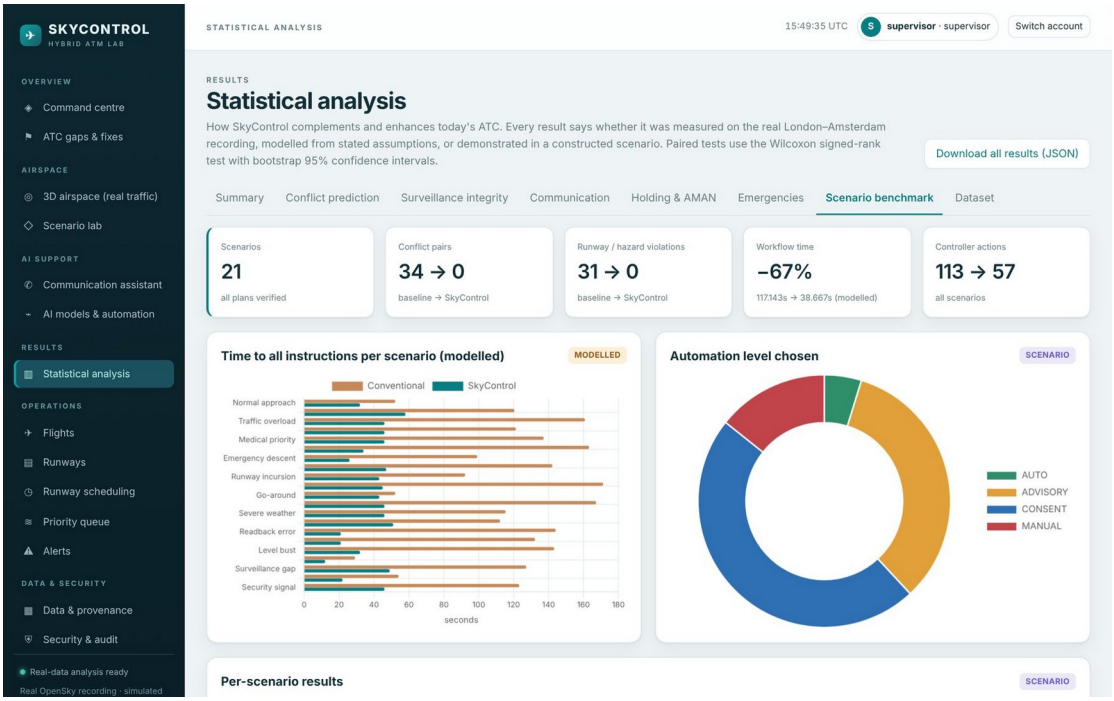


Fig. 7.39 Statistical Analysis: Scenarios Tab

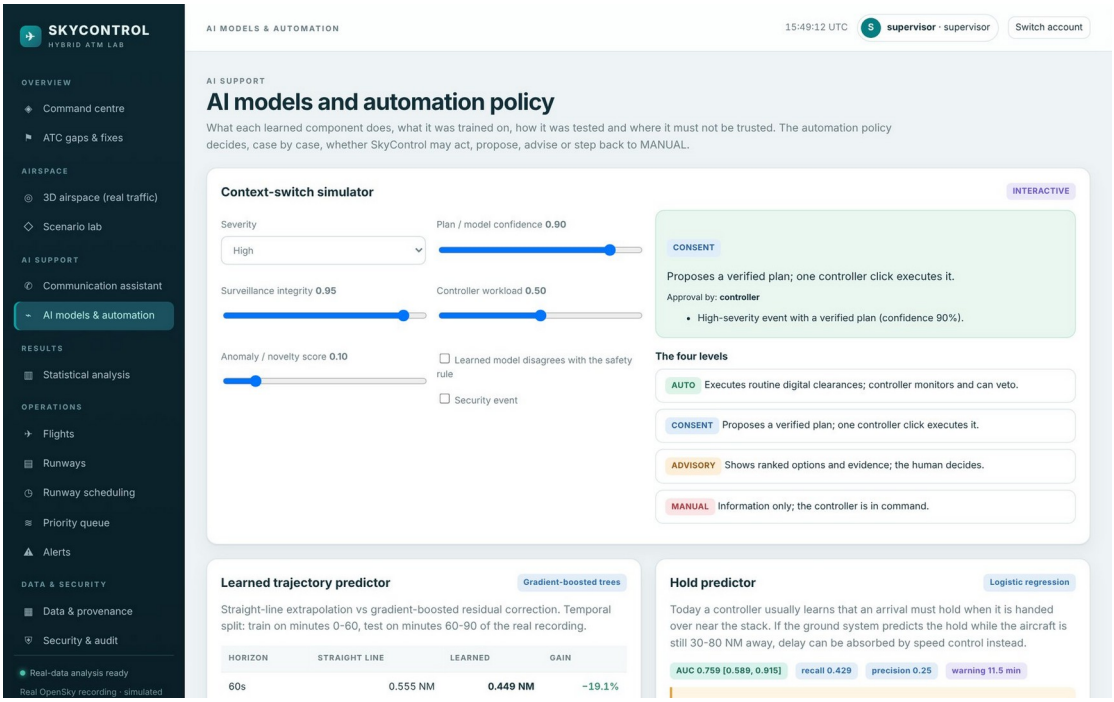


Fig. 7.40 AI Models and Automation Page with the Context-Switch Simulator

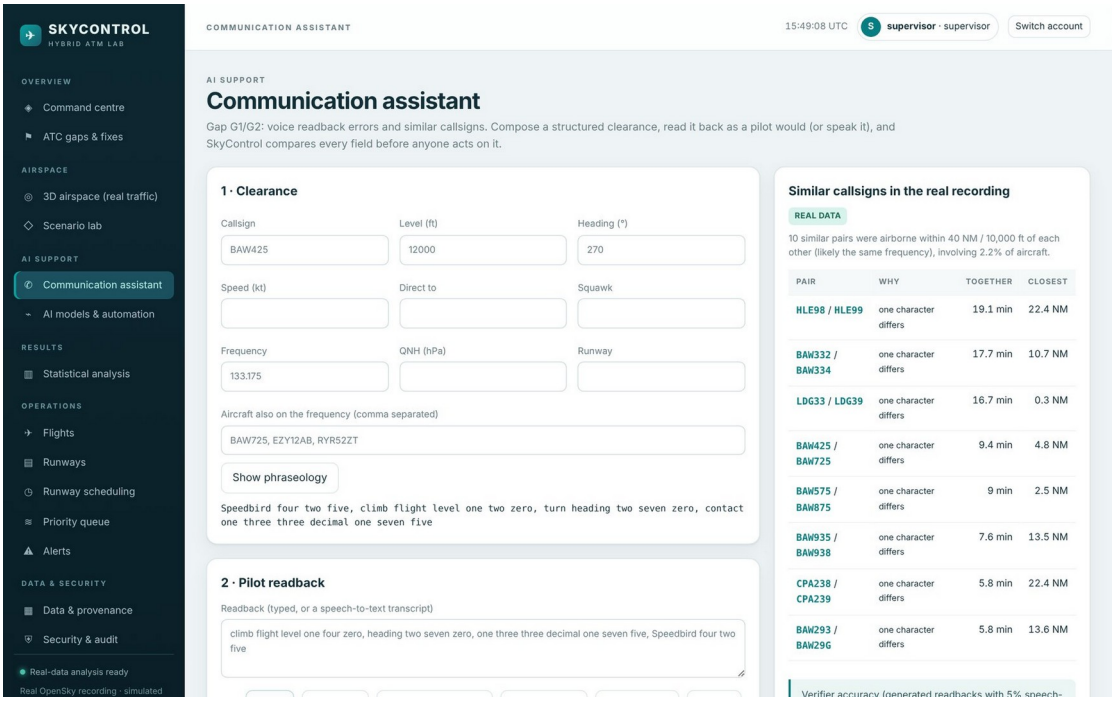


Fig. 7.41 Communication Assistant Verifying a Readback

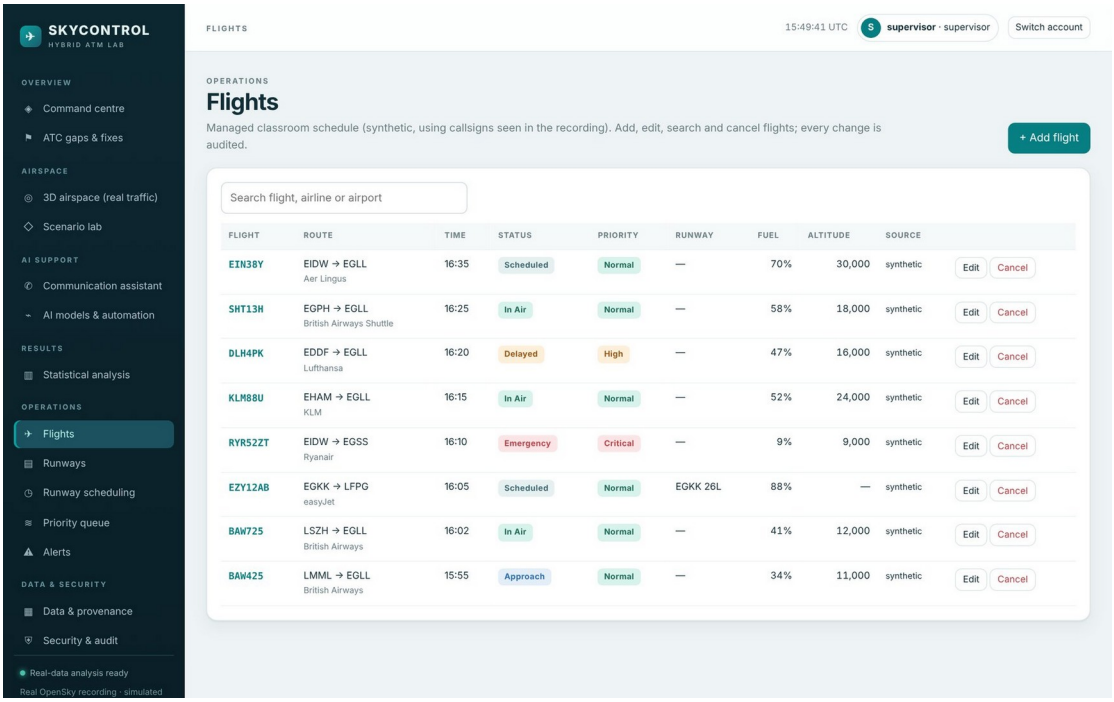


Fig. 7.42 Operations: Flight Management

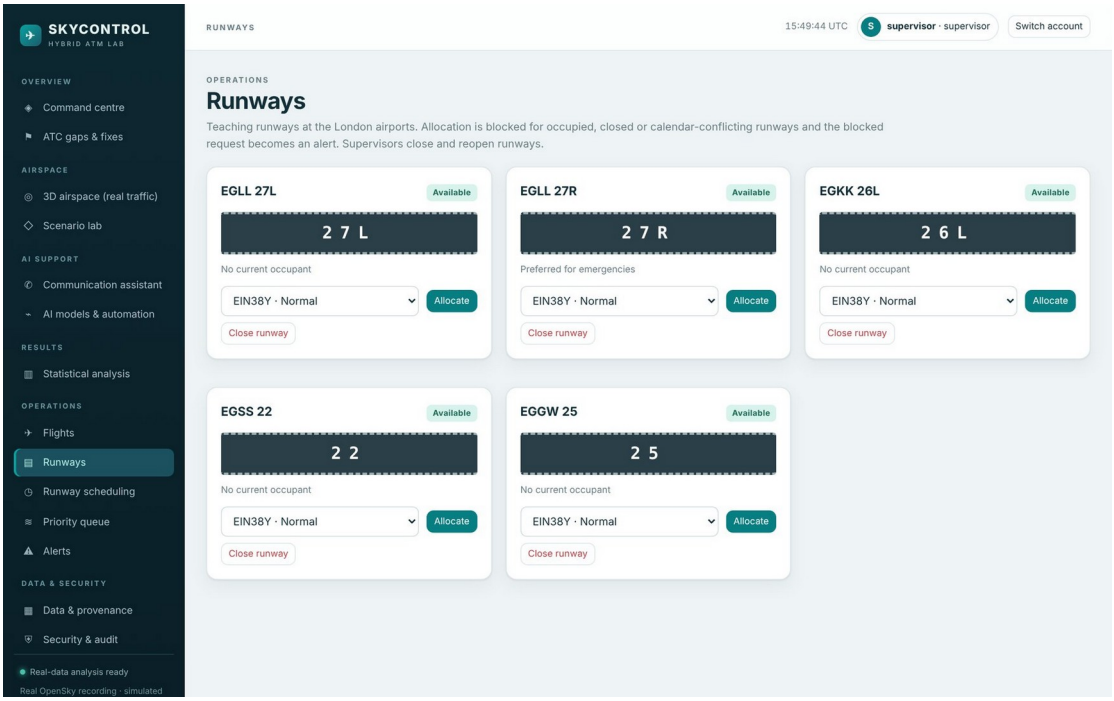


Fig. 7.43 Operations: Runway Management

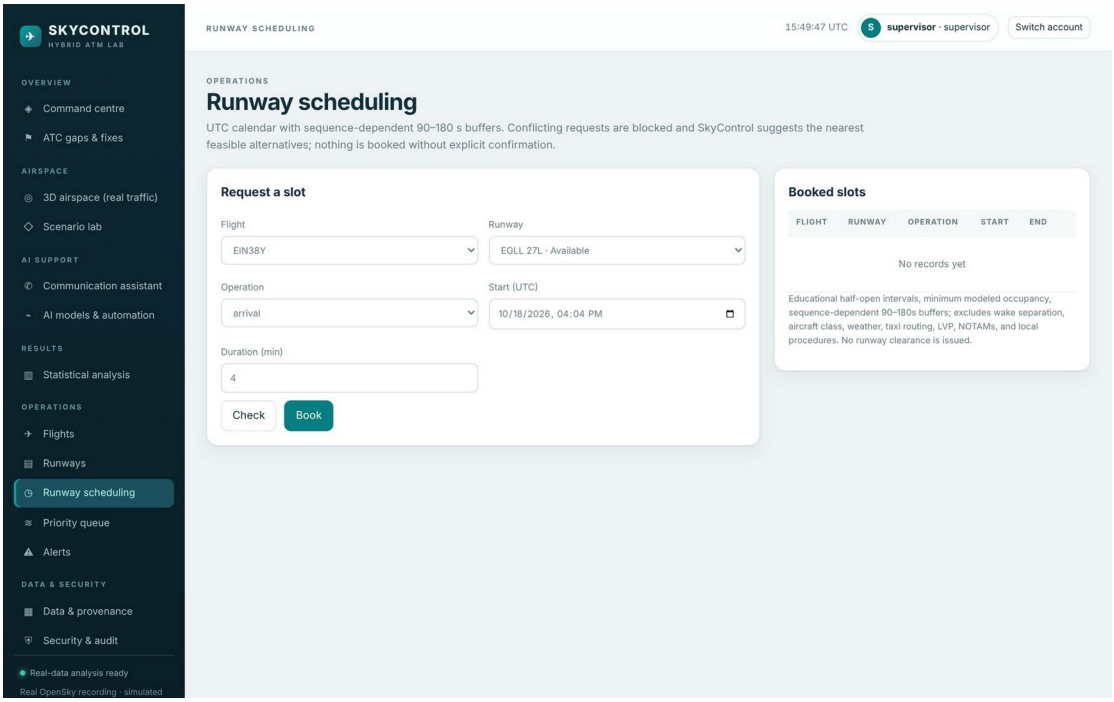


Fig. 7.44 Operations: Runway Scheduling Calendar

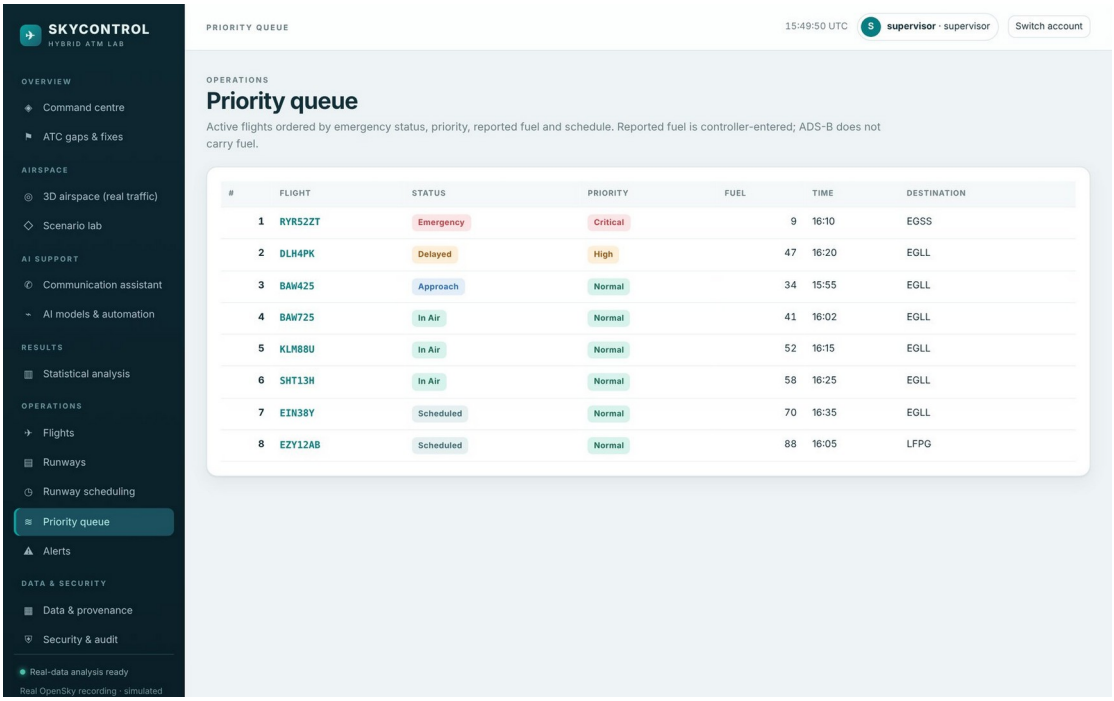


Fig. 7.45 Operations: Priority Queue

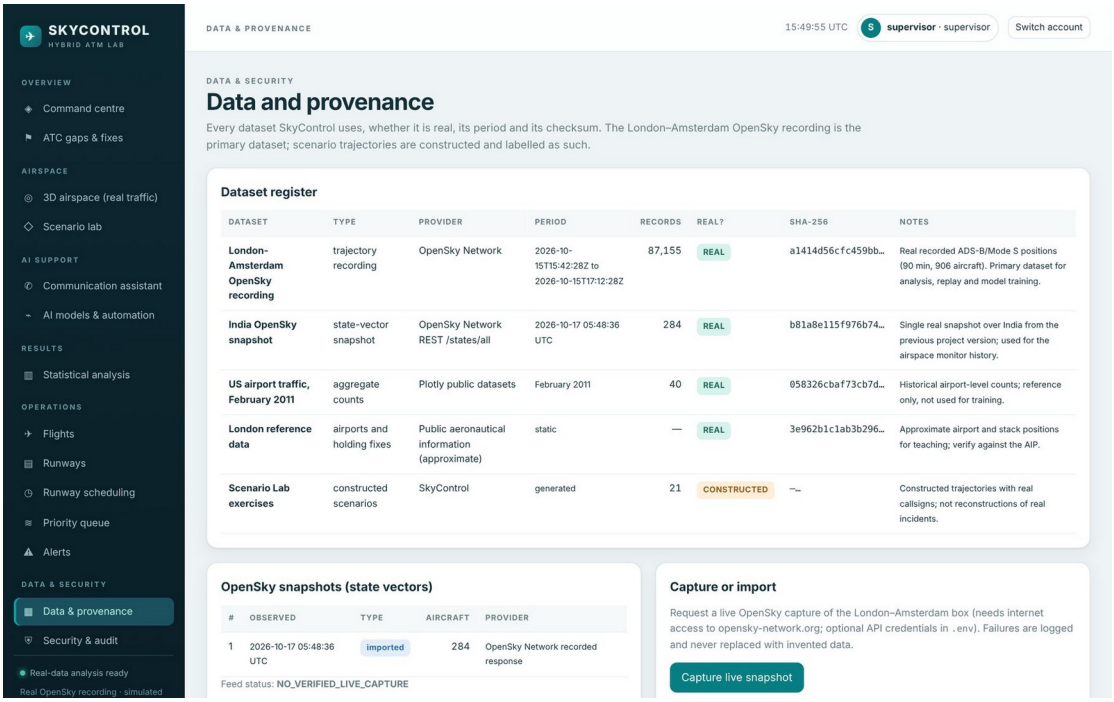


Fig. 7.46 Data and Provenance Page

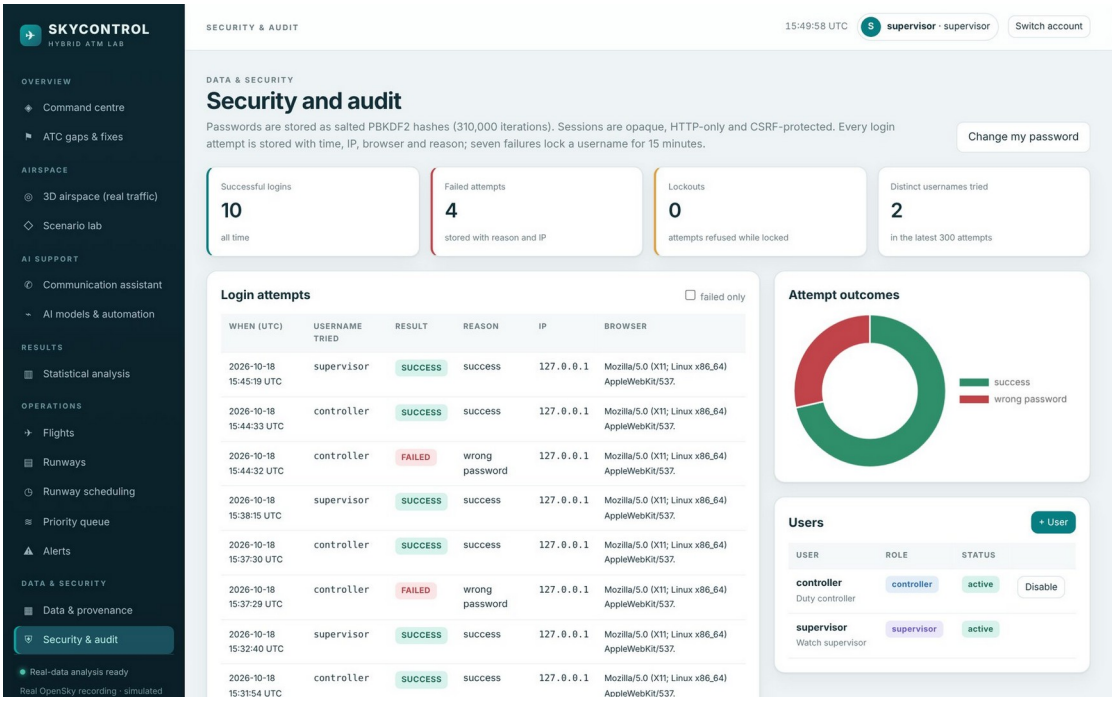


Fig. 7.47 Security and Audit Page

7.15 Cost Analysis

SkyControl is built entirely from open-source software, so there are no licence costs. Table 7.14 estimates the cost of developing and running it. Effort is taken from the sprint plan (205 story points over 12 weeks for a team of two); infrastructure costs are indicative prices for a small cloud virtual machine and are stated as ranges.

Table 7.14 Development and Operating Cost (Indicative)

Item	Basis	Cost
Software licences	Python, FastAPI, scikit-learn, three.js, Chart.js, SQLite, Docker, Jenkins (open source)	₹0
Data	OpenSky Network data for research and teaching	₹0
Planning and CI	Jira Cloud free plan (up to 10 users), GitHub free, Jenkins self-hosted	₹0
Development hardware	Existing student laptops (8 GB RAM)	₹0 additional
Development effort	2 students × 12 weeks × about 15 h/week ≈ 360 person-hours	Academic project (not charged)
Hosting (optional)	Small cloud VM, 2 vCPU / 4 GB, for a shared demonstration	about ₹1,500-2,500 per month
Domain and TLS (optional)	Domain name; free certificate (Let's Encrypt)	about ₹800-1,200 per year

On the benefit side, the measured holding in the 90-minute recording corresponds to about 10,728 kg of fuel, of which about 3,943 kg (12,461 kg of CO_2) could be saved by early speed control in the modelled extended-AMAN case. These figures are for one region and one busy afternoon period; extrapolating them to a day

or a year would multiply the uncertainty and is not attempted here. The communication and emergency benefits are safety benefits, which are not priced.

7.16 Discussion and Threats to Validity

The results support the main claim of the project: on real traffic, AI and well-known algorithms can close specific gaps in current ATC while the human stays in charge, and the hybrid improves reliability (fewer undetected communication errors, more accurate predictions, trustworthy surveillance, all verified plans conflict-free) and efficiency (fewer alerts for a longer look-ahead, faster emergency handling, less frequency time). The context switch turned out to be essential rather than decorative: the emergency example in Section 7.9 and the AI-disagreement scenario both show cases in which acting on the automation alone would have been wrong.

The results also have clear limits, which are stated here so that they are not over-read:

- **One recording.** All measured results come from one 90-minute recording of one region. The learned models must be retrained and re-validated on more days and other airspace before any operational claim.
- **Small samples.** Eight real encounters and 39 held-out arrivals are too few for strong conclusions about warning time and hold prediction; these results are reported as indicative.
- **Approximated intent.** The cleared level used by the intent-aware probe is the next observed level-off, a stand-in for a digital clearance record that makes the probe optimistic.
- **Modelled human performance.** Communication error rates, controller miss rates and task times come from the literature and assumptions, not from controllers; sensitivity analyses bound their effect.
- **Generated corpus.** The readback verifier was evaluated on generated transcripts with simulated recognition noise, not on real R/T audio.
- **No user study.** The interface and the automation policy were not evaluated with air traffic controllers.

Within these limits, the evidence is consistent across very different methods (held-out machine-learning evaluation, injected events in real tracks, Monte Carlo

modelling and exact verification), which gives confidence that the direction of each result is right even where its size is uncertain.

CHAPTER 8
SUMMARY

This project set out to find where current air traffic control is weak and to show, on real traffic, how a hybrid human-AI system can close those gaps without taking authority away from the controller. SkyControl, the resulting system, is a complete and deployable web application built on a real 90-minute OpenSky recording of London-Amsterdam traffic. It has a login portal with verified credentials that rejects wrong credentials clearly and stores every attempt; a 3D replay of the real airspace with live monitors and emergency injection; a scenario lab that handles 21 scenarios in ten visible steps with explicit context switches and approval gates; a communication assistant; and a statistical analysis section. It is tested, containerised and delivered by a Jenkins pipeline, and it was planned and run as six Scrum sprints.

8.1 Findings

- **Prediction and alerting.** A learned residual predictor cut the five-minute position error on held-out real traffic by 34.5%. Combined with cleared-level intent and duplicate merging, it allowed a five-minute conflict probe to run at the alert rate of a two-minute safety net (145.3 against 148.7 alerts per hour, against 522.7 for a naive probe).
- **Surveillance integrity.** Duplicate targets and dropouts are real in today's data (21 ghost pairs, 234 dropouts); dead-reckoned coasting reduced the error of dropped tracks by 76%.
- **Communication.** The readback verifier caught 98% of injected errors under recognition noise with 0.22% false errors; the sector model predicts 91.1% fewer undetected communication errors and 63% less frequency time, with the reduction staying between 65% and 98% across the sensitivity range.
- **Holding.** 268 aircraft-minutes of holding in 90 minutes, about half of which could be absorbed en route with early prediction; the hold predictor gives about 11.5 minutes of warning (indicative).
- **Level busts and emergencies.** Cleared-level monitoring with a digital clearance record detected all 81 injected events; for 60 emergencies in real

traffic SkyControl resolved 91.5% of conflicts in about 90 ms and cut the modelled handling time by 75%.

- **Scenarios and context switching.** All 21 plans were verified conflict-free; every comparator problem was removed, and the automation policy kept humans in charge of every critical plan and switched to MANUAL when the AI and the rule disagreed, when data was degraded or when a plan could not be fully resolved.

8.2 Conclusions

The central conclusion is that AI is most useful in ATC as a *complement* that removes single points of failure around the controller: a second check behind hearback, a longer and quieter look-ahead beside the safety net, an integrity signal in front of the display, an earlier view of holding and an assistant for emergencies. The second conclusion is that the context switch is as important as the AI itself. The cases in which SkyControl stepped back were exactly the cases in which blind automation would have been wrong, which supports the levels-of-automation literature with concrete, testable behaviour.

The evidence is strongest for the measured results on real data (prediction, alert load, coasting, ghosts, holding and detection of injected events) and weaker, though consistent, for the modelled human-performance results and the small-sample results (warning time, hold prediction). The report labels each result accordingly.

8.3 Recommendations and Future Work

1. Retrain and validate the learned models on several weeks of recordings from different regions and seasons, and evaluate warning time on hundreds of real encounters.
2. Replace the approximated cleared levels with real clearance data (for example from a simulator or CPDLC logs) to measure the intent-aware probe without the counterfactual.
3. Evaluate the readback verifier on real R/T recordings with an aviation-trained speech recogniser.
4. Run a human-in-the-loop study with controllers or trained students to measure workload, trust and decision time with and without the context-switch display.

5. Add probabilistic trajectory prediction (uncertainty bands) so that the probe can adapt its thresholds to confidence.
6. Harden the deployment for multi-user use: PostgreSQL, multi-factor authentication and HTTPS by default.

CHAPTER 9

REFERENCES

Ayhan, S., & Samet, H. (2016). Aircraft trajectory prediction made easy with predictive analytics. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 21–30). ACM. https://doi.org/10.1145/2939672.2939694

Bainbridge, L. (1983). Ironies of automation. *Automatica, 19*(6), 775–779. https://doi.org/10.1016/0005-1098(83)90046-8

Cardosi, K. M. (1993). *An analysis of en route controller-pilot voice communications* (Report No. DOT/FAA/RD-93/11). Federal Aviation Administration.

Cardosi, K., Falzarano, P., & Han, S. (1998). *Pilot-controller communication errors: An analysis of Aviation Safety Reporting System (ASRS) reports* (Report No. DOT/FAA/AR-98/17). Federal Aviation Administration.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors, 37*(1), 32–64. https://doi.org/10.1518/001872095779049543

Erzberger, H. (2006). Automated conflict resolution for air traffic control. In *Proceedings of the 25th International Congress of the Aeronautical Sciences (ICAS 2006)*. International Council of the Aeronautical Sciences.

EUROCONTROL. (2004). *Air-ground communication safety study: An analysis of pilot-controller occurrences* (Edition 1.0). EUROCONTROL.

EUROCONTROL. (2007). *EUROCONTROL specification for short term conflict alert* (EUROCONTROL-SPEC-0108, Edition 1.0). EUROCONTROL.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63b

Helmke, H., Ohneiser, O., Mühlhausen, T., & Wies, M. (2016). Reducing controller workload with automatic speech recognition. In *2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*. IEEE. https://doi.org/10.1109/DASC.2016.7778024

Hoekstra, J. M., & Ellerbroek, J. (2016). BlueSky ATC simulator project: An open data and open source approach. In *Proceedings of the 7th International Conference on Research in Air Transportation (ICRAT)*.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

International Civil Aviation Organization. (2007). *Manual of radiotelephony* (Doc 9432, 4th ed.). ICAO.

International Civil Aviation Organization. (2016). *Procedures for air navigation services: Air traffic management* (Doc 4444, 16th ed.). ICAO.

International Civil Aviation Organization. (2018). *Annex 11 to the Convention on International Civil Aviation: Air traffic services* (15th ed.). ICAO.

International Organization for Standardization. (2011). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models* (ISO/IEC 25010:2011).

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates.

Kochenderfer, M. J., Holland, J. E., & Chryssanthacopoulos, J. P. (2012). Next-generation airborne collision avoidance system. *Lincoln Laboratory Journal, 19*(1), 17–33.

Kuchar, J. K., & Yang, L. C. (2000). A review of conflict detection and resolution modeling methods. *IEEE Transactions on Intelligent Transportation Systems, 1*(4), 179–189. https://doi.org/10.1109/6979.898217

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Article 2.

Moriarty, K., Kaliski, B., & Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

Olive, X. (2019). traffic, a toolbox for processing and analysing air traffic data. *Journal of Open Source Software, 4*(39), 1518. https://doi.org/10.21105/joss.01518

Olive, X., Grignard, J., Dubot, T., & Saint-Lot, J. (2018). Detecting controllers' actions in past Mode S data by autoencoder-based anomaly detection. In *Proceedings of the 8th SESAR Innovation Days*.

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

Pang, Y., Zhao, X., Yan, H., & Liu, Y. (2021). Data-driven trajectory prediction with weather uncertainties: A Bayesian deep learning approach. *Transportation Research Part C: Emerging Technologies, 130*, 103326. https://doi.org/10.1016/j.trc.2021.103326

Parasuraman, R., & Manzey, D. H. (2010). Complacency and bias in human use of automation: An attentional integration. *Human Factors, 52*(3), 381–410. https://doi.org/10.1177/0018720810376055

Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans, 30*(3), 286–297. https://doi.org/10.1109/3468.844354

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Prinzo, O. V., & Britton, T. W. (1993). *ATC/pilot voice communications: A survey of the literature* (Report No. DOT/FAA/AM-93/20). Federal Aviation Administration, Office of Aviation Medicine.

Schäfer, M., Strohmeier, M., Lenders, V., Martinovic, I., & Wilhelm, M. (2014). Bringing up OpenSky: A large-scale ADS-B sensor network for research. In *Proceedings of the 13th International Symposium on Information Processing in Sensor Networks (IPSN)* (pp. 83–94). IEEE. https://doi.org/10.1109/IPSN.2014.6846743

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. Scrum.org.

SESAR Joint Undertaking. (2020). *European ATM master plan: Digitalising Europe's aviation infrastructure* (2020 ed.). Publications Office of the European Union.

Strohmeier, M., Lenders, V., & Martinovic, I. (2015). On the security of the automatic dependent surveillance-broadcast protocol. *IEEE Communications Surveys & Tutorials, 17*(2), 1066–1087. https://doi.org/10.1109/COMST.2014.2365951

Sun, J., Hoekstra, J. M., & Ellerbroek, J. (2020). OpenAP: An open-source aircraft performance model for air transportation studies and simulations. *Aerospace, 7*(8), 104. https://doi.org/10.3390/aerospace7080104

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin, 1*(6), 80–83. https://doi.org/10.2307/3001968

Data Sets and Software

OpenSky Network. (2026). *London-Amsterdam state vectors, 15 October 2026, 15:42-17:12 UTC* [Data set, recorded through the OpenSky REST API]. https://opensky-network.org

Cabello, R., & three.js authors. (2023). *three.js* (Version 0.160.0) [Computer software]. https://threejs.org

Chart.js contributors. (2024). *Chart.js* (Version 4.4.4) [Computer software]. https://www.chartjs.org

Ramírez, S. (2026). *FastAPI* (Version 0.144.0) [Computer software]. https://fastapi.tiangolo.com

APPENDIX A
SAMPLE CODE, USER MANUAL AND ADDITIONAL SCREENSHOTS

This appendix contains the faculty demonstration script, the commands to run the project, selected source code and additional screenshots. The complete source code, tests and guides are in the project archive and the GitHub repository.

A.1 User Manual and Demonstration Script

Running the application. On Windows with VS Code, open the SkyControl folder and run:

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe run.py     # choose the two passwords at first start
```

Or with Docker (any operating system):

```
cp .env.example .env                    # then set the two account passwords in .env
docker compose up -d --build            # open http://localhost:5000
```

Ten-minute faculty demonstration:

1. Login portal: type a wrong password; the page shows "Wrong username or password" and the attempts left. Sign in as supervisor.
2. Command centre: headline results from the real recording.
3. ATC gaps and fixes: the nine gaps with today's practice, the fix and the evidence.
4. 3D airspace: press Play at 15×, turn on the 5-minute predictions, watch predicted conflicts, similar callsigns, holds and duplicates; click an aircraft above FL200 and run a medical emergency; approve the plan.
5. Scenario lab: run Readback error through its ten steps; run Fuel emergency as controller (blocked), switch to supervisor and approve; run AI disagreement to show the switch to MANUAL.
6. Communication assistant: choose "Wrong aircraft answers" and Verify.
7. Statistical analysis: Summary, Conflict prediction and Communication tabs.
8. AI models: move the sliders of the context-switch simulator.

9. Security and audit: the attempt log shows the wrong password from step 1.

A.2 Context-Switch Policy (backend/core/automation.py)

```
def decide(severity='Low', confidence=0.95, data_quality=1.0, workload=0.5, disagreement=False,
           security=False, novelty=0.0):
    reasons = []
    if security:
        reasons.append('Security event: decisions stay with the human chain of command.')
        return 'MANUAL', 'supervisor', reasons
    if data_quality < 0.6:
        reasons.append(f'Surveillance integrity {data_quality:.0%} is below 60%: '
                       'automation cannot be trusted.')
        return 'MANUAL', 'controller', reasons
    if disagreement:
        reasons.append('The learned model disagrees with the deterministic safety rule; '
                       'the rule wins and a human decides.')
        return 'MANUAL', 'controller', reasons
    if novelty > 0.8:
        reasons.append('Situation is outside the learned traffic patterns (high anomaly
score).')
        return 'ADVISORY', 'supervisor' if severity == 'Critical' else 'controller', reasons
    if severity == 'Critical':
        if confidence < 0.7:
            reasons.append(f'Critical event with low plan confidence ({confidence:.0%}).')
            return 'MANUAL', 'supervisor', reasons
        reasons.append('Critical event: the plan is verified but a supervisor must authorise
it.')
        return 'ADVISORY', 'supervisor', reasons
    if severity == 'High':
        level = 'CONSENT' if confidence >= 0.85 else 'ADVISORY'
        return level, 'controller', reasons
    if confidence >= 0.9 and data_quality >= 0.9:
        reasons.append('Routine traffic, high confidence and clean data.')
        return 'AUTO', None, reasons
    return 'CONSENT', 'controller', reasons
```

A.3 Exact Conflict Verification (backend/core/separation.py)

```
def interval_conflict(route_a, route_b, h=SEPARATION_NM, v=SEPARATION_FT):
    """First time (s) at which two routes are inside h NM and v ft of each other, else None."""
    for a, b in zip(route_a, route_a[1:]):
        for c, d in zip(route_b, route_b[1:]):
            lo, hi = max(a['t'], c['t']), min(b['t'], d['t'])
            if hi - lo < 1e-7:
                continue
            pa, pb = pos([a, b], lo), pos([c, d], lo)
            rx, ry, rz = pa['x'] - pb['x'], pa['y'] - pb['y'], pa['altitude_ft'] - pb['altitude_ft']
            vx, vy = vel(a, b, 'x') - vel(c, d, 'x'), vel(a, b, 'y') - vel(c, d, 'y')
            vz = vel(a, b, 'altitude_ft') - vel(c, d, 'altitude_ft')
            A, B, C = vx * vx + vy * vy, 2 * (rx * vx + ry * vy), rx * rx + ry * ry - h * h
            left, right = 0.0, hi - lo
            if A < 1e-12:
                if C >= -1e-8:
                    continue
            else:
                disc = B * B - 4 * A * C
                if disc <= 0:
                    continue
                left = max(left, (-B - math.sqrt(disc)) / (2 * A))
                right = min(right, (-B + math.sqrt(disc)) / (2 * A))
            if abs(vz) < 1e-12:
                if abs(rz) >= v - 1e-7:
                    continue
            else:
                z1, z2 = (-v - rz) / vz, (v - rz) / vz
                left, right = max(left, min(z1, z2)), min(right, max(z1, z2))
            if right - left > 1e-6:
                return round(lo + max(0.0, left), 2)
    return None
```

A.4 Container Smoke Test (scripts/smoke_test.sh)

```
CODE=$(curl -s -o /dev/null -w '%{http_code}' -H 'Content-Type: application/json' \
  -d "{\"username\":\"$USER\",\"password\":\"wrong-password-123\"}" "$BASE/api/auth/login")
[ "$CODE" = "401" ]
echo "Wrong credentials rejected: OK ($CODE)"

TOKEN=$(curl -fsS -H 'Content-Type: application/json' \
```

```
  -d "{\"username\":\"$USER\",\"password\":\"$PASS\"}" \
  "$BASE/api/auth/login" | sed -n 's/.*"token":"\([^"]*\)".*/\1/p')
[ -n "$TOKEN" ]
echo "Login with verified credentials: OK"

ATTEMPTS=$(curl -fsS -H "Authorization: Bearer $TOKEN" "$BASE/api/auth/attempts?
failed_only=true")
echo "$ATTEMPTS" | grep -q '"reason":"wrong_password"'
echo "Failed login stored in the backend: OK"
```

A.5 Docker Compose File

```
services:
  skycontrol:
    build: .
    image: skycontrol:latest
    container_name: skycontrol
    ports:
      - "5000:5000"
    env_file: .env          # copy .env.example to .env and choose the two passwords
first
    volumes:
      - skycontrol-db:/app/data/db
    restart: unless-stopped

volumes:
  skycontrol-db:
```

A.6 Additional Screenshots

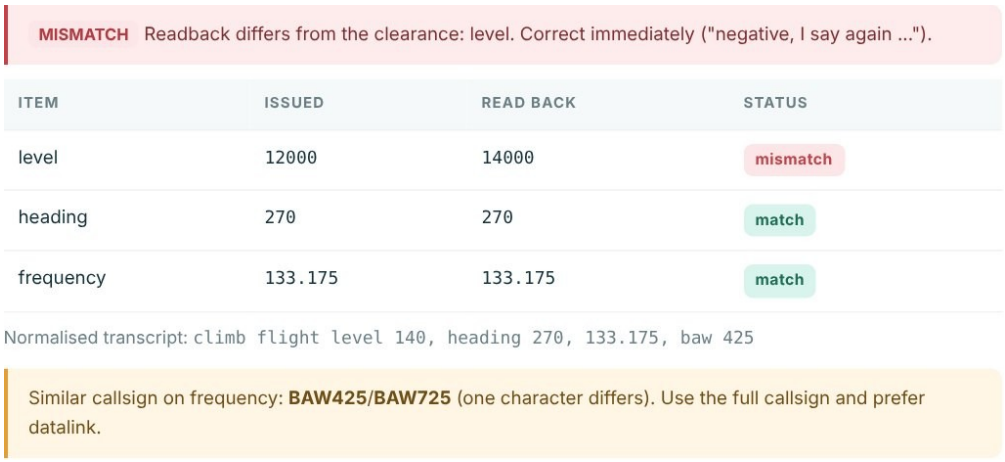


Fig. A.1 Readback Verification Result in the Communication Assistant

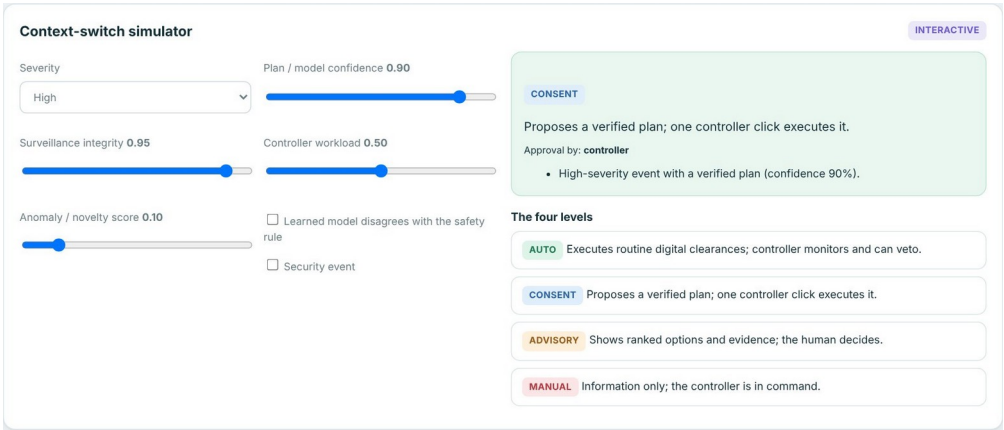


Fig. A.2 Context-Switch Simulator on the AI Models Page

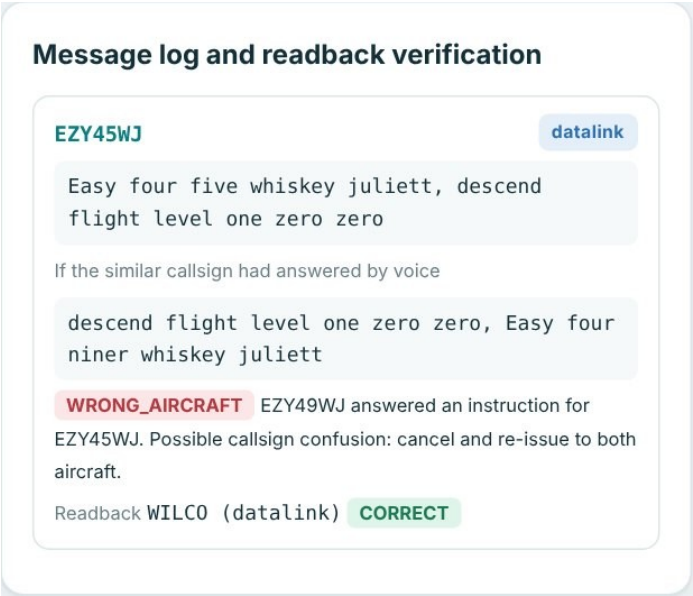


Fig. A.3 Callsign Confusion Scenario: Wrong Aircraft Detected

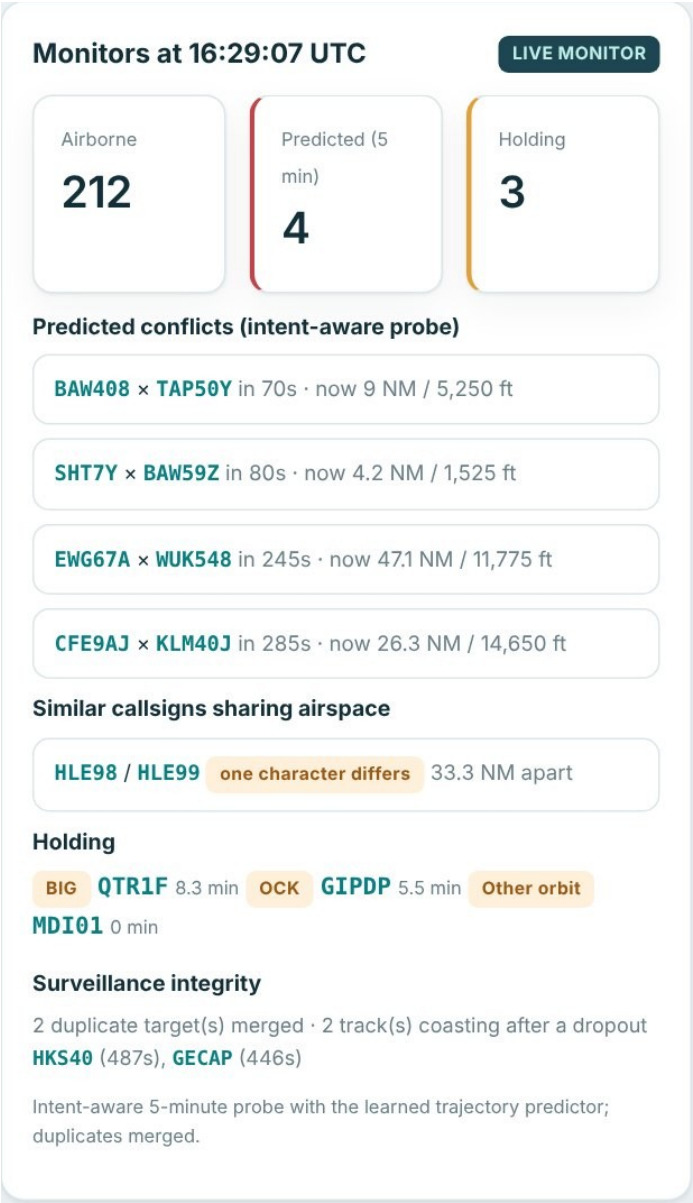


Fig. A.4 Live Monitor of the 3D Airspace at One Moment of the Recording

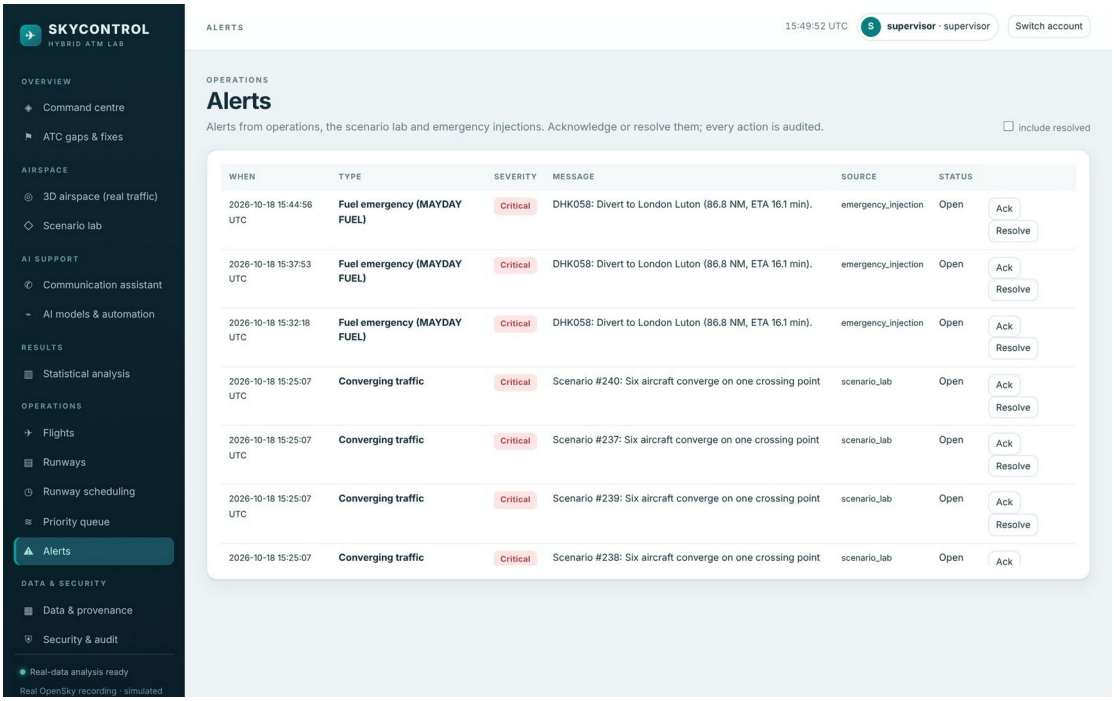


Fig. A.5 Alert Centre

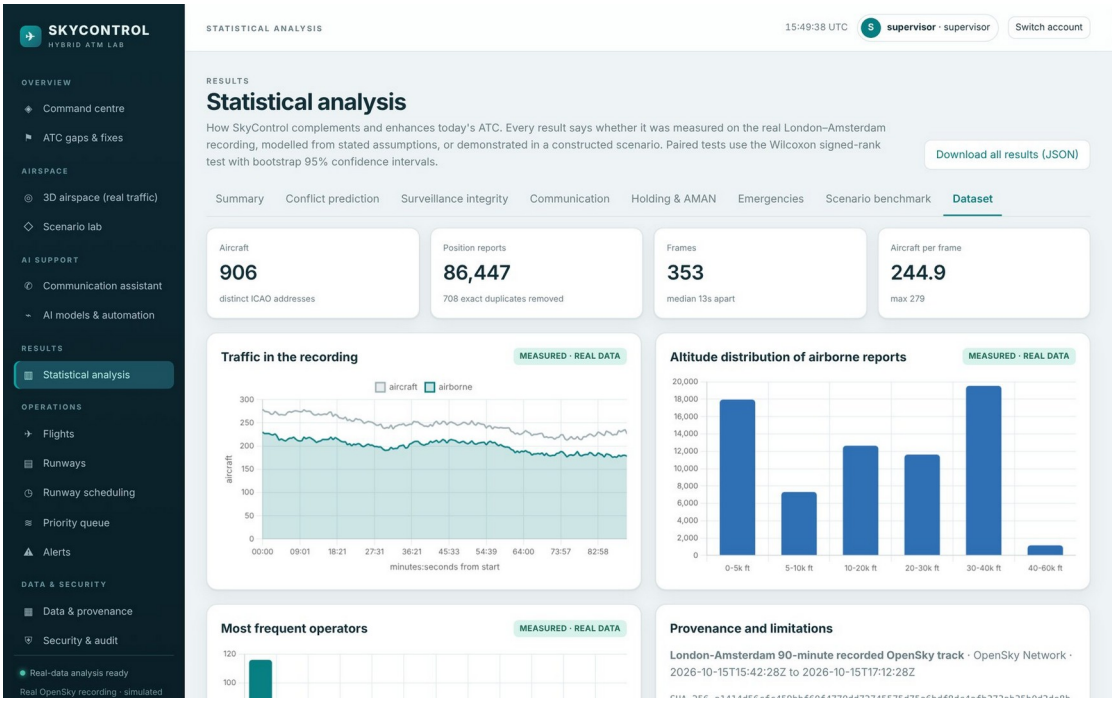


Fig. A.6 Statistical Analysis: Dataset Tab